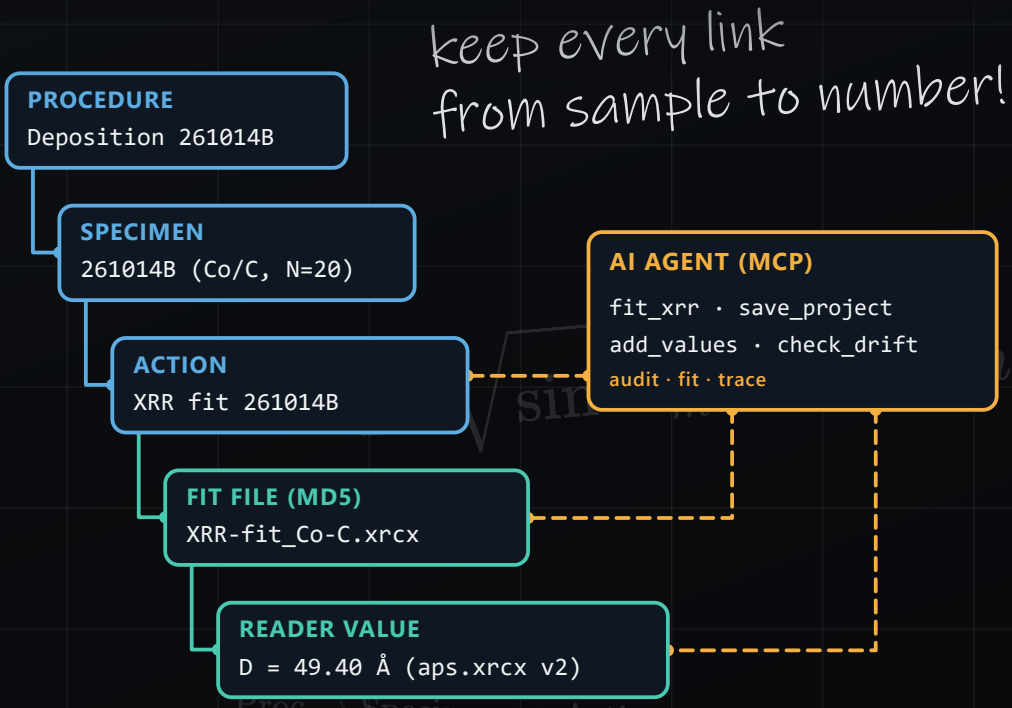
MASTERING ELN

Recording, Tracing, and Analyzing Laboratory Data with APS ELN and AI Agents

Oleksiy V. Penkov



© 2026 Oleksiy V. Penkov
CC BY 4.0

MASTERING ELN

Recording, Tracing, and Analyzing Laboratory Data with APS ELN and AI Agents

Oleksiy V. Penkov

ZJU-UIUC Institute, International Campus,
Zhejiang University

Edition 1.1.0, 2026
DOI: 10.5281/zenodo.23466959
Web edition: https://oleksiypenkov.github.io/MasteringELN/

© 2026 Oleksiy V. Penkov. ORCID: 0000-0003-3675-5301
Text, figures and data are licensed under CC BY 4.0; code under MIT.

Cite as: Penkov, O. V. (2026). Mastering ELN: Recording, Tracing, and Analyzing Laboratory Data with APS ELN and AI Agents (Edition 1.1.0). Zenodo. https://doi.org/10.5281/zenodo.23466959

Contents

FRONT MATTER

Introduction 1

About This Book 1
Foolish Assumptions 3
Icons Used in This Book 3
Beyond the Book 5
Where to Go from Here 5

PART 1: GETTING STARTED

1 What an ELN Is, and Why Links Matter 7

The four links in every laboratory result 7
Why the fourth link is the one that breaks 8
Two running examples: Study A and Study B 10
Four rules that keep your data trail unbroken 11
The whole student workflow on one page 12

2 Opening a Local or Team Lab Notebook 16

Two ways ELN stores your laboratory work 16
Creating a local notebook in Community 17
Logging into a shared team server 20
How to back up a local notebook safely 21

3 A Tour of the Main Window 24

The three-panel workspace 24
Reading the item tree and its row buttons 25
The Dashboard and keyboard navigation 28
Setting preferences and plugin paths 29

4 Your First Sample Record in Twenty Minutes 31

Meet Study A: sample 261015A 31
Step 1: Build the whole item tree first 33
Step 2: Attach the 11 raw files to their actions 34
Step 3: Extract Reader values from the files 35
Step 4: Compute the Derived wear rate 37

PART 2: RECORDING DAILY LAB WORK

5 Projects, Groups, and Folders 40

How Groups, Projects, and Folders fit together 40
Creating, sorting, and moving projects and folders 42

Archiving finished projects 43
Sharing projects in a team laboratory 44
Deleting containers and using the Recycle Bin 44

6 The Item Tree and Experiment Templates 47

Designing a clean item hierarchy 47
Creating, duplicating, and moving items safely 49
Working on multiple records at once 50
Building and applying Experiment Templates 52

7 Raw Files, Drafts, and Instrument Viewers 55

How file attachment and extension routing work 55
Inside the five core instrument viewers 56
Working drafts, external edits, and conflict resolution 64
Replacing, moving, exporting, and viewing files outside a notebook 66

8 Filling Out Protocols and Checklists 68

Adding and completing a protocol on an item 68
How protocol templates and AutoFill work 69
The protocol review workflow 71
Submitting protocols from the instrument in a team lab 72

9 Values, Units, Formulas, and Provenance 74

Conditions, Results, units, and plausible-range checks 74
Reader mappings, unit factors, and standard deviations 76
The four value sources and Derived formulas 77
Checking value sources and understanding drift codes 80

PART 3: EXPERIMENT PLANNING WITH ELN

10 Principles of Experiment Planning and the Taguchi Method 83

Why unplanned experiments fail in the laboratory 83
Factors, levels, fixed conditions, and responses 84
Sweeps, full grids, and Taguchi orthogonal arrays 85
Reading main effects: level means, Delta, and factor rank 90

11 The Hypothesis Tree and Evidence 93

Why connect experiments to a hypothesis tree? 93
Proposing a hypothesis and starting or approving work 94
Attaching Runs, promoting References, and citing Evidence 96
Concluding, closing or accepting, and checking the content seal 99

12 Experiment Series and Taguchi Arrays in ELN 103

Planning a series in the Series window 104
Making or linking depositions and confirming planned values 106

Reading the Series Table and setting column roles 108
Plotting sweeps, Taguchi level means, and exporting to OriginLab 109

PART 4: ORGANIZING, SEARCHING, AND REVIEWING

13 Project Wiki, Notes, Tags, and Links 112
The five-section Project Wiki 112
Navigating between Wiki pages and experimental records 114
Classifying items with Tags and Favorites 115
Recording Comments and typed item relationships 116

14 Full-Text, Panel, and Semantic Search 119
Three ways to find a record in APS ELN 119
Filtering panels and jumping to a known ID 120
Searching notebook text (Ctrl+F) 121
Searching by meaning (Semantic Search) and reranking 122

15 Cross-Project Charts, PDF Reports, and Packages 127
Plotting conditions against results 127
Exporting a formatted PDF report 129
Exporting and importing scientific packages 130

16 Completeness, Audit, and Signatures 133
Three layers of record integrity 133
Completeness rules and Data Quality Checks 134
Inspecting the Audit Log 135
Signing an item and verifying signature integrity 136

PART 5: WORKING WITH AN AI AGENT (MCP)

17 Connecting an AI Agent to Your Notebook 139
What the ELN MCP server does 139
Preparing a local session in APS ELN Community 141
Registering with DSH or OMP in a team laboratory 143
Security, upload-root, and review boundaries 144

18 Reading Data and Auditing Drift via MCP 147
Navigating and querying values 147
Searching by text and meaning (search and search_semantic) 149
Reading tabular instrument logs (list_file_columns and get_file_columns) 150
Running the three project-wide health checks 152

19 Study B: XRR Fitting with Provenance 156
Why agent-written records need the Value Route 156
Part 1: The Record Agent builds Study B (261014B) 157
Part 2: Fitting Agent records XRR fit 261014B 159

20 Drafting Trees, Hypotheses, and Wikis 165
Passing viewer selections and Derived inputs 165
Updating and removing values and files safely 167
Drafting and updating Wiki pages 169
Drafting experiment series and preparing hypotheses 170
21 Importing Legacy Folders with an AI Agent 176
Why legacy folders break direct import 176
Phases 1 and 2: Reconnaissance and the migration plan 180
Phase 3: Staging files and separating Reader from Manual provenance 182
Phase 4: Linking an Experiment Series, filing compilations, and auditing 186

PART 6: THE PART OF TENS

22 Ten Mistakes That Break Your Data Trail 190
1. Leaving an action empty 190
2. Attaching a screenshot instead of raw data 190
3. Putting result numbers in a file name 191
4. Typing a number that a reader can extract 191
5. Typing a value before attaching its file 191
6. Hiding results inside prose notes 192
7. Putting a fit under the wrong sample 192
8. Mixing angstroms, nanometers, and meters 192
9. Renaming storage files in Windows Explorer 193
10. Backing up the database without its files 193
23 Ten Checks Before You Sign or Submit 195
1. Every action has its raw files attached 195
2. Every file opens in its built-in viewer 195
3. Shared regions and intervals are applied 196
4. Headline values carry Reader or Derived sources 196
5. Repeated measurements show mean and SD 196
6. Sample codes match across items and files 197
7. Required protocol forms are completed 197
8. Data Completeness reports zero missing items 197
9. Check Value Sources reports zero drift 198
10. Hypothesis runs and citations are current 198
24 Ten Things an Agent Can and Cannot Do 200
1. Can: Build item trees and attach raw files 200
2. Can: Extract Reader values with snapshots 200
3. Can: Compute Derived values from formulas 201
4. Can: Audit storage, ranges, and drift 201
5. Can: Stage curves to external fitting tools 201

6. Cannot: Approve or accept a hypothesis 202
7. Cannot: Sign an item or edit signed data 202
8. Cannot: Read or write outside its scope 202
9. Cannot: Access files outside upload-root 202
10. Cannot: Judge physical sample quality 203

APPENDICES

A Toolbar, Menu, and Tree Icons Reference 204
Projects and Items panel toolbars 204
Item type icons in the tree 206
Tree row buttons and status badges 207
Main menu and administration icons 208

B Instrument Viewers and File Formats 211
General document, image, and data viewers 211
Deposition, recipe, and process log viewers 213
Characterization, tribology, and simulation viewers 214
Extracted keys and shared viewer selections 216

C Value Sources, Units, and Formulas 218
Value source kinds and provenance fields 218
Standard quantities, units, and conversions 220
Derived value formulas 221
Plausible-range and drift diagnostic codes 222

D Keyboard Shortcuts 225
Global and main window shortcuts 225
Projects panel and Items panel shortcuts 226
Markdown Editor shortcuts 228
Help window and ELN File Viewer shortcuts 228

E Glossary 230
A 230
C 231
D 232
E 232
F 232
G 233
H 233
M 233
P 234
R 234
S 235

T 235
U 236
V 236
W 236

Introduction

About This Book

This book shows you how to record, trace, and analyze laboratory work in APS ELN. You start with a sample you are about to make or measure. By the end, you have a complete experimental tree, the raw instrument files with verified checksums, and the extracted and derived numbers with their exact origins. A teammate, your supervisor, or your future self can verify any result in two clicks.

Every procedure in the book is something you do in the program: a menu command, a dialog field, a viewer button, or a tool call through the Model Context Protocol (MCP) server. You learn by building two real laboratory records from start to finish:

- **Study A (`261015A`)** is a boron nitride and diamond-like carbon (BN/DLC) coating grown by magnetron sputtering. You test it for step height, nanoindentation hardness and modulus, reciprocating friction, and wear rate.
- **Study B (`261014B`)** is a 20-period cobalt/carbon (Co/C) multilayer X-ray mirror measured by X-ray reflectivity (XRR) and fitted in X-Ray Calc 3.

Both studies come from our thin-film and tribology laboratory (`arXiv:2610.27859`). In daily work, students usually attach the raw file under the right sample, and then break the link between that raw file and the final number. This book teaches the workflow that keeps every link intact.

APS ELN comes in two editions that use the same data model and the same 26 instrument viewers. **APS ELN Community** is the free single-user edition that stores a local SQLite database on your computer. **APS ELN (“For Teams”)** is the multi-user laboratory edition connected to a shared server. This book teaches the shared workflow once and marks team-server steps

with a dedicated icon. Server installation and IT setup belong in administrator documentation.

The book has six Parts:

- **Part 1, Getting Started**, explains why links between files and numbers matter and shows how to open a local or team notebook. It also tours the main window and walks you through recording Study A (`261015A`) in twenty minutes.
- **Part 2, Recording Daily Lab Work**, covers projects, folders, the **Procedure -> Specimen -> Action** hierarchy, and experiment templates. It also shows you how to use the 26 instrument viewers, working drafts, protocols, and value extraction (`Reader`, `Derived`, and `Manual`).
- **Part 3, Experiment Planning with ELN**, explains why and how to plan experiments before going to the instruments. It covers one-factor sweeps, full factorial grids, **Taguchi orthogonal arrays** (`L4` through `L27`), the standalone **Hypotheses** window, and the standalone **Experiment Series** window (**Level means (Taguchi)** with Δ and factor rank).
- **Part 4, Organizing, Searching, and Reviewing**, covers the five-section project wiki, tags and links, full-text and semantic search, cross-project correlation charts, PDF reports, completeness rules, and electronic signatures.
- **Part 5, Working with an AI Agent (MCP)**, shows how to connect DeepSeek Harness (DSH) or OMP to your notebook and audit storage and value drift. It also covers the Study B (`261014B`) XRR fitting workflow, migrating legacy folders from disk, and the human review gates that remain in the Desktop GUI.
- **Part 6, The Part of Tens**, gives three short checklists for avoiding broken links, preparing a record for supervisor approval, and working safely with an AI agent.

Conventions used in this book

- **Menu commands are in bold**, with › between the levels. **Item > New Procedure** means: open the **Item** menu and choose **New**

Procedure. Buttons, tab names, dialogs, and field labels are also in bold, spelled as the program shows them.

- **File names, extensions, reader IDs, and MCP tool names** appear in code font: `agent_tribo.sqlite`, `.depo.json`, `.fibps`, `.mit`, `.zrw`, `.xrdml`, `.xrcx`, `aps.mit`, `extract_file_key_values`.
- **Quantities and units** match the value types stored in the notebook: meters (m), nanometers (nm), or ångströms ($\mathrm{Å}$) for length, and gigapascals (GPa) for hardness and modulus. Normal force is stored in newtons (N), and grazing (θ) and scattering (2θ) angles are stored in degrees ($^\circ$).

Each chapter works on its own. When a chapter relies on an earlier one, it gives a direct link to the exact section. The web edition and the print PDF edition are built from the same source text.

Foolish Assumptions

This book makes a few assumptions about you:

- You run laboratory syntheses, depositions, measurements, or simulations, or you are about to start.
- You know what your instruments measure, even if this is your first week using an electronic laboratory notebook.
- You have a computer running 64-bit Windows 10 or Windows 11 with the Microsoft Edge WebView2 Runtime installed.
- You want to spend your time doing science rather than hunting for lost files when you write your thesis or paper.

You do not need to know SQL, database administration, or Delphi programming. Even in Part 5, where an AI assistant talks to the notebook through MCP, you give instructions in plain English.

Icons Used in This Book

Six margin icons appear throughout the book. Each icon has one job.

Tip

A keyboard shortcut or a faster way to complete a step in APS ELN.

Remember

A rule you'll use across many chapters to keep your records clean and searchable.

Warning

A mistake that breaks your data trail, overwrites a draft, or stores an untraceable number without showing an error message.

Going Deeper

How the data model, file hashing, derived-value formulas, or semantic search work internally. You can skip this box and still complete every step. On the web, click its title to open it.

In ELN

The exact menu path, toolbar button, dialog, and field for an action in the Desktop window.

In a Team Lab

How a step works when you use the Commercial ("For Teams") edition connected to a shared laboratory server. It covers login, project permissions, mobile QR protocols, and supervisor review gates.

Beyond the Book

APS ELN includes a built-in manual that covers every menu command and reference dialog.

> **In ELN**
>
> **Help > Contents...** opens the built-in Help manual. **Help > Keyboard Shortcuts...** (shortcut `F1` in Community) opens the shortcut reference table.

The raw instrument files for both running examples (**Study A**, `261015A`, and **Study B**, `261014B`) ship with this book as downloadable `.zip` archives. You can use them to complete both practice records in your own account:

In the web edition of this book (`https://oleksiypenkov.github.io/MasteringELN/`), you can download `Study-A-261015A-raw.zip` (11 raw files), `Study-B-261014B-raw.zip` (7 raw files, without the pre-fitted `.xrcx`), and `ELN-Cheat-Sheet.pdf`. The full demonstration records are also deposited on Zenodo (`doi:10.5281/zenodo.23466959`).

The source code for **APS ELN Community** is at github.com/APS-Techno/ELN-Community. If you use APS ELN in published research, please cite the paper listed in the Sources section.

Where to Go from Here

Pick the route that matches what you need today:

- **You are opening ELN for the first time:** start with Chapter 1: What an ELN Is, and Why Links Matter. Then read Chapter 2: Opening a Local or Team Lab Notebook and Chapter 3: A Tour of the Main Window.

- **You have a sample to record right now:** go straight to Chapter 4: Your First Sample Record in Twenty Minutes. It records every item, file, and value of Study A (`261015A`) step by step.
- **You want to see the whole workflow on one page:** turn to Chapter 1: The whole student workflow on one page (`Table 1-1`).
- **You are setting up your notebook or checking backups:** read Chapter 2: How to back up a local notebook safely.

Sources

- **The ELN provenance study and demonstration deposit:** *Value-level data provenance in an electronic laboratory notebook used by humans and language-model agents: An experience report*, arXiv preprint arXiv:2610.27859, and software/data deposit doi:10.5281/zenodo.23466959 (`zenodo/demonstration/` provides Study A `261015A` and Study B `261014B`).
- **APS ELN Community Help:** `GettingStarted.html` (system requirements, creating a notebook, main window panels); `Shortcuts.html` (`F1` for **Help > Keyboard Shortcuts...** and **Help > Contents...**); `FilesAndNotes.html` (the 26 viewer plugins).
- **APS ELN Commercial ("For Teams") Help:** `GettingStarted.html` (64-bit Windows 10/11, WebView2 runtime, SQLite vs. remote MySQL/WebDAV connection modes, and team login).
- **APS ELN Community repository:** `README.md` and `CITATION.cff` (github.com/APS-Techno/ELN-Community, Apache-2.0 license).

Cross-references: Chapter 1; Chapter 2; Chapter 3; Chapter 4.

What an ELN Is, and Why Links Matter

IN THIS CHAPTER

- Tracing the four links from a physical sample to a published number
- Seeing how a notebook can keep every file and still lose the link to its numbers
- Following the four rules that keep your data trail unbroken
- Viewing the complete student workflow on one page

When you put a number into a thesis table or a journal figure, someone will eventually ask where it came from. Perhaps your supervisor asks why one coating wore through twice as fast as another. Perhaps a reviewer asks how many indents were averaged in your hardness value, or which period thickness came out of your X-ray reflectivity fit. An electronic laboratory notebook (ELN) does its job when you can answer that question in seconds without guessing.

The four links in every laboratory result

Every experimental number depends on four linked records (Figure 1-1):

1. **The Sample (`Specimen`):** the physical piece or model you made, identified by its sample code (such as `261015A`).
2. **The Operation (`Procedure` or `Action`):** what you did to that sample, such as a sputter deposition, a 3D profilometer scan, a nanoindentation series, or a friction test.
3. **The Raw File:** the unaltered file written by the instrument (`.depo.json`, `.fibps`, `.mit`, `.zrw`, `.xrdml`) or by an analysis program (`.xrcx`).
4. **The Stored Number (`Value`):** the quantity, numerical value, unit, and tolerance saved on the procedure or action (for example, an

indentation hardness of $5.92 \pm 0.47\ \mathrm{GPa}$).

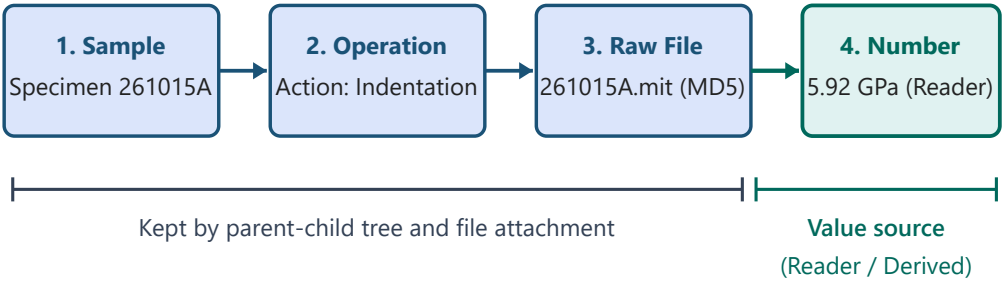


Figure 1-1: The four-link provenance chain for a laboratory measurement. Placing an Action under its Specimen and attaching the raw file keeps the first three links. Extracting the number through a data file reader (Reader source) or a server formula (Derived source) keeps the fourth link.

In APS ELN, the first three links are structural. You create an **Action** under a **Specimen** and attach the instrument file to that action. The tree then records which sample was measured, which operation ran, and where the raw bytes are stored.

Why the fourth link is the one that breaks

Most students keep the first three links without even trying. You create a folder for your sample, drop your measurement files inside it, and move on. Weeks later, when you write a report or prepare a presentation, you open those files, calculate an average or run a fit, and type the final number into a table.

Six months later, that fourth link is gone:

- **A number was typed by hand (`Manual` source) with no link to its file.** Nobody can tell which `.mit` or `.fibps` file produced it, which

outliers were excluded in the viewer, or whether a unit conversion was done in someone's head.
- **A screenshot was saved instead of the raw data.** A `.png` of a fit window looks convincing on screen, but no reader plugin can extract numbers from it or re-verify the fit.
- **An evaluation window or reader changed.** Perhaps you changed the cycle range on a friction curve or updated a reader plugin. The stored number still shows the old value, and nothing warns you unless the notebook recorded the file hash and reader version when the number was extracted.

> ⚠ **Warning: typing numbers by hand**
>
> When you type a result by hand (`Manual` source) instead of extracting it from the attached file (`Reader` source), APS ELN cannot check that number later. If a reader version, calibration, or evaluation window changes, a `Manual` number stays unverified. Use **Extract Values** whenever a reader supports your file format.

> **Going Deeper: how APS ELN stores value-level provenance**
>
> Starting in APS ELN 2.1 (notebook format 5 in Community), every stored value carries a structured source record:
>
> - **`Reader`:** records the `file_id`, the file's MD5 hash at extraction time, the `reader_id` (such as `aps.mit` or `aps.xrcx`), and the integer `reader_version`. It also stores the emitted key name, the emitted unit, and the conversion factor, plus an optional analysis snapshot (such as ticked `.mit` indents or a `.zrw` cycle interval).
> - **`Derived`:** records the `formula_id` (such as `wear_rate_reciprocating`), the `formula_version`, and the exact `value_id` list of the stored input values used in the calculation.
> - **`Manual`:** records that a person typed or edited the number. If you edit the number or tolerance of a `Reader` or `Derived` value by hand, its source switches to `Manual` while preserving the previous reader or formula note.

> - **`Unknown`:** marks legacy values created before format 5 source tracking existed.

Two running examples: Study A and Study B

- **Study A (`261015A`, a BN/DLC tribological coating):** A sputter deposition (`Deposition 261015A`) produces specimen `261015A`. You measure it in three actions directly under the specimen: `Thickness 261015A` (`1.fibps–3.fibps`), `Indentation 261015A` (`BN-DLC(261015A).mit`), and `Friction test 261015A` (`BN-DLC.zrw`). A child action under the friction test, `Wear track 261015A`, holds three optical profiler scans across the wear scar (`wear 1.fibps–wear 3.fibps`). The finished record has 11 raw files, 23 `Reader` values, and 1 `Derived` wear-rate value.
- **Study B (`261014B`, a Co/C multilayer X-ray mirror):** A sputter deposition (`Deposition 261014B`) includes a plasma cleaning action (`Cleaning 261014B`) and produces specimen `261014B` (20 periods of cobalt and carbon on soda-lime glass). Under the specimen sit the X-ray reflectivity scan `XRR-1 261014B` (`XRR-1_Co-C(261014B).xrdml`) and its child analysis action `XRR fit 261014B` (`XRR-fit_Co-C(261014B).xrcx`). The finished record has 8 files and 22 `Reader` values.

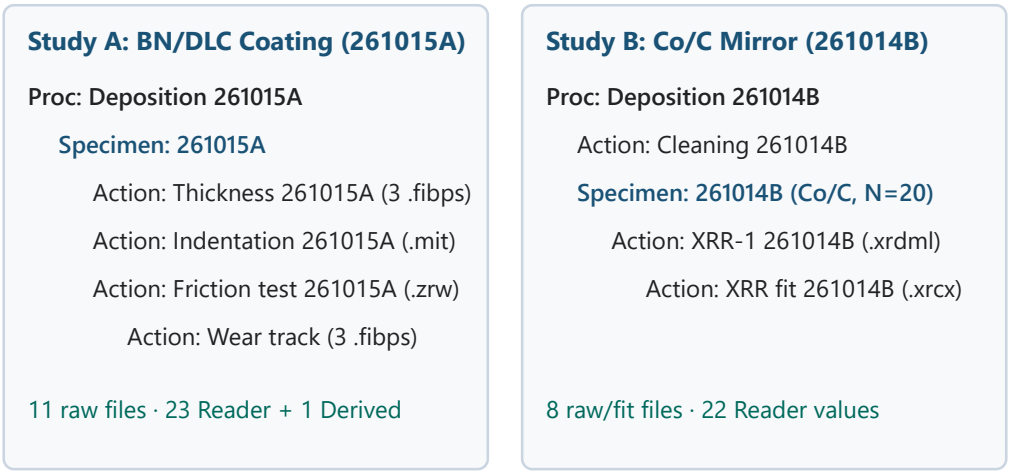


Figure 1-2: The item trees of the two running examples used across this book. Study A (left) records a BN/DLC coating and its mechanical and tribological tests. Study B (right) records a 20-period Co/C X-ray mirror from deposition through XRR fitting.

Chapter 4: Your First Sample Record in Twenty Minutes builds Study A step by step in the Desktop window. Chapter 19: Study B: XRR Fitting with Provenance builds and fits Study B with the MCP server and X-Ray Calc 3.

Four rules that keep your data trail unbroken

Four habits prevent these broken links in daily work:

1. **Build the whole item tree at the start of the run.** Create the **Procedure**, the **Specimen**, and every planned **Action** before you walk to the instruments. An empty action in the tree reminds you that a file is still missing. At the end of the experiment, no action should remain empty.
2. **Attach the raw instrument file, never a screenshot.** A `.png` screenshot of an XRR fit or an indentation curve cannot be re-analyzed or checked for drift. It also gives zero values to a reader. Always attach the native file (`.depo.json`, `.tsv`, `.fibps`, `.mit`, `.zrw`, `.xrdml`, `.xrcx`).

3. **Keep result numbers out of file names.** Name a file after the sample code, the measurement type, and the position or counterbody, such as `BN-DLC(261015A).mit` or `wear 1.fibps`. Never name a file `351.9nm.fibps`. Results belong in stored **Values**, where they carry units, tolerances, and provenance.
4. **Extract values from files instead of typing them.** Click **Extract Values** in the viewer so every number carries its `Reader` source. For multiple files, right-click the files and choose **Extract values from N files...**. Use **Add > Derived Value...** for formula outputs.

> **Remember**
>
> A file name tells you *which* sample and *which* spot was measured. A stored **Value** tells you *what number* the measurement gave.

> **In ELN**
>
> To check where any stored number came from, open the **Procedure** or **Action** with **Item > Edit**, select the **Values** tab, and click the value row. The line below the list names the reader, its version, the key name, and the source file, and enables **Go to file**.

The whole student workflow on one page

Table 1-1 is a one-page cheat sheet grouped by when you do each step in the lab.

Table 1-1: Student workflow cheat sheet for recording an experiment in APS ELN.

WHEN & STEP	MENU / SHORTCUT (OR MCP TOOL)	RULE TO CHECK	CH.
Before the run: 1. Open notebook	**File > New Notebook...** / **Open Notebook...** or Team login	Keep `.sqlite` and `<name> files` folder together.	2
Before the run: 2. Pick Group, Project & Folder	**Project > New Group**, **New Project**, and **New Folder** (`Ctrl+Shift+D`) in left **Projects** panel	Organize projects under a **Group** (such as `Practice`) and related runs in one specimen **Folder**.	3, 5
Before the run: 3. Build full tree	**Item > New Procedure** -> **New Specimen** -> **New Action** (or **New from Template...**)	Create every planned action first; leave no action empty at the end of the day.	4, 6
At the instrument: 4. Attach raw files	**Item > Add File** (`Ctrl+Shift+F` in Team) or `upload_file`	Attach native raw files (`.depo.json` + `.tsv`, `.fibps`, `.mit`, `.zrw`, `.xrdml`, `.xrcx`); never screenshots or numbers in filenames.	4, 7
At the instrument: 5. Fill protocol	**Item > Add Protocol** (`Ctrl+Shift+P`) or mobile **Quick Protocol**	Attach after extracting values if the template uses `values.all`; submit `Draft -> In Review`.	8
After measuring: 6. Extract & derive values	**Extract Values** / **Extract values from N files...** and **Add > Derived Value...**	Check viewer regions/indents and click **Apply** in `.zrw` first; keep 0 `Manual` values where a reader exists.	4, 9

WHEN & STEP	MENU / SHORTCUT (OR MCP TOOL)	RULE TO CHECK	CH.
Planning & write-up: 7. Hypotheses, Series & Wiki	**Project > Hypotheses**, **Project > Series...** (Full grid / Taguchi), and **Project > New Wiki**	Confirm `planned` series values after deposition; use `[[item:N]]` and `[[specimen:CODE]]` links; human review gates stay in GUI.	10–13
Write-up & analysis: 8. Search & correlate	**Navigate > Search...** (`Ctrl+F`), **Go to Item...** (`Ctrl+G`), and **Tools > Analysis...**	Store inputs (`Condition`) on the **Procedure** and outputs (`Result`) on child **Actions**.	14, 15
Before review: 9. Audit & sign	**Tools > Data Completeness...**, **Check Value Sources...**, then **Signature > Sign Item...**	Verify 0 missing files/values and 0 drift *before* signing (`SIGNED` is permanent).	16, 23
With an AI agent: 10. Automate via MCP	`ELNCommunityMCP.exe` or `ELNMCPServer.exe` (`check_storage`, `check_value_drift`, `report_value_issues`)	Enforce the **Value Route** (`Reader` / `Derived` only), save `.xrcx` fits with `curves.job_id`, and review legacy migration plans first.	17–21, 24

> **Tip**
>
> Print Table 1-1 or bookmark this page while you record your first few samples. Once you complete Chapter 4 once from top to bottom, the sequence becomes automatic.

Sources

- **The ELN provenance study and demonstration deposit:** *Value-level data provenance in an electronic laboratory notebook used by humans and language-model agents: An experience report* (arXiv:2610.27859; Zenodo deposit doi:10.5281/zenodo.23466959). Sections 1–2 (the four-link chain from sample to number); Section 4 (how value-level links break); Section 5 (value source metadata for `Reader`, `Derived`, `Manual`, and `Unknown`); Section 5.8 (`tribo_study` sample `261015A` and `xrr_study` sample `261014B`).
- **Student checklist for Study A:** `review/demo_A_student_checklist.md` in the paper repository (building the whole tree at the start, keeping results out of filenames, and extracting values rather than typing numbers).
- **APS ELN Community Help:** `Items.html` (the `Procedure`, `Specimen`, and `Action` hierarchy, and the **Values** tab source line with **Go to file**); `Values.html` (`Condition` vs. `Result`, `Reader`, `Manual`, `Derived`, and `Unknown` sources, multi-file extraction, and **Check Value Sources…**); `FilesAndNotes.html` (the 26 viewer plugins).

Cross-references: Introduction: About This Book; Chapter 2; Chapter 3; Chapter 4.

Opening a Local or Team Lab Notebook

IN THIS CHAPTER

- Understanding how local Community notebooks and shared Team servers store data
- Creating, opening, and switching local SQLite notebooks in APS ELN Community
- Logging into a shared laboratory server and using Demo Mode in a team lab
- Backing up your database and attachment directory together without losing files

Before you can create a project or record a sample (Chapter 1: The four links in every laboratory result), you need an open notebook. You can open a local notebook on your PC in **APS ELN Community** or sign into a shared server in **APS ELN ("For Teams")**. Once the notebook is open, the scientific workspace is the same (Chapter 3: The three-panel workspace).

Two ways ELN stores your laboratory work

APS ELN keeps two things in sync (Figure 2-1):

1. **The database:** projects, folders, procedures, specimens, actions, values, protocols, hypotheses, experiment series, tags, links, signatures, and the audit log.
2. **The attachment store:** the raw measurement files, note images, and analysis projects you attach.

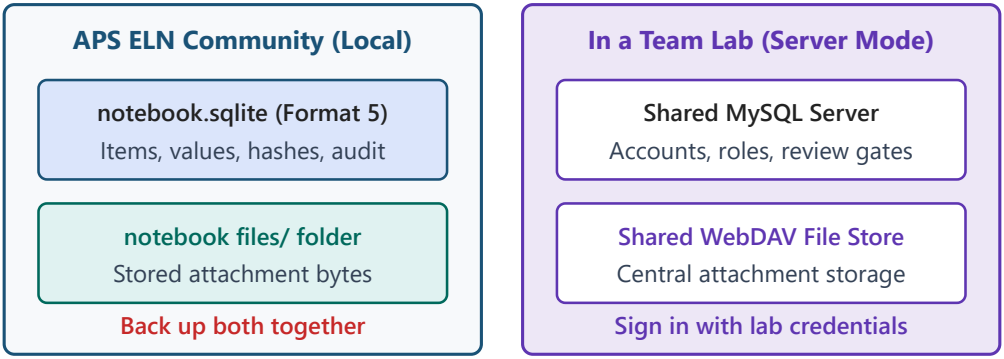


Figure 2-1: The two storage modes in APS ELN. In APS ELN Community (left), your computer holds a format 6 SQLite database (.sqlite) and a companion attachment folder. In a Team Lab (right), the client connects over the network to a shared MySQL database and a shared WebDAV file store.

In **APS ELN Community**, both parts sit in normal folders on your Windows PC: one SQLite database file (such as `notebook.sqlite`) and one separate attachment directory (such as `notebook files`). There is no server login.

In **APS ELN ("For Teams")**, the structured records are stored on a shared MySQL server and the attachment bytes are stored on a shared WebDAV or server store. You log in with the username and password issued by your laboratory administrator.

Creating a local notebook in Community

When you launch `ELNCommunity.exe` for the first time, no notebook is open yet. Follow these steps to create your first notebook:

1. Choose **File > New Notebook...**.
2. In the file dialog, pick a folder on a local drive and type a new database file name, such as `notebook.sqlite`. Do not select an existing file.
3. Select a separate, empty attachment directory. By default, the dialog creates and opens a new empty `<name> files` folder right beside

your `.sqlite` file; click **Select Folder** to accept it. The `.sqlite` file itself must sit outside the attachment folder.

4. Enter your **First name**, **Last name**, **Signing password**, and password confirmation in the local-profile dialog, then complete the dialog.
5. Wait a moment while APS ELN checks the seven scientific reference catalogs (`ValueTypes.csv`, `FileTypes.csv`, `ActionTypes.csv`, `EquipmentTypes.csv`, `Equipment.csv`, `ActionTypeEquipment.csv`, `FileTypeLinks.csv`) and imports the reference data into your new notebook.

> **In ELN**
>
> Use **File > New Notebook...** to create a new `.sqlite` notebook, **File > Open Notebook...** to open an existing one, and **File > Recent Notebooks** to switch among the last eight notebooks you opened.

If you accidentally choose an existing file in **File > New Notebook...**, Community stops with a warning before asking for an attachment folder and leaves the existing file untouched. Use **File > Open Notebook...** instead.

When you open an existing notebook with **File > Open Notebook...**, Community checks whether the database already remembers its attachment directory:

- On the same drive, the notebook stores the attachment folder as a relative path (for example, `..\Files` when `agent.sqlite` sits in a `DB` folder beside `Files`). If the folder is present, the notebook opens immediately.
- If the notebook has no saved attachment path, or if the folder moved separately, Community asks you to select that notebook's attachment folder. It then asks whether to remember the path for next time.

⚠ Warning: your signing password is not a file-encryption password

The signing password you set when creating a local notebook is checked only when you apply an electronic signature to lock a record (Chapter 16: Signing an item and verifying signature integrity). Opening a local `.sqlite` notebook does not ask for a password, and the database is not encrypted. Protect your computer account and backup drives with Windows access controls.

☰ Going Deeper: opening older format 2, 3, 4, or 5 notebooks and command-line flags

Current Community builds use **local notebook format 6**, which includes schema tables for `Derived` values, experiment series (**Project > Series...**), default reader mappings, and the tribology value types (`Wear area`, `Normal force`, `Cycles`, and `Wear rate`). When you open an older format 2, 3, 4, or 5 notebook, Community asks before changing anything. Clicking **Yes** writes a timestamped backup copy beside your database (named with `.format2-backup-`, `.format3-backup-`, `.format4-backup-`, or `.format5-backup-`). It then upgrades the database to format 6 and preserves all existing values and sources. Older versions of Community cannot open a format 6 database, so keep that backup file if you ever need an older build.

You can also launch Community from a script or shortcut with explicit paths:

```
ELNCommunity.exe --config "C:\ELN\desktop.ini" --notebook
"C:\ELN\notebook.sqlite" --storage "C:\ELN\notebook
files" --plugins "C:\ELN\Plugins"
```

When you launch Community without command-line flags, it automatically reopens the last notebook that opened cleanly.

Logging into a shared team server

In a research group that runs the Commercial team edition (`ELabNotebook3.exe`), your notebooks sit on the laboratory server. Your teammates and supervisor can then view shared projects, review protocols, and approve hypotheses.

> **In a Team Lab: first-run connection and server login**
>
> 1. **First-run setup:** The first time you start `ELabNotebook3.exe` on a workstation, the First-Run Connection Wizard asks how to connect. In a managed lab, select the `.elnconfig` configuration file provided by your administrator; it fills in the MySQL database and WebDAV attachment server automatically.
> 2. **Logging in:** On the Login screen, enter your **Username** and case-sensitive **Password**, then click **Log in**. A **Caps Lock warning** appears automatically if Caps Lock is on. Click the **eye icon** beside the password box to check what you typed, and tick **Remember credentials** to store your login in Windows Credential Manager.
> 3. **Three-strike lockout:** After three failed login attempts in a row, the program closes. Relaunch it to try again, or ask your administrator to reset a forgotten password.

In a team laboratory, your account carries one of five roles (Table 2-1). Most students work with a **Standard** or **PowerUser** account.

Table 2-1: User roles in a shared Team laboratory notebook.

ROLE	WHAT A USER WITH THIS ROLE CAN DO
Guest	Read projects granted to them; cannot create or edit scientific records or see review queues.

ROLE	WHAT A USER WITH THIS ROLE CAN DO
Standard	Create and edit procedures, specimens, actions, files, protocols, and values in projects where they have write access; submit protocols and hypotheses for review.
PowerUser	Everything a Standard user can do, plus create new projects of their own and manage reference data where permitted.
Super (Supervisor)	Review and approve student protocols and hypotheses across supervised projects, and filter the Dashboard by student and time range.
Admin	Manage user accounts, roles, permissions, audit rules, and server configuration.

> **Tip**
>
> To practice creating items or testing value extraction without touching your group's live server database, use **Demo Mode** on the login screen. You can also open the Study A and Study B local `.sqlite` demonstration notebooks in **APS ELN Community**.

How to back up a local notebook safely

In APS ELN Community, changes save to the `.sqlite` file as soon as you close a dialog. There is no **File > Save Notebook** command. When you back up your work, remember that the `.sqlite` file alone is only half of your notebook. The database stores file names and MD5 hashes, while the actual curves and images sit in the attachment folder.

Follow these five steps whenever you back up or move a local notebook:

1. **Save open editors.** Save any open Markdown note or external working draft (`Tools > Save Attachment Working Draft`), and resolve any conflict prompts.

2. **Close the notebook and stop MCP processes.** Close `ELNCommunity.exe` and verify that no `ELNCommunityMCP.exe` agent process is using the database. Never copy a SQLite database while a program is writing to it.
3. **Copy the database and attachment folder together.** Copy both the `.sqlite` file and the entire `<name> files` folder into one backup folder or archive. Keep the subfolder structure and filenames unchanged.
4. **Copy retained working drafts if needed.** If you declined to save a working draft at close and kept its temporary recovery path, copy that draft folder separately.
5. **Test your backup in a new folder.** When restoring, extract both the `.sqlite` file and the attachment folder into a new directory. Do not overwrite an existing notebook in place. Then open it with **File > Open Notebook...** and check a few files in the viewer.

> **Remember**
>
> To move a closed Community notebook to another folder or drive, move the parent folder that holds both the `.sqlite` database and its attachment folder. Because the notebook saves a relative path on the same drive, it will find its attachments at the new location automatically.

> **Warning: never delete SQLite sidecar files by hand**
>
> If a notebook was not closed cleanly, you may see `-journal`, `-wal`, or `-shm` files next to the `.sqlite` database. Never delete those files in Windows Explorer to clear a lock warning; they hold uncommitted database pages. Open and close the database in its owning client so SQLite can recover them cleanly.

Sources

- **APS ELN Community Help:** `GettingStarted.html` (creating your first notebook, **File > New Notebook...**, the seven scientific catalogs, and the signing password); `Notebooks.html` (keeping the `.sqlite` database and attachment directory together, **File > Open Notebook...**, **File > Recent Notebooks**, relative path storage on the same drive, upgrading format 2/3/4 notebooks to format 5 with `.format4-backup-`, command-line launch flags `--config`, `--notebook`, `--storage`, `--plugins`, and the 5-step consistent backup procedure).
- **APS ELN Commercial ("For Teams") Help:** `GettingStarted.html` (Local SQLite vs. remote MySQL and WebDAV modes, the First-Run Connection Wizard and `.elnconfig`, the Login screen with Caps Lock warning, eye icon, three-attempt close, and **Remember credentials** in Windows Credential Manager); `Administration.html` (user roles `Guest`, `Standard`, `PowerUser`, `Super`, and `Admin`).
- **Zenodo demonstration deposit (`doi:10.5281/zenodo.23466959`):** `zenodo/README.md` (how `agent.sqlite` and `agent_tribo.sqlite` sit in `DB/` beside `Files/` or `agent_tribo files/` in format 5).

Cross-references: Introduction: Where to Go from Here; Chapter 1; Chapter 3; Chapter 4.

A Tour of the Main Window

IN THIS CHAPTER

- Navigating the three main panels: Projects, Items, and Content Viewer
- Telling Procedures, Specimens, and Actions apart in the item tree
- Using row focus buttons, panel filters, and the Dashboard
- Setting display preferences, themes, and plugin paths safely

Once you open a local notebook or sign into your laboratory server (Chapter 2: Two ways ELN stores your laboratory work), APS ELN opens its main window. Almost everything you do in the program follows one left-to-right flow across three resizable panels (Figure 3-1).

The three-panel workspace

The main window is divided into four areas (Figure 3-1):

1. **The Projects panel (left):** shows your active projects, project groups, and specimen folders as a tree. Clicking a project or folder loads its scientific records into the center panel.
2. **The Items panel (center):** shows the scientific hierarchy inside the selected project or folder: **Procedures** (`Proc`), **Specimens** (`Item`), and **Actions** (`Action`). It also lists the files, rich-text notes, and protocol forms attached to each record.
3. **The Content Viewer (right):** previews the file, protocol, or item details selected in the center panel using the 26 built-in viewer plugins (Chapter 7: Inside the five core instrument viewers). Project-level **Hypotheses** (**Project > Hypotheses**, Chapter 11) and **Experiment Series** (**Project > Series...**, Chapter 12) open in their own separate top-level windows with their own taskbar buttons so they stay visible beside the main window.

4. **The Status bar (bottom):** displays the active connection mode (**Local** or **Remote**), the open `.sqlite` database path or server address, the selected attachment, and the build version number.

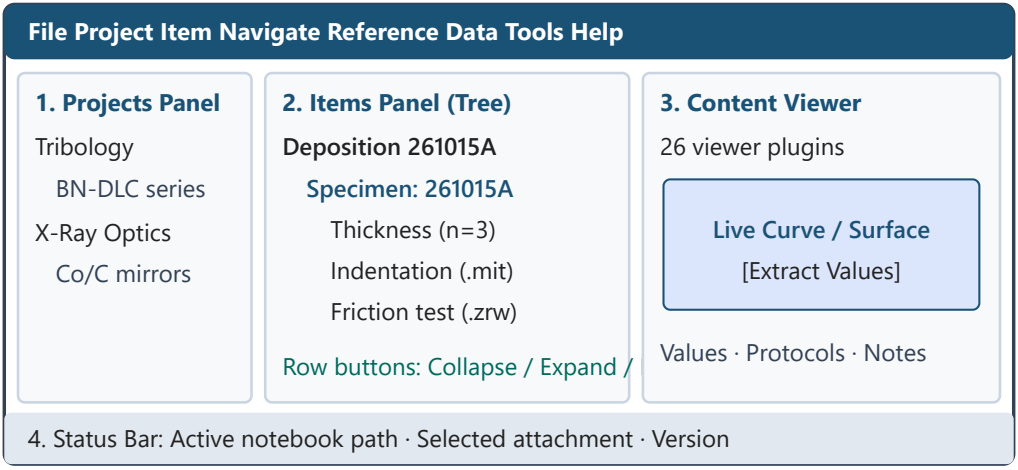


Figure 3-1: The three-panel workspace of APS ELN Desktop. Select a project or folder on the left (1), click a procedure, specimen, action, or attached file in the center (2), and inspect or extract data in the Content Viewer on the right (3).

You can drag the vertical splitter bars between the three panels to give more width to the item tree or to a wide chart in the Content Viewer.

> **In ELN**
>
> The main menu bar across the top of the window groups every command under seven headings: **File**, **Project**, **Item**, **Navigate**, **Reference Data**, **Tools**, and **Help**. Before a notebook is open, commands that require an active notebook remain disabled.

Reading the item tree and its row buttons

In the center **Items** panel, every scientific record has one of three hierarchy levels (Table 3-1).

Table 3-1: The three scientific record levels in the Items panel.

LEVEL IN UI	INTERNAL / MCP LEVEL	WHAT BELONGS ON THIS ROW	WHERE IT SITS IN THE TREE
Procedure	`Proc`	A synthesis, deposition run, or top-level experimental campaign (for example `Deposition 261015A`), along with its run conditions and deposition logs.	Root of a project or folder (`parent_id = 0`).
Specimen	`Item`	A physical sample, substrate piece, or computational model produced or studied in the procedure (for example `261015A`), with its description and specimen properties.	Child of a **Procedure**.
Action	`Action`	A preparation, treatment, measurement, test, or analysis step (for example `Thickness 261015A`, `Indentation 261015A`, or `XRR fit 261014B`), along with its raw files, protocol, and extracted values.	Child of a **Procedure**, a **Specimen**, or another **Action**.

When an action holds repeated measurements of the same quantity, such as the three step-height scans on `Thickness 261015A`, the item tree averages them on one line (Figure 3-2). It displays the mean, sample standard deviation, and count: `d = 335.4 ± 16.5 nm (n=3)` (or `d = 343.0 ± 23.5 nm (n=3)` with the manual rectangle selections in Chapter 20). To inspect each individual value and its source file, select the action and choose **Item > Edit**.

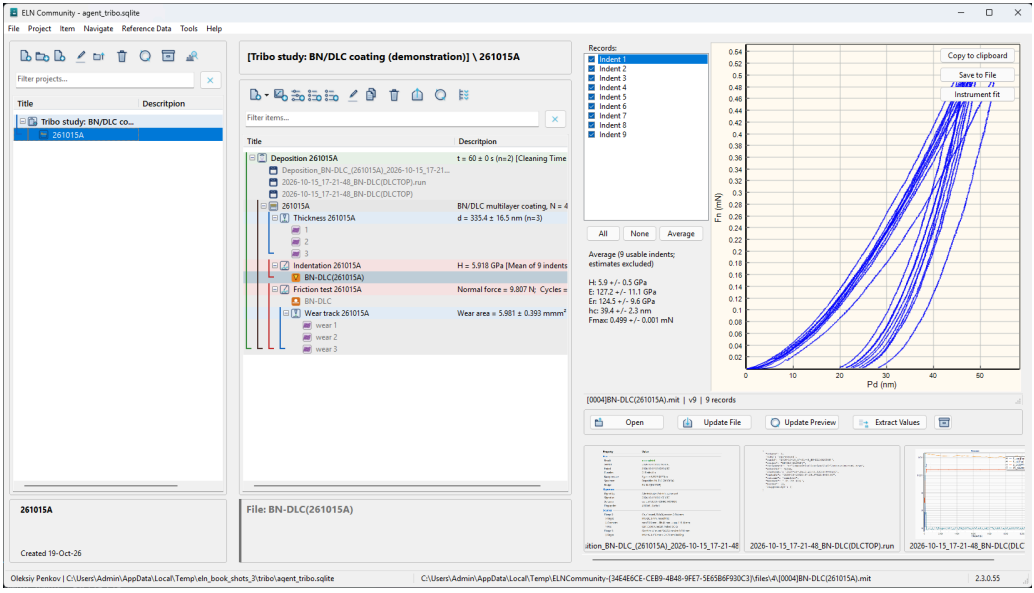


Figure 3-2: The Study A (261015A) notebook open in APS ELN Community. The center Items tree shows the full Procedure -> Specimen -> Action hierarchy with inline value summaries, while the right panel previews the selected nanoindentation file and its companion thumbnails.

When you select a **Procedure**, **Specimen**, or **Action** row that has child rows, three small buttons appear at the right end of that row:

- **Collapse all below:** closes every nested level under the selected row.
- **Expand all below:** opens every nested level under the selected row.
- **Focus** (tooltip: **Focus: collapse everything else**): collapses all other branches in the tree while keeping the path to your selected row open and scrolled into view.

> **Tip**
>
> When a project has dozens of depositions, click a procedure row and click **Focus** on its right edge. Every other procedure folds away so you can work on your current sample without scrolling.

⚠ Warning: dragging items in the tree

Dragging one procedure, specimen, or action row onto another row in the item tree can make the dragged record a child of the target row rather than reordering siblings. To change the manual order of a top-level root procedure safely, select the root item and press `Ctrl+Up` or `Ctrl+Down` (or use **Move Up** and **Move Down**). To move an item to another parent or folder, right-click it and choose **Move to...**.

The Dashboard and keyboard navigation

Each panel has a search bar at the top that filters the rows shown in that panel. When a panel filter is active, clear its search box to see all rows again. For a notebook-wide search or an overview of recent work, use the **Navigate** menu and the **Dashboard**.

In ELN

In **APS ELN Community**, press `Ctrl+D` (or choose **Navigate > Dashboard**) to open the notebook overview, and press `Ctrl+F` (or choose **Navigate > Search...**) to search across all notebook records. Press `Ctrl+G` (or choose **Navigate > Go to Item...**) to jump to an item by ID, and press `F1` for **Help > Keyboard Shortcuts...**.

In a Team Lab: the Team Lab Dashboard and Ctrl+D

In the Commercial ("For Teams") edition, open the Dashboard from **Edit > Dashboard**. On a shared laboratory server, the Dashboard displays five live cards:

- **Recent Items:** your most recently modified records across all projects.
- **Protocols Awaiting Review:** protocol forms submitted by students that need supervisor sign-off.

- **Waiting for You:** hypotheses waiting on a human gate (**Approve proposal** or **Accept conclusion**), showing the project, author, and waiting time.
- **Project Statistics:** item counts across your active projects.
- **Activity Feed:** recent changes across your group (supervisors with **Super** or **Admin** roles can filter by student account and time range).

Double-click any entry on the Dashboard to jump straight to that project and item. In the Commercial client, pressing `Ctrl+D` while the **Items** panel has focus duplicates the selected item (**Item > Duplicate**).

Setting preferences and plugin paths

You can adjust how the main window looks and where it loads templates and plugins by opening **Tools > Preferences...**:

- **General settings:** switch the VCL visual theme (light or dark), configure the startup Dashboard, set default chart appearance, and control preview scope. You can also toggle whether numeric item IDs appear beside titles in the tree.
- **Tree settings:** adjust font scale, row spacing, hierarchy tint, and level colors so long item trees are comfortable to read on your monitor.
- **Path settings:** choose the folders for protocol/experiment templates, viewer plugins (`Plugins`), and your laboratory report logo. Any change to the plugin directory takes effect the next time you start the program.

Remember

The attachment storage root displayed in **Tools > Preferences...** is shown for information only. Changing preferences does not move your stored attachment files. To move a local Community notebook, close the program and move the folder containing both the `.sqlite` file and

> its attachment directory (Chapter 2: How to back up a local notebook safely).

Sources

- **APS ELN Community Help:** `GettingStarted.html` (the project panel, item tree, content panel, status bar, **Navigate > Dashboard**, **Navigate > Search...**, and **Tools > Preferences...**); `Items.html` (`Procedure`, `Specimen`, and `Action` hierarchy, the three row buttons **Collapse all below**, **Expand all below**, and **Focus**, dragging caution, and `Ctrl+Up`/`Ctrl+Down` manual ordering); `Values.html` (automatic tree summary of repeated values as mean, sample standard deviation, and count); `Shortcuts.html` (`Ctrl+F`, `Ctrl+G`, `Ctrl+D`, `F1`, and main menu headings).
- **APS ELN Commercial ("For Teams") Help:** `GettingStarted.html` (the three-panel layout, **Edit > Dashboard**, the five Dashboard cards **Recent Items**, **Protocols Awaiting Review**, **Waiting for You**, **Project Statistics**, and **Activity Feed**, supervisor user/time filters, and `Ctrl+D` item duplication when the Items panel has focus).

Cross-references: Chapter 1; Chapter 2; Chapter 4.

Your First Sample Record in Twenty Minutes

IN THIS CHAPTER

- Building the complete Study A (261015A) item tree before attaching files
- Naming raw measurement files so sample codes stay consistent and results stay out of filenames
- Attaching deposition, step-height, nanoindentation, and tribometry files in one pass
- Extracting Reader values and calculating the reciprocating wear rate as a Derived value

The fastest way to learn APS ELN is to record one real sample from start to finish. In this chapter, you'll build **Study A (`261015A`)**, a boron nitride and diamond-like carbon (BN/DLC) coating grown by magnetron sputtering and tested on three instruments. All 11 raw files are in the book's demonstration bundle (`/MasteringELN/data/study-a/`). When you finish, your notebook will contain 6 items, 11 raw files with verified MD5 hashes, 23 `Reader` values, and 1 `Derived` wear-rate value (Figure 4-1).

Meet Study A: sample 261015A

You deposit a BN/DLC coating (`261015A`) onto silicon and masked steel substrates in a magnetron sputtering system, and then run four characterization steps:

1. **Deposition (`Deposition 261015A`):** the sputtering system writes a summary file (`Deposition_BN-DLC_(261015A)_2026-10-15_17-21-48.depo.json`), a run metadata file (`2026-10-15_17-21-48_BN-`

`DLC(DLCTOP).run.json`), and a time-series log (`2026-10-15_17-21-48_BN-DLC(DLCTOP).tsv`).

2. **Coating thickness (`Thickness 261015A`):** a Filmetrics Profilm3D optical profiler scans across the masked step edge on the steel piece at three positions (`1.fibps`, `2.fibps`, `3.fibps`).
3. **Hardness and modulus (`Indentation 261015A`):** an Anton Paar UNHT nanoindenter runs a 9-indent series on the coating (`BN-DLC(261015A).mit`).
4. **Friction and wear (`Friction test 261015A` and `Wear track 261015A`):** a DaVinci 1000 tribometer runs a 1,000-cycle reciprocating test at $1\,\mathrm{kgf}$ ($F_n = 9.81\,\mathrm{N}$) (`BN-DLC.zrw`). Next, the optical profiler measures the cross-sectional area of the wear track at three positions (`wear 1.fibps`, `wear 2.fibps`, `wear 3.fibps`).

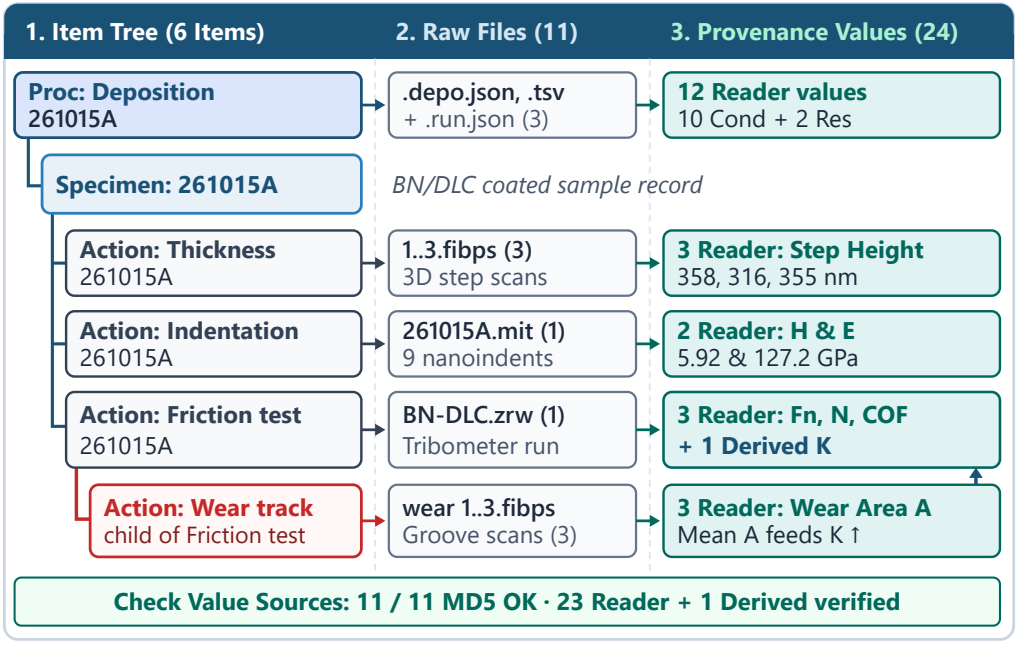


Figure 4-1: The Study A (261015A) item tree, raw files, and provenance values. Notice the four-level nesting at the bottom: Wear track 261015A is a child of Friction test 261015A, and its three Wear Area values feed the Derived wear rate K.

Table 4-1: The six items and eleven raw files of Study A (`261015A`).

ITEM ID	LEVEL & TITLE	PARENT	RAW FILES TO ATTACH	VALUES STORED
1	**Procedure:** `Deposition 261015A`	Project root / Folder	`.depo.json`, `.run.json`, `.tsv` (3 files)	12 `Reader` (`aps.deposition` v1)
2	**Specimen:** `261015A`	`Deposition 261015A`	None (specimen record)	0 (holds measurement actions)
3	**Action:** `Thickness 261015A`	`261015A`	`1.fibps`, `2.fibps`, `3.fibps` (3 files)	3 `Reader` (`aps.fibps` v3)
4	**Action:** `Indentation 261015A`	`261015A`	`BN-DLC(261015A).mit` (1 file)	2 `Reader` (`aps.mit` v2/v3)
5	**Action:** `Friction test 261015A`	`261015A`	`BN-DLC.zrw` (1 file)	3 `Reader` (`aps.zrw` v2) + 1 `Derived`
6	**Action:** `Wear track 261015A`	`Friction test 261015A`	`wear 1.fibps`, `wear 2.fibps`, `wear 3.fibps` (3 files)	3 `Reader` (`aps.fibps` v3)

Step 1: Build the whole item tree first

Always build the entire item tree before you start attaching files (Chapter 1: Four rules that keep your data trail unbroken). That way, every planned measurement has a home in the tree, and you can see immediately if any step is still missing its files.

1. Download and unpack `Study-A-261015A-raw.zip` from `/MasteringELN/data/`. Open your notebook (Chapter 2). In the **Projects** panel on the left, choose **Project > New Group** and enter the group name `Practice`. Next, right-click your `Practice` group and choose **New Project...** to create and select `Study A Practice` (Chapter 5).
2. Choose **Item > New Procedure**. Set **Title** to `Deposition 261015A`, select the item type `Synthesis` and **Action type** `Deposition`, and click **Save**.
3. Select `Deposition 261015A` in the **Items** panel and choose **Item > New Specimen**. Set **Title** to `261015A`, describe the substrates in **Description**, and click **Save**.
4. Select specimen `261015A` and choose **Item > New Action** three times to create its three child actions:
    - `Thickness 261015A` (**Action type:** `3D Profiling`)
    - `Indentation 261015A` (**Action type:** `Indentation`)
    - `Friction test 261015A` (**Action type:** `Tribotest`)
5. Select `Friction test 261015A` (not the specimen!) and choose **Item > New Action** to create its child action `Wear track 261015A` (**Action type:** `3D Profiling`).

> **In ELN**
>
> If your laboratory equipment is not yet listed in the **Equipment** drop-down, select an **Item type** and **Action type** first. Then click the **+** button beside **Equipment** (tooltip: **Add new equipment**) and fill in **Model**, **Manufacturer**, and **Location**; all three fields are required to enable **OK**.

Step 2: Attach the 11 raw files to their actions

Now attach the raw files from `/MasteringELN/data/study-a/` to the five procedure and action items in Table 4-1:

1. Select `Deposition 261015A` in the item tree and choose **Item > Add File** (or click **Add File** on the toolbar).
2. In the file browser, select `Deposition_BN-DLC_(261015A)_2026-10-15_17-21-48.depo.json`, `2026-10-15_17-21-48_BN-DLC(DLCTOP).run.json`, and `2026-10-15_17-21-48_BN-DLC(DLCTOP).tsv`.
3. Because multiple files are selected, click **Review files** to open **Files to attach**. Verify that the `.depo.json` file uses the **Deposition** file type (not plain JSON), keep **Keep display names** selected, and click **Attach**.
4. Select `Thickness 261015A`, choose **Item > Add File**, select `1.fibps`, `2.fibps`, and `3.fibps`, and attach them.
5. Select `Indentation 261015A`, choose **Item > Add File**, select `BN-DLC(261015A).mit`, and click **Attach**. For a single file, ELN attaches it directly from the file browser without a second dialog.
6. Select `Friction test 261015A` and attach `BN-DLC.zrw`.
7. Select `Wear track 261015A` and attach `wear 1.fibps`, `wear 2.fibps`, and `wear 3.fibps`.

> **Remember**
>
> Never put measurement results into file names. Name files after the sample code and position (`1.fibps`, `wear 1.fibps`, `BN-DLC(261015A).mit`). Adding a file copies its bytes into notebook storage and records its MD5 checksum; your original file in the source folder is never moved or altered.

Step 3: Extract Reader values from the files

With all 11 files attached, extract their numbers so every stored value carries a `Reader` source (Figure 4-1):

1. **Deposition parameters and logs (`Deposition 261015A`):** Click the `.depo.json` attachment to preview it in the **Deposition** viewer, then click **Extract Values**. Select the 10 condition keys

(`Parameters.CleaningTime` = $60\,\mathrm{s}$, `Parameters.P` = $0.267\,\mathrm{Pa}$, `Parameters.PowerBN` = $134\,\mathrm{W}$, `Parameters.TimeBN` = $50\,\mathrm{s}$, `Parameters.Bias` = $8\,\mathrm{V}$, `Parameters.Warmuptime` = $500\,\mathrm{s}$, `Parameters.N` = 46, `Parameters.CurrentC` = $0.312\,\mathrm{A}$, `Parameters.TimeC` = $190\,\mathrm{s}$, `Parameters.TimeCtop` = $340\,\mathrm{s}$). Also select the 2 result keys (`Duration` = $12941.5\,\mathrm{s}$, `BasePressure` = $9.10 \times 10^{-5}\,\mathrm{Pa}$). Keep **Remember choices for this reader** ticked and click **OK**.

2. **Coating step height (`Thickness 261015A`):** In the **FIBPS** viewer, verify the step evaluation regions on `1.fibps` and `2.fibps` (and automatic step detection on `3.fibps`). Select all three `.fibps` files in the item tree (`Ctrl`+click), right-click, and choose **Extract values from 3 files...**. Map `Step Height` to the thickness/step-height value type (`m`). ELN writes one `Reader` value per file ($3.58 \times 10^{-7}\,\mathrm{m}$, $3.16 \times 10^{-7}\,\mathrm{m}$, and $3.55 \times 10^{-7}\,\mathrm{m}$), storing each `Step Height STD` as the tolerance.
3. **Hardness and modulus (`Indentation 261015A`):** Click `BN-DLC(261015A).mit` to view the 9 loading-unloading curves in the **MIT** viewer, then click **Extract Values** (Figure 4-2). Store `Hardness` ($5.92 \pm 0.47\,\mathrm{GPa}$) and `Modulus` ($127.17 \pm 11.06\,\mathrm{GPa}$). The reader pairs `Hardness STD` and `Modulus STD` into the **Tolerance** field automatically.
4. **Friction test (`Friction test 261015A`):** Click `BN-DLC.zrw` in the **ZRW** viewer and check the plateau evaluation interval. Click **Extract Values** to store the conditions `Fn` ($9.81\,\mathrm{N}$) and `Cycles` (1000) and the result `COF` (0.528 ± 0.019).
5. **Wear track cross-sectional area (`Wear track 261015A`):** Select `wear 1.fibps`, `wear 2.fibps`, and `wear 3.fibps` in the item tree, right-click, and choose **Extract values from 3 files...**. Store the three `Wear Area` values ($0.00620\,\mathrm{mm}^2$, $0.00553\,\mathrm{mm}^2$, and $0.00621\,\mathrm{mm}^2$).

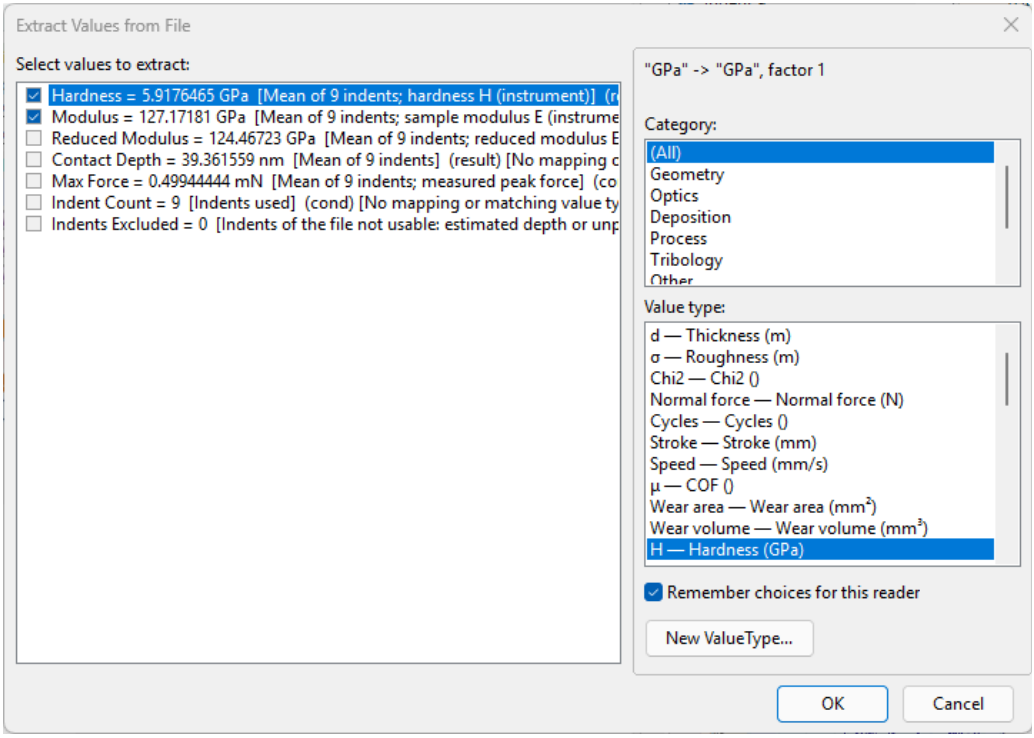


Figure 4-2: The Extract Values from File dialog on BN-DLC(261015A).mit. Each ticked key on the left maps to a notebook Value Type on the right, applying any unit conversion factor and pairing STD keys into the Tolerance field automatically.

> **Tip**
>
> If you extract values from the same file a second time, APS ELN updates the existing `Reader` value that came from that file instead of creating a duplicate row. Your `(n=3)` count on `Thickness 261015A` stays at 3.

Step 4: Compute the Derived wear rate

Finally, compute the reciprocating wear rate K from the stored wear-track areas, normal load, and cycle count without using an external calculator or

spreadsheet:

1. Right-click `Friction test 261015A` in the item tree and choose **Add > Derived Value...**.
2. In the **Formula** drop-down, select the reciprocating wear rate formula (`wear_rate_reciprocating`, version 1):

$$K = \frac{A}{2 \cdot F_n \cdot N}$$

3. For the `area` input role, `Ctrl`+click all three stored `Wear Area` values ($0.00620\,\mathrm{mm}^2$, $0.00553\,\mathrm{mm}^2$, and $0.00621\,\mathrm{mm}^2$) from child action `Wear track 261015A`. The dialog automatically takes their mean ($A = 0.00598\,\mathrm{mm}^2$) and records all three value IDs as inputs.
4. For `normal_force`, select `Fn` ($9.81\,\mathrm{N}$). For `cycles`, select `Cycles` (1000).
5. Check the previewed result ($K = 3.05 \times 10^{-7}\,\mathrm{mm}^3/(\mathrm{N} \cdot \mathrm{mm})$) and click **Save**.
6. Right-click the root procedure `Deposition 261015A` and choose **Check Value Sources...**. APS ELN re-reads all 11 source files through their current viewer plugins and re-evaluates the `Derived` wear rate from its stored inputs. It then reports all 24 values unchanged (Figure 4-1).

Going Deeper: what Check Value Sources verifies on a Derived value

When you run **Check Value Sources...** on an item tree (or call `check_value_drift` through MCP), APS ELN checks both levels of provenance:

- For every **`Reader`** value, it runs the current reader plugin on the stored attachment bytes and applies the saved unit conversion factor. It also applies any saved analysis snapshot, such as ticked indents or a ZRW cycle interval, and compares the output with the stored number.

- For every **Derived** value, it re-reads the `Reader` inputs from their files and computes their mean when multiple inputs fill one role (such as the three `Wear Area` scans). It then runs the saved formula version (`wear_rate_reciprocating` version 1). If an input value was edited or deleted after the `Derived` value was created, the check flags `input_changed` or `input_missing`.

Sources

- **Study A demonstration record and verification report:** `zenodo/demonstration/tribo_study/reports/record_report.md` and `content_check.md` in the ELN data deposit (doi:10.5281/zenodo.23466959). Gives the exact 6 items, 11 files (`file_id` 1–11 with MD5 hashes), 23 `Reader` values (`aps.deposition` v1, `aps.fibps` v3, `aps.mit` v2, `aps.zrw` v2), and 1 `Derived` value (`value_id` 24, `wear_rate_reciprocating` v1, $K = 3.04929 \times 10^{-7}\ \mathrm{mm}^3/(\mathrm{N}\cdot\mathrm{mm})$).
- **Student checklist for Study A:** `review/demo_A_student_checklist.md` (creating the full tree first, naming files without results, and extracting values from all attached files).
- **APS ELN Community Help:** `Items.html` (**Item > New Procedure**, **Item > New Specimen**, **Item > New Action**, and the **+** button for **Add new equipment**); `FilesAndNotes.html` (**Item > Add File**, single-file attach vs. **Review files** -> **Files to attach**, and `.depo.json` compound extension routing); `Values.html` (**Extract Values**, **Extract values from N files...**, **Remember choices for this reader**, `<Key> STD` tolerance pairing, **Add > Derived Value...** with `wear_rate_reciprocating`, multi-select `Wear area` inputs, and **Check Value Sources...**).

Cross-references: Introduction: Beyond the Book; Chapter 1: Four rules that keep your data trail unbroken; Chapter 2: Creating a local notebook in Community; Chapter 3: Reading the item tree.

Projects, Groups, and Folders

IN THIS CHAPTER

- Creating and organizing Projects, Groups, and Folders in the left panel
- Archiving finished projects without losing access to their records
- Sharing projects and setting review permissions in a team laboratory
- Restoring deleted items from the Recycle Bin safely

When you open your notebook (Chapter 2: Two ways ELN stores your laboratory work), you first select a project or folder in the left panel (Chapter 3: The three-panel workspace). Keeping that left panel clean lets you find a specimen in two seconds instead of scrolling through hundreds of unrelated depositions.

How Groups, Projects, and Folders fit together

The **Projects** panel on the left uses three levels of containers (Figure 5-1 and Table 5-1):

1. **Groups:** optional top-level headings (such as `Coatings & Optics`) that group your projects visually in the tree. Groups never hold scientific records directly.
2. **Projects:** your main research topics (such as `Tribology` or `Co/C`). Every procedure, specimen, action, hypothesis, and wiki page belongs to one project.
3. **Folders:** subfolders inside a project (such as `BN-DLC Series` inside `Tribology`) that organize related procedures and specimens. You can nest folders to any depth.

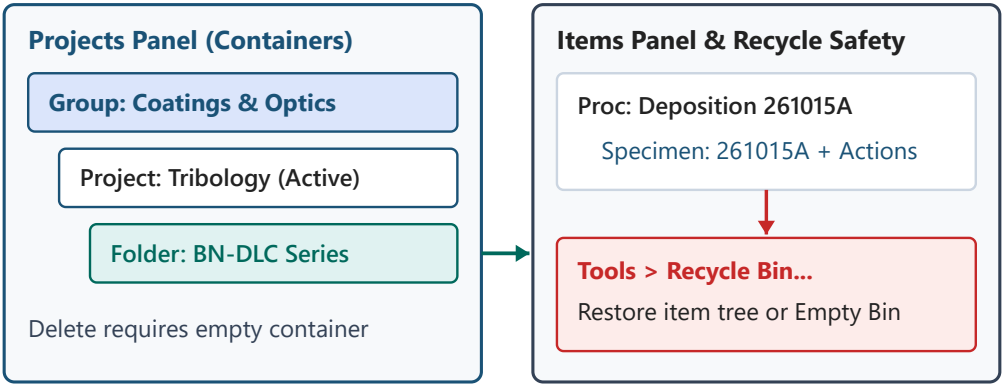


Figure 5-1: How containers and scientific items separate in APS ELN. Groups, Projects, and Folders in the left panel organize your work. Procedures, Specimens, and Actions in the center panel hold your scientific data and move to the Recycle Bin when deleted.

Table 5-1: Containers in the Projects panel.

CONTAINER	WHERE IT SITS	WHAT IT HOLDS	WHAT HAPPENS WHEN DELETED
Group	Top level of the Projects panel	Projects (visual grouping only)	Projects inside the group move back to the top level; no data is deleted.
Project	Top level or inside a Group	Folders, Procedures, Specimens, Actions, Hypotheses, and Wiki pages	Allowed only when the project is completely empty (including the Recycle Bin).
Folder	Inside a Project or parent Folder	Subfolders and root Procedures (`Proc`)	Allowed only when the folder has no items, subfolders, or binned items.

When you reopen a notebook, APS ELN automatically re-selects the project or folder you were working in when you closed it.

Creating, sorting, and moving projects and folders

Follow these steps to set up a new project and organize its folders:

1. **Create a project:** Choose **Project > New Project** (or click **New Project** at the top of the Projects panel). Enter the project **Title** (for example, `Tribology`) and **Description**, then click **Save**.
2. **Edit a project:** Select the project and choose **Project > Edit** (or double-click the project row) to update its title or description.
3. **Create a group:** Choose **Project > New Group** (or right-click empty space in the Projects panel and choose **New Group...**). Enter a non-blank **Group name:**. Drag any project onto the group to file it there, or right-click the group and choose **New Project...** to create a project directly inside it.
4. **Create a folder:** Select a project (or an existing folder) in the Projects panel and choose **New Folder** (`Ctrl+Shift+D` in the team client). Enter the folder title and description and click **Save**.
5. **Reorganize folders:** Use **Move Folder Up** in the folder's context menu to move a nested subfolder one level higher toward the project root.

> **Remember**
>
> Changing a project's title or moving a project into a group updates the database record only. Never rename or move files inside your notebook's attachment storage folder in Windows Explorer to match a new project title.

> **In ELN**
>
> Like rows in the **Items** panel (Chapter 3: Reading the item tree and its row buttons), any selected Group, Project, or Folder row with children

displays three buttons on its right edge. Use them to **Collapse all below**, **Expand all below**, or **Focus** (**Focus: collapse everything else**).

> **In a Team Lab: sorting and dragging projects and folders**
>
> In the Commercial ("For Teams") client, right-click any project or folder and open the **Sort** submenu. You can order siblings **By Name**, **By Date Created**, or manually with **Move Up** and **Move Down**. The saved order is shared across every workstation in the lab. You can also drag a root procedure (`Proc`) from the center **Items** panel and drop it onto a different folder or project in the **Projects** panel. ELN moves the entire item subtree and relocates its stored attachment files automatically.

Archiving finished projects

When a grant ends or a sample series is finished, archive the project so it stays out of your daily workspace without breaking links from later work:

1. Select the finished project in the **Projects** panel and choose **Archive** from its context menu (or double-click the project, tick the **Archived** check box, and click **Save**).
2. To view archived projects at any time, click **Show Archived** in the Projects panel toolbar (`Ctrl+Shift+A` in the team client).
3. To reopen an archived project for new experimental work, select it while archived projects are shown and choose **Archive** again (or clear **Archived** in **Project > Edit** and click **Save**).

> **Tip**
>
> An archived project remains 100% readable and searchable. You can still inspect its curves, copy data from its viewers (Chapter 7: Inside the five core instrument viewers), and cite its items as reference evidence. Meanwhile, the archive lock prevents anyone from accidentally editing or deleting its records.

Sharing projects in a team laboratory

In **APS ELN Community**, your local notebook has one identity and you own every project in it. In a team laboratory running the Commercial server, each project has an owner who controls who can read, edit, or review records inside it.

> **In a Team Lab: granting project access and naming hypothesis reviewers**
>
> Right-click a project you own in the **Projects** panel and choose **Share…** to open the project permissions dialog:
>
> - **Read access:** lets a teammate browse the project's items, preview its files, and search its values without changing anything.
> - **ReadWrite access:** lets a teammate create and edit procedures, specimens, actions, files, protocols, and values inside the project.
> - **Hypothesis approval rights:** administrators and supervisors can tick **Can approve proposals** and **Can accept conclusions** for designated senior researchers who hold `ReadWrite` access on the project.
>
> **Groups** in the Projects panel are personal to your account. Projects that a teammate shares with you appear at the top level of your tree, and arranging your own groups never changes how teammates see the project tree.

Deleting containers and using the Recycle Bin

Deleting an empty container (a Group, Project, or Folder) works differently from deleting a scientific item (a Procedure, Specimen, or Action):

- **Deleting a Group** (**Delete Group**) never removes any projects; its projects return to the top level of the Projects panel.

- **Deleting a Project or Folder** is permanent and is refused unless the container is completely empty. Even deleted items sitting in the **Recycle Bin** still belong to their original project and folder, so you must restore or permanently remove them before the container itself can be deleted.

When you delete a **Procedure**, **Specimen**, or **Action** in the center **Items** panel, it is not destroyed immediately. Instead, it moves to the **Recycle Bin** (Figure 5-1):

1. Select the item in the **Items** panel and choose **Item > Delete** (or right-click and choose **Delete**).
2. Confirm the prompt **Move item … to the recycle bin?**. The item and all of its child actions disappear from the active tree.
3. If you deleted the item by mistake, choose **Tools > Recycle Bin…**, select the binned record, and click **Restore**. Close the dialog to see the restored subtree back in your project.
4. To remove a binned record forever, open **Tools > Recycle Bin…**, select it, and click **Delete** (or click **Empty Bin** to purge all unprotected items in the bin).

> ⚠ **Warning: files and notes do not go to the Recycle Bin**
>
> The Recycle Bin holds deleted **Procedures**, **Specimens**, and **Actions**. Deleting an individual **File** or **Note** attached to an active item deletes that file or note permanently (along with the `Reader` values extracted from that file) after confirmation. Never use **Empty Bin** to delete a folder faster without checking what is inside the bin.

> ☰ **Going Deeper: why Empty Bin sometimes keeps a deleted item**
>
> A binned item may still be cited as evidence or attached as a run on a hypothesis that is `InReview` or `Accepted` (Chapter 11: Concluding, closing or accepting, and checking the content seal). It may also carry an electronic signature (Chapter 16: Signing an item and verifying signature integrity). In either case, **Empty Bin** keeps that item tree,

> purges the other binned items, and tells you which records were kept and why.

Sources

- **APS ELN Community Help:** `Projects.html` (**Project > New Project**, **Project > Edit**, **Project > New Group**, **New Folder**, **Move Folder Up**, the three row buttons in the project panel, **Archive** and **Show Archived**, container deletion rules, **Delete Group**, and **Tools > Recycle Bin...** with **Restore**, **Delete**, and **Empty Bin**); `Items.html` (items go to the Recycle Bin, whereas files and notes are deleted permanently).
- **APS ELN Commercial ("For Teams") Help:** `Projects.html` (`Ins` for **New Project**, `F2` for **Edit**, `Ctrl+Shift+A` for **Active/Archive**, `Ctrl+Shift+D` for **New Folder**, drag and drop in the Projects panel, **Sort** submenu with **Move Up**, **Move Down**, **By Name**, and **By Date Created**, and **Share...** with `Read`, `ReadWrite`, **Can approve proposals**, and **Can accept conclusions**).

Cross-references: Chapter 2: Two ways ELN stores your laboratory work; Chapter 3: The three-panel workspace; Chapter 6: Designing a clean item hierarchy; Chapter 7: Raw Files, Drafts, and Instrument Viewers.

The Item Tree and Experiment Templates

IN THIS CHAPTER

- Choosing between Procedures, Specimens, and Actions for each step of an experiment
- Adding missing Action Types and Equipment without leaving the item dialog
- Selecting multiple items for batch value extraction, tagging, moving, or deletion
- Creating and reusing Experiment Templates (.elntemplate) for repeatable workflows

In Chapter 4: Step 1: Build the whole item tree first, you built the six-item tree for Study A (`261015A`) by hand. Doing that click by click works for one sample, and gets old fast when you run a series of twenty coatings with the same tests. In this chapter, you'll see how to nest multi-step experiments cleanly, run batch actions across many rows at once, and create a complete sample tree in seconds from an **Experiment Template** (`.elntemplate`).

Designing a clean item hierarchy

Every scientific record in the center **Items** panel is a **Procedure** (`Proc`), a **Specimen** (`Item`), or an **Action** (`Action`). Picking the right level and parent keeps your sample tree easy to read (Table 6-1):

- **Put a synthesis or deposition run at the root as a `Procedure`.** In both Study A (`Deposition 261015A`) and Study B (`Deposition 261014B`), the sputtering run is a top-level **Procedure**. It creates the sample and holds the run-wide deposition conditions and logs.
- **Put a preparation step that happens *before* or *during* the run as a child `Action` of the `Procedure`.** In Study B, plasma substrate cleaning (`Cleaning 261014B`) is a child **Action** under `Deposition 261014B`.

- **Put each physical sample produced by the run as a `Specimen` under the `Procedure`.** If one deposition run coats both a silicon wafer and a steel coupon, list the substrates in the specimen description (as in `261015A`). When the substrates serve different experimental purposes, create separate **Specimen** rows under the same **Procedure**.
- **Put every measurement or test on that sample as a child `Action` under the `Specimen`.** In Study A, `Thickness 261015A`, `Indentation 261015A`, and `Friction test 261015A` sit directly under specimen `261015A`.
- **Nest a follow-up measurement or analysis under the `Action` it depends on.** In Study A, `Wear track 261015A` measures the groove created by `Friction test 261015A`, so it sits under `Friction test 261015A`. In Study B, `XRR fit 261014B` fits the curve measured in `XRR-1 261014B`, so it sits directly under `XRR-1 261014B` (Figure 1-2).

Table 6-1: Choosing the right level, Item Type, and Action Type.

LABORATORY STEP	LEVEL IN UI	ITEM TYPE	TYPICAL ACTION TYPE	PARENT ROW
Sputter deposition or chemical synthesis	**Procedure** (`Proc`)	`Synthesis`	`Deposition`	Project or Folder root
Substrate plasma cleaning or annealing	**Action** (`Action`)	`Treatment`	`Cleaning` or `Annealing`	The **Procedure** or **Specimen**
Coated wafer, coupon, or simulation model	**Specimen** (`Item`)	`Specimen` or `Model`	(None)	The **Procedure** that made it
Optical profiler step height, XRD/XRR, or indentation	**Action** (`Action`)	`Characterization` or `Testing`	`3D Profiling`, `XRD`, `Indentation`	The **Specimen**

LABORATORY STEP	LEVEL IN UI	ITEM TYPE	TYPICAL ACTION TYPE	PARENT ROW
Tribometer friction test	**Action** (`Action`)	`Testing`	`Tribotest`	The **Specimen**
Wear-track 3D scan after friction test	**Action** (`Action`)	`Characterization`	`3D Profiling`	The `Friction test` **Action**
X-Ray Calc 3 reflectivity fit (`.xrcx`)	**Action** (`Action`)	`Computing` or `Characterization`	`XRR Fitting` or `Analysis`	The `XRR` measurement **Action**

> **Remember**
>
> A file, a note, or a protocol is attached *to* an item; it is not a child item of its own. Always create the **Action** row for a measurement or fit first, then attach the file and extract its values onto that action.

Creating, duplicating, and moving items safely

When you create or edit a **Procedure** or **Action** (**Item > New Procedure**, **Item > New Action**, or **Item > Edit**), the dialog asks for **Title**, **Description**, **Item type**, **Action type**, and **Equipment**.

When you install a new instrument or start a new test, the drop-down list may lack your **Action type** or **Equipment**. You can add it inside the item dialog without opening **Reference Data**:

1. Select an **Item type** first. This enables the **+** button right beside **Action type** (tooltip: **Add a new action type**). Click **+**, enter a non-blank **Title**, select a **Kind**, and click **OK**.

2. Select an **Action type** that takes equipment. This enables the **+** button right beside **Equipment** (tooltip: **Add new equipment**). Click **+**, enter **Model**, **Manufacturer**, and **Location** (all three are required), and click **OK**.

> **In ELN**
>
> The **APS ELN Community** installer ships with empty equipment tables (`Equipment.csv` and `ActionTypeEquipment.csv` contain only the header and one blank row) because every laboratory has its own instruments. Use the **+** button beside **Equipment** the first time you record each instrument, or manage the full list under **Reference Data**.

To copy an existing item structure, select the item and choose **Item > Duplicate**. In the duplicate dialog, pick the destination folder and tick the option to link the copy to the original item if the new run is a repeat or continuation.

To move an item to a different parent or folder after creation, right-click the item and choose **Move to...**. Select the new parent item or destination folder from the list and click **Move**.

> **Warning: do not drag items onto each other to reorder siblings**
>
> In the center **Items** panel, dragging one item row onto another row can reparent the dragged item as a child of the target row. To change the manual display order of top-level root procedures, select the root procedure and press `Ctrl+Up` or `Ctrl+Down` (or use **Move Up** and **Move Down**). Child actions, files, and values are not reordered by `Ctrl+Up` or `Ctrl+Down`.

Working on multiple records at once

You can select several rows at once in the **Items** panel to run batch operations across an entire sample series:

- Hold `Ctrl` and click to add or remove individual rows from your selection.
- Hold `Shift` and click (or press `Shift+Up` and `Shift+Down`) to select a contiguous range of rows.
- Right-click any row *inside* your selection to open the batch context menu while keeping all rows selected. (Right-clicking a row *outside* the selection switches the selection to that single row.)

When multiple records are selected, APS ELN disables single-item commands (**New**, **Edit**, **Duplicate**, **Add File**, **Add Note**, **Add Protocol**, **Add Value**, **Links...**). Instead, it enables six batch commands:

1. **Extract values** (**Extract values from N files...**): groups the selected files by reader plugin and asks you to map the keys once per reader. It then extracts values across all matching files (Chapter 9: Reader mappings, unit factors, and standard deviations).
2. **Link to plan point...**: matches selected top-level procedures to the free points of an active **Experiment Series** by their stored factor values (Chapter 12: Making or linking depositions and confirming planned values).
3. **Move to...**: moves all selected items or files to a chosen destination.
4. **Tags...**: assigns or removes tags across all selected items.
5. **Export**: exports all selected files into a destination folder on disk, automatically appending `(2)`, `(3)` if two files share the same display name so no file is ever overwritten.
6. **Delete**: moves selected items to the Recycle Bin, or permanently deletes selected files and their extracted `Reader` values (Chapter 5: Deleting containers and using the Recycle Bin).

Every batch command shows a single confirmation dialog before it starts. If any selected row cannot be processed (for example, because an item is signed or sits in a read-only project), the confirmation names that row and the reason. It then skips that row safely and shows a summary report at the end.

Building and applying Experiment Templates

When your group runs a standard sequence of synthesis and characterization steps, save that tree as an **Experiment Template** (Figure 6-1).

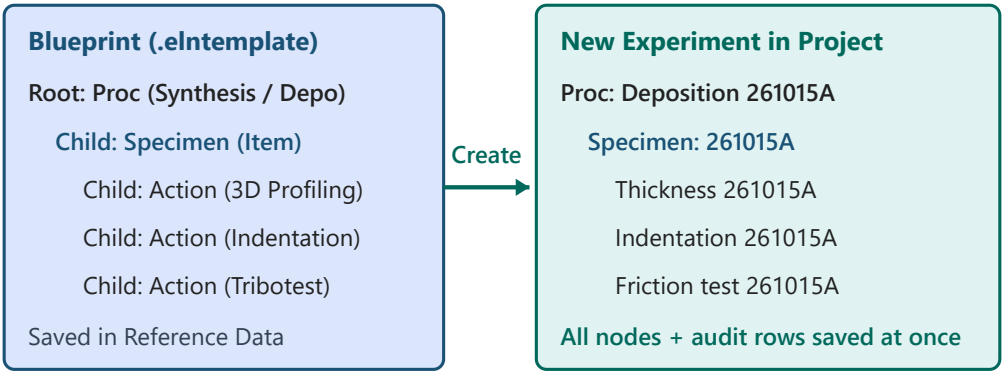


Figure 6-1: How an Experiment Template (.elntemplate) works. You define the Procedure, Specimen, and Action nodes once in Reference Data > Experiment Templates, then stamp out the entire hierarchy for each new sample with Item > New from Template.

APS ELN ships with ready-made `.elntemplate` files in `Assets/Runtime/ExperimentTemplates/` (such as `Deposition_Mechanical.elntemplate`, `Deposition_XRayOptics.elntemplate`, `Deposition_FullCharacterization.elntemplate`, and `Specimen_SurfaceMorphology.elntemplate`). You can also build your own template in five steps:

1. Choose **Reference Data > Experiment Templates...**.
2. Click **Add**, and choose whether the root of your template is a **Procedure** (`Proc`) or a **Specimen** (`Item`).
3. Enter the template **Name** (for example, `BN-DLC Tribology Standard`), **Category**, and **Description**.

4. Select the root node in the structure tree and set its **Title**, **Level**, **Type**, and **Action Type**. Use **Add Child** and **Add Sibling** to add the specimen and measurement action nodes (`Thickness`, `Indentation`, `Friction test`, and child `Wear track`).
5. Click **Save** and inspect the hierarchy preview.

To start a new sample from your template:

1. Select your destination project and folder in the **Projects** panel.
2. Choose **Item > New from Template...**.
3. Pick the template from the list and check the hierarchy preview (which shows the levels, action types, and equipment).
4. Enter the **Experiment title** (for example, `Deposition 261015A`) and click **Create**.

> **Tip**
>
> You can share experiment templates with colleagues or move them between a team server and a local Community notebook. In **Reference Data > Experiment Templates...**, click **Export** to save a template as an `.elntemplate` file, and use **Import** (or **Import All...** for a folder) to load `.elntemplate` files into another notebook.

> **Going Deeper: atomic template creation and Protocol vs. Experiment templates**
>
> Do not confuse **Experiment Templates** with **Protocol Templates** (Chapter 8: How protocol templates and AutoFill work):
>
> - An **Experiment Template** (`.elntemplate`, managed in **Reference Data > Experiment Templates...**) creates a multi-item tree of **Procedures**, **Specimens**, and **Actions** in the center **Items** panel. When you click **Create** in **Item > New from Template...**, APS ELN writes the entire hierarchy and its audit-log rows in one atomic transaction. If anything fails, no half-built tree is left behind.

- A **Protocol Template** (`.json`, managed in **Reference Data > Protocol Templates...**) defines a structured form with sections, rows, and input fields that you attach to *one* item via **Item > Add Protocol**.

Sources

- **APS ELN Community Help:** `Items.html` (`Procedure`, `Specimen`, and `Action` hierarchy, **Item > New Procedure**, **Item > New Specimen**, **Item > New Action**, the **+** buttons for **Add a new action type** and **Add new equipment**, **Item > Edit**, **Item > Duplicate**, **Move to...**, dragging caution, `Ctrl+Up`/`Ctrl+Down`, and multi-record selection with `Ctrl`+click / `Shift`+click for batch **Delete**, **Extract values**, **Move to...**, **Tags...**, and **Export**); `Templates.html` (difference between Protocol Templates and Experiment Templates, **Reference Data > Experiment Templates...**, **Add Child** / **Add Sibling**, **Item > New from Template...**, atomic hierarchy creation, and `.elntemplate` **Export**, **Import**, and **Import All...**).
- **APS ELN Community repository:** `README.md` (why the Community `Equipment.csv` and `ActionTypeEquipment.csv` catalogs ship with an empty logical row) and `ELN.Desktop/Assets/Runtime/ExperimentTemplates/` (the shipped `.elntemplate` blueprints).
- **Demonstration studies (`doi:10.5281/zenodo.23466959`):** `tribo_study/reports/record_report.md` (Study A `261015A` hierarchy) and `xrr_study/reports/record_report.md` (Study B `261014B` hierarchy).

Cross-references: Chapter 1: Two running examples: Study A and Study B; Chapter 4: Step 1: Build the whole item tree first; Chapter 5: Deleting containers and using the Recycle Bin; Chapter 8: How protocol templates and AutoFill work; Chapter 9: Reader mappings, unit factors, and standard deviations.

Raw Files, Drafts, and Instrument Viewers

IN THIS CHAPTER

- Attaching single and multiple files and understanding compound extensions
- Inspecting data in the specialist viewers: Deposition, FIBPS, MIT, ZRW, and X-Ray Calc
- Saving and reloading working drafts when editing attachments in external programs
- Using the Markdown Editor and the standalone ELN File Viewer

Every measurement starts with a raw file (Chapter 1: Four rules that keep your data trail unbroken). When you attach a file in APS ELN, the program records its MD5 checksum, builds a preview thumbnail, and opens it in one of 26 built-in **Content Viewer** plugins. In the specialist viewers, you can also mark step-height regions, tick valid indents, or set a friction cycle window. APS ELN saves those selections with the file so your extracted values stay reproducible (Chapter 9).

How file attachment and extension routing work

To attach one or more files to a **Procedure**, **Specimen**, or **Action**, select the item in the **Items** panel and choose **Item > Add File** (or click **Add File**):

- **Attaching one file:** Select the file in the built-in file browser, check **Attachment file type** and **Display name**, and click **Attach**. There is no second confirmation dialog.
- **Attaching multiple files:** Select several files in the browser and click **Review files** to open **Files to attach**. There you can inspect each row's **Source filename**, **Display name**, and **Type**, and either keep **Keep display names** selected or choose **Use prefix:** to number a series of scans automatically.

> **Remember**
>
> APS ELN routes files by their longest matching extension. A `.depo.json` snapshot opens in the specialist **Deposition** viewer rather than the plain **JSON** viewer. Similarly, a `.process.log` file opens in the **Process Log** viewer rather than the sputtering `.log` viewer. When suggested display names are created, ELN strips the full compound suffix (`.depo.json` or `.process.log`) instead of stripping only `.json` or `.log`.

When you select a `.depo.json` file in the file browser, APS ELN can automatically expand your selection to include its paired `.tsv` time-series log file. It then opens the **Files to attach** review dialog so both files are attached to the procedure together.

Inside the five core instrument viewers

APS ELN ships with 26 viewer plugins (`NOTATION.md`, Section 5). General-purpose viewers handle `.pdf`, `.html`, `.md` (Markdown), images (`.png`, `.jpg`, `.bmp`, `.tif`), columnar text (`.dat`, `.csv`, `.txt`, `.ascii`), and `.json`. Other viewers handle Office documents (`.xlsx`, `.docx`, `.pptx` via OLE), OriginLab projects (`.opju`), Gatan TEM images (`.dm3`), and LAMMPS simulations (`.lmpin`, `.dump`, `.data`).

Five specialist viewers handle the files in Study A (`261015A`) and Study B (`261014B`) (Table 7-1):

1. **Deposition (`.depo.json`, `.tsv`), Recipe (`.rcpx`), and DSC (`.dsc`):**
   - Attaching both the `.depo.json` snapshot and its paired `.tsv` log to the same **Procedure** unlocks the **Overview**, **Recipe**, **Pressure**, **Temperature**, **Gas Flow**, **Bias**, **V / I**, and **Stage** tabs. If the `.tsv` file is missing from the item, the viewer warns *Not attached to this item - charts unavailable*.
   - **Extract Values** reads the `.depo.json` file (`aps.deposition`) to give recipe conditions such as pressure, power, current, gas

flow, layer times, and period count N. It also extracts the run results `Duration` and `BasePressure` (Figure 7-2).

2. **FIBPS 3D optical profilometer (`.fibps`, reader `aps.fibps`):**
   - Displays the surface height map in micrometers (μm, or nm below $1\ \mu$m) with six interactive tools: **Profile**, **Distance**, **Roughness** (R_a, R_q, R_z inside a dragged rectangle), **Flatten**, **Step Height**, and **Wear Track**. In **Profile**, click for a horizontal line profile or `Shift`+click for a vertical profile.
   - In **Step Height**, two rectangles (**A** and **B**) mark the coated and uncoated levels. Dragging them updates the step $h \pm s$ and saves those rectangles as the *shared regions* of the file in the notebook.
   - In **Wear Track**, the viewer automatically detects the wear groove and levels the outer surface to zero (Figure 7-3). It then integrates the cross-sectional area of each row between the groove edges to give `Wear Area` (mean $\pm$ SD along the track).
   - Clicking **Copy Data** copies the active profile to the clipboard in OriginLab's three-header-row layout (Long Name, Units, Comments).
3. **MIT nanoindentation (`.mit`, reader `aps.mit`):**
   - Plots the contact-zeroed load-displacement curves (subtracting frame compliance when stored) and lists every indent under **Records:**.
   - Use **All**, **None**, or individual checkboxes to tick the valid indents, and click **Average** or **Instrument fit** to overlay the mean curve or the unloading power-law fit (Figure 7-4). The ticked indents are saved as the file's shared selection and recorded in the provenance snapshot of the extracted `Hardness` (H_{IT}), `Modulus` (E_{IT}), and `Reduced Modulus` (E_r) values.
4. **ZRW tribometer (`.zrw`, `.zrv`, reader `aps.zrw`):**
   - Reads both classic `Data.dat` archives and logger packages (`force.raw`, `resistance.tsv`, `test.json`) and plots friction coefficient (`COF`) against cycles.
   - In the **Interval** bar, choose **Automatic** (first stable plateau covering at least 20% of cycles) or **Manual** (**From** / **To** cycles).

Click **Apply** to save the shared interval (Figure 7-5). **Extract Values** uses the applied interval to compute `COF` and its tolerance (`COF STD`).

5. **X-Ray Calc (`.xrcx`, reader `aps.xrcx`) and XRDML (`.xrdml`):**
    - The **XRDML** viewer plots raw X-ray diffraction and reflectivity scans.
    - The **X-Ray Calc** viewer opens `.xrcx` project archives and plots the measured and calculated reflectivity curves on a log axis. Beside the plot, it shows the **Structure** table (H, σ, and ρ of each layer; Figure 7-6). **Extract Values** reads the main periodic stack's mean `Period`, layer thicknesses, and `Chi2`.

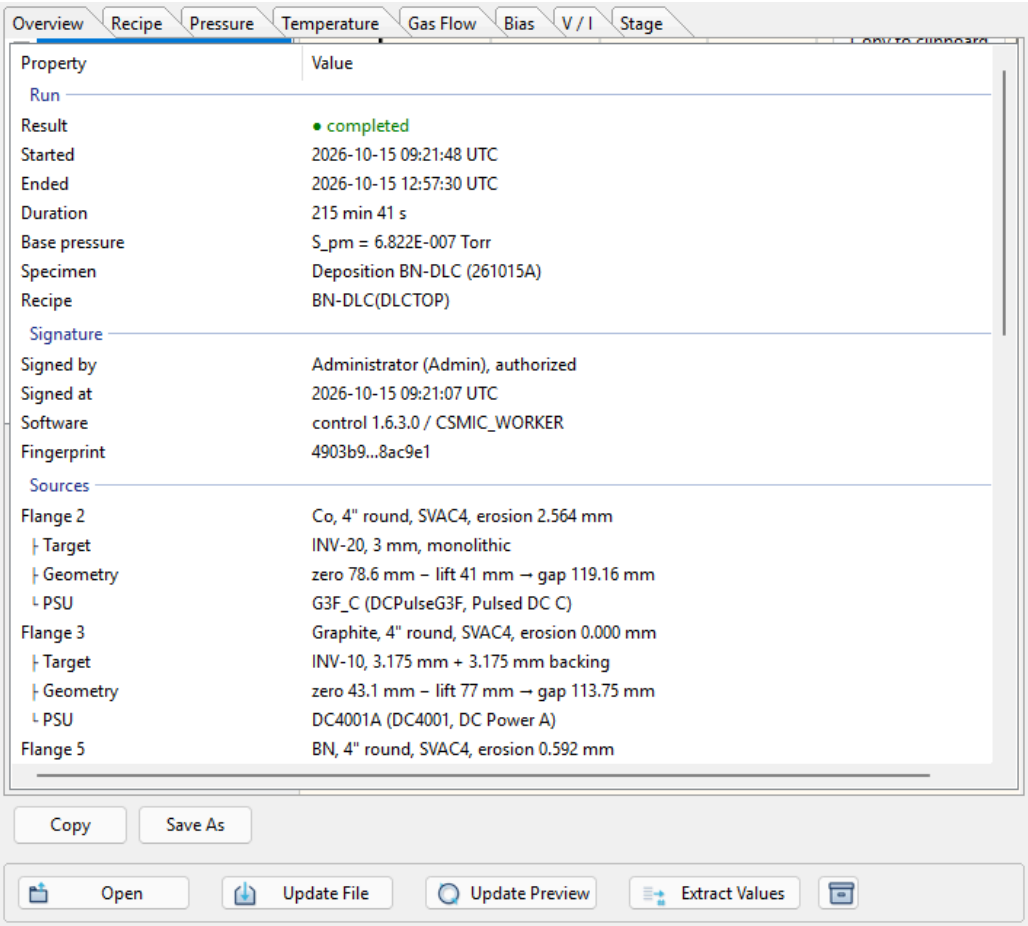


Figure 7-2: The Deposition viewer displaying the Overview tab for Study A (261015A), with time-series tabs unlocked by the paired .tsv log attached to the same procedure.

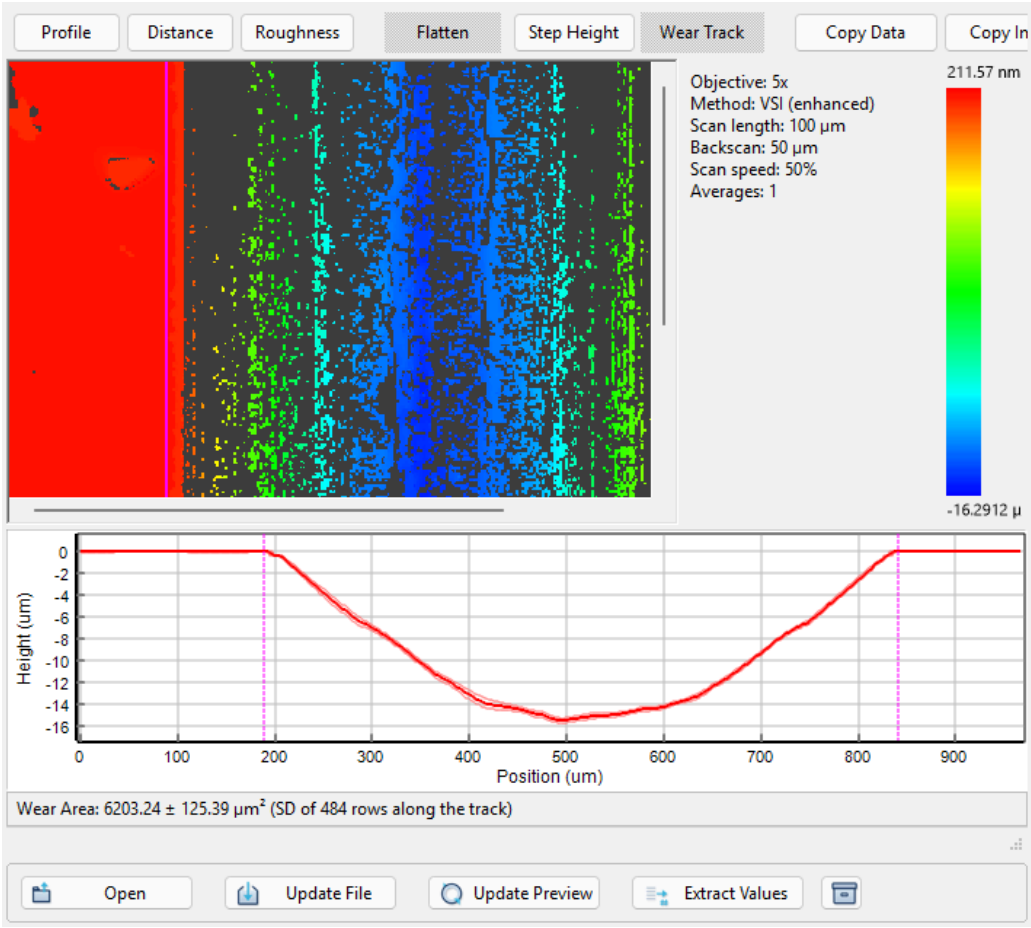


Figure 7-3: The FIBPS viewer in Wear Track mode on wear 1.fibps (Study A). The viewer levels the outer surface, brackets the wear groove between vertical dashed markers, and integrates the cross-sectional area across all 484 scan rows.

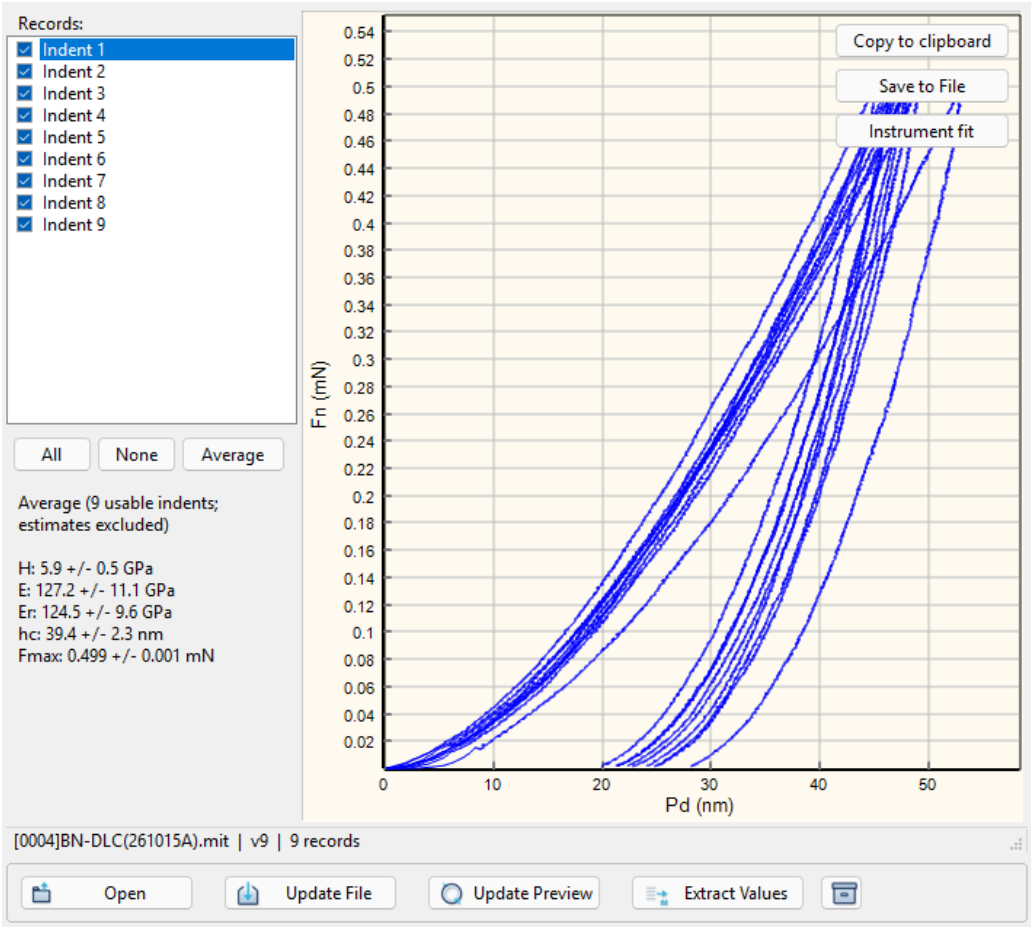


Figure 7-4: The MIT nanoindentation viewer on BN-DLC(261015A).mit (Study A). Ticking or unticking indents under Records updates the mean Hardness and Modulus and saves the selection for Extract Values.

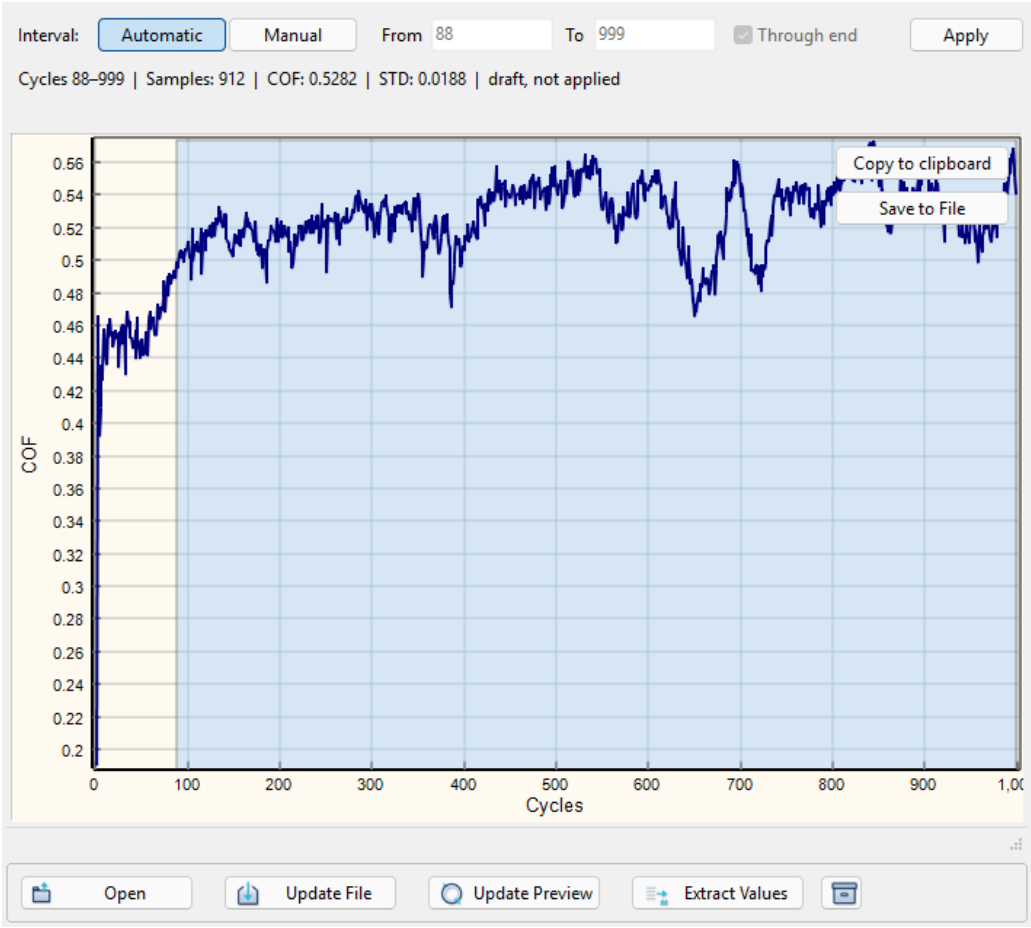


Figure 7-5: The ZRW tribometer viewer on BN-DLC.zrw (Study A), showing the shaded evaluation interval (cycles 88–999) and the Apply button that saves the shared interval to the notebook.

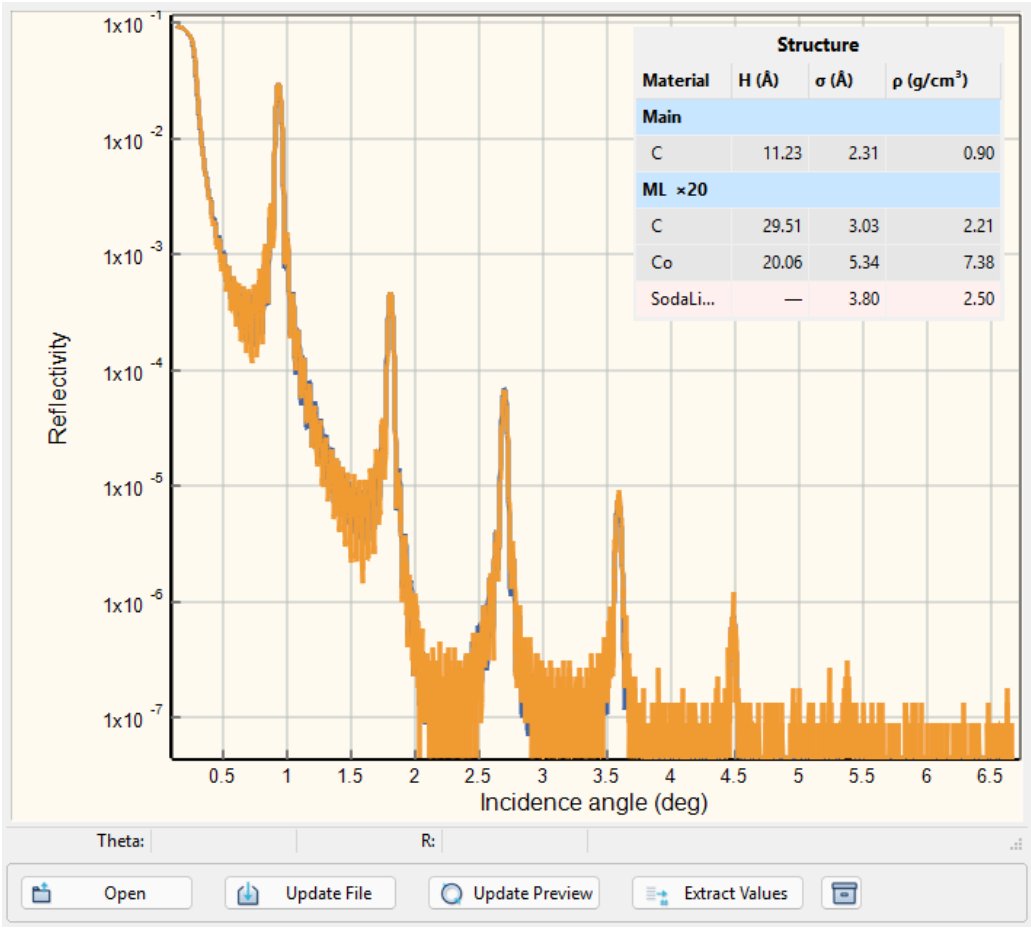


Figure 7-6: The X-Ray Calc (.xrcx) viewer on XRR-fit$_{Co}$-C(261014B).xrcx (Study B), overlaying the measured and fitted reflectivity curves alongside the fitted layer Structure table.

Table 7-1: Shared selection states in the specialist instrument viewers.

VIEWER & EXTENSION	READER ID	WHAT YOU CAN ADJUST IN THE VIEWER	HOW THE SELECTION IS SAVED FOR PROVENANCE
FIBPS (`.fibps`)	`aps.fibps`	Step-height rectangles **A** and **B**, or automatic **Wear Track** groove	Dragged step rectangles become the file's shared regions (`regions` snapshot mode).
MIT (`.mit`)	`aps.mit`	Ticked indents in the **Records:** list	Ticked indent indices are saved as the shared selection and stored on extracted values.
ZRW (`.zrw`, `.zrv`)	`aps.zrw`	**Automatic** plateau or **Manual** cycle range (**From** / **To**)	Clicking **Apply** saves the shared cycle interval with the file hash; unapplied drafts are ignored.
X-Ray Calc (`.xrcx`)	`aps.xrcx`	Zoom on reflectivity chart and inspect **Structure** table	Read-only display; extracts mean layer thicknesses, `Period`, and `Chi2` from the saved fit.

> ⚠ **Warning: always click Apply in the ZRW viewer after changing the cycle interval**
>
> In the **ZRW** viewer, typing new **From** and **To** cycle numbers previews the mean COF in the status line, but **Extract Values** never reads an unapplied draft. Click **Apply** first so your interval is saved as the shared selection of the file. If a file is later replaced via **Reupload...**, its old interval is marked stale until you click **Apply** again.

Working drafts, external edits, and conflict resolution

When you double-click an attachment (or click **Open** in the viewer toolbar), it opens in an external Windows program such as OriginLab, Excel, or X-Ray

Calc 3. APS ELN never lets the external program edit the file inside your notebook storage folder directly. Instead, it writes a separate **local working copy** (Figure 7-1).

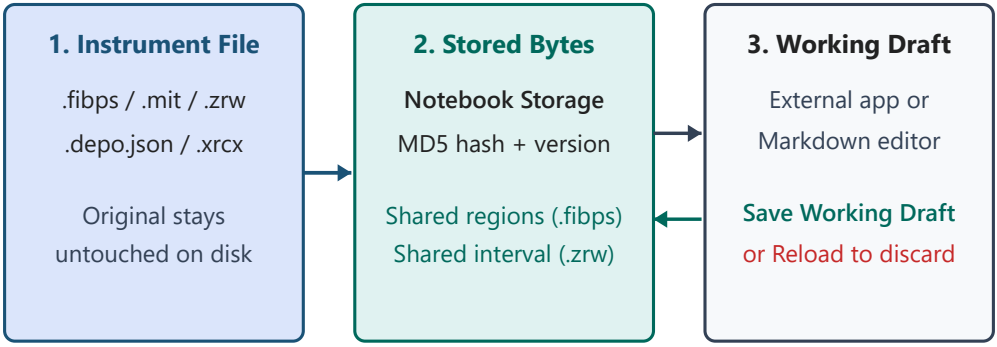


Figure 7-1: How APS ELN protects stored attachment bytes. Attaching a file copies it into notebook storage (2). Opening it externally creates a separate working draft (3) that only updates notebook storage when you choose Tools > Save Attachment Working Draft.

To save changes from an external program back into your notebook:

1. Save the file inside the external application (which updates the local working copy), and close the external editor window if required.
2. Select the attachment in APS ELN and choose **Tools > Save Attachment Working Draft** to copy the updated bytes into notebook storage and update the attachment's MD5 hash.
3. If you want to discard your external edits and revert to the bytes stored in the notebook, choose **Tools > Reload Attachment Working Draft** and confirm the discard prompt.

If you close the notebook or exit APS ELN while a working draft has unsaved changes, the program prompts you before closing. Clicking **Cancel** stops the close, while clicking **No** leaves your draft file at the temporary path shown in the message so you do not lose your work.

For `.md` attachments and project Wiki pages (Chapter 13: The five-section Project Wiki), clicking **Edit** in the **Markdown** viewer opens the built-in **Markdown Editor**. Press `Ctrl+S` to save, `Ctrl+B` for bold, `Ctrl+I` for italic, `Ctrl+K` for links, and `Ctrl+V` to paste clipboard images directly as uploaded attachments. If the stored attachment was updated by another session or by an MCP agent while you were editing, the conflict dialog offers three explicit choices:

- **Cancel:** keep your editor text open without overwriting the stored attachment.
- **Reload (discard my edits):** replace your editor text with the current stored attachment.
- **Save Anyway (overwrite server):** overwrite the stored attachment with your draft (if write permissions and signature guards allow it).

Replacing, moving, exporting, and viewing files outside a notebook

Right-click any attached file in the **Items** panel to replace, rename, move, or export it without breaking its links:

- **Reupload...:** replaces the stored bytes of the attachment with a new file from disk (keeping the same `file_id` and preserving compound file types such as `.depo.json`).
- **Edit:** renames the attachment's display title or updates its description.
- **Move to...:** moves the attachment to another writable, unsigned item in the project.
- **Export:** copies the stored attachment bytes out to a folder on disk (adding `(2)`, `(3)` if a filename already exists there).

> **In a Team Lab: inspecting files outside a notebook with ELNFileViewer.exe**
>
> The Commercial build includes a standalone personal viewer program, **`ELNFileViewer.exe`** (marked by a red notebook icon). It loads the

> same folder browser and all 26 viewer plugins without asking for a login or connecting to MySQL or WebDAV. Press `Ctrl+0`, drop a single file onto the window, or pass a file path on the command line (`ELNFileViewer.exe "C:\Measurements\sample.mit"`). You can inspect `.mit`, `.fibps`, `.zrw`, `.xrcx`, `.vk4`, or `.depo.json` files on an instrument PC before you attach them to a notebook.

Sources

- **APS ELN Community Help:** `FilesAndNotes.html` (**Item > Add File**, single-file attach vs. **Review files** -> **Files to attach**, compound extensions `.depo.json` and `.process.log`, the 26 shipped viewers, local working copies with **Tools > Save Attachment Working Draft** and **Tools > Reload Attachment Working Draft**, the **Markdown Editor** conflict choices **Cancel**, **Reload (discard my edits)**, and **Save Anyway (overwrite server)**, and **Reupload...**, **Edit**, **Move to...**, **Export**, and **Delete**); `Values.html` (ZRW plateau intervals, **Apply**, **Reload**, stale selections, MIT **Records:** selection and compliance, and X-Ray Calc `.xrcx` extraction); `Shortcuts.html` (Markdown editor shortcuts `Ctrl+S`, `Ctrl+B`, `Ctrl+I`, `Ctrl+K`, `Ctrl+V`).
- **APS ELN Commercial ("For Teams") Help:** `Plugins.html` (FIBPS tools **Profile**, **Distance**, **Roughness**, **Flatten**, **Step Height** shared regions, **Wear Track** groove integration, and **Copy Data** OriginLab layout; ZRW automatic vs. manual intervals; Recipe `.rcpx`, Process Log `.process.log`, and Deposition `.depo.json` + `.tsv` pairing; X-Ray Calc `.xrcx` viewer; preview thumbnails; and standalone `ELNFileViewer.exe`).
- **APS ELN Commercial README:** `APS-ELN-Commercial/README.md` (building and using `ELNFileViewer.exe`).

Cross-references: Chapter 1: Four rules that keep your data trail unbroken; Chapter 4: Step 2: Attach the 11 raw files; Chapter 6: Working on multiple records at once; Chapter 9: Values, Units, Formulas, and Provenance.

Filling Out Protocols and Checklists

IN THIS CHAPTER

- Attaching a structured protocol form to a procedure, specimen, or action
- Using AutoFill fields (values.all, conditions.all, results.all) in protocol templates
- Moving a protocol through Draft, In Review, Approved, and Rejected states
- Submitting protocols from a phone at the instrument with the Quick Protocol web app

Raw files record what the instrument measured, and extracted **Values** (Chapter 9) store the numbers for plotting and search. Plenty of lab work, though, never shows up in a data file: how you cleaned and mounted the substrate, which target clamp you replaced, whether the cooling-water interlock passed, or why a coating looked hazy when you vented the chamber. **Protocols** record those checklists and setup notes right on the item.

Adding and completing a protocol on an item

A protocol is a structured form (or free-text entry) attached to a **Procedure**, **Specimen**, or **Action** (Chapter 6: Designing a clean item hierarchy). APS ELN ships with ready-to-use protocol templates across twelve categories in `Assets/Runtime/ProtocolTemplates/`. These include **Deposition** (`CSMICSputtering.json`, `SputterIII_1Source.json`–`3Source.json`), **Characterization** (`XRD.json`, `XRF.json`, `SEM.json`, `TEM.json`, `AFM.json`, `3DProfiling.json`), and **Testing** (`Indentation.json`, `TribotestProtocol.json`, `Adhesion.json`), plus **Sample Preparation**, **Safety**, **Quality**, and **Reporting** (`WeeklyLabReport.json`).

Follow these steps to attach and complete a protocol:

1. Select the parent **Procedure**, **Specimen**, or **Action** in the **Items** panel.
2. Choose **Item > Add Protocol** (or press `Ctrl+Shift+P` in the team client).
3. In the **Select Template** dialog, pick a category and a template (or choose **(No template)** for a free-text protocol), then click **OK**.
4. In the **Protocol Editor**, check **Title**, **Protocol Type** (`Experiment`, `Measurement`, `Analysis`, `Synthesis`, `Testing`, `Note`, or `Other`), and **Validation Status** (`Draft`), and fill in the form fields.
5. Click **Save**. When you select the protocol in the **Items** panel, the **Content Viewer** renders the completed form as formatted HTML.

> ⚠ **Warning: switching templates clears your entered form fields**
>
> Inside the **Protocol Editor**, changing the **Template** drop-down clears all saved form fields on that protocol so the new schema can load. Never switch the **Template** drop-down to browse other templates after you have typed experimental notes into a form.

When you edit an existing protocol, an asterisk (*) marks unsaved changes. If you close the editor or click another item before saving, APS ELN prompts you to save, discard, or cancel so you never lose entries by accident.

How protocol templates and AutoFill work

You can inspect or customize protocol templates by choosing **Reference Data > Protocol Templates...** (or **Templates > Protocol Templates...** in the team client). Each template is built from sections (**+Section**), rows (**+Row**), and typed fields (**+Field**), and can be tagged with equipment types via **Tags...** (Table 8-1).

When a protocol is created on an item that already holds values or metadata, the template's **AutoFill** bindings can pre-populate fields automatically (Table 8-1).

Table 8-1: Protocol field types and `AutoFill` value bindings.

FIELD TYPE OR AUTOFILL KEY	KIND	WHAT IT DISPLAYS OR INSERTS INTO THE PROTOCOL
`text`, `textarea`, `richtext`	Field type	Single-line text, multi-line text, or formatted rich-text blocks (use `text` or `textarea` for `AutoFill` summaries).
`number`, `date`, `select`	Field type	Numeric entry, date picker, or drop-down list of predefined choices.
`checkbox`, `checkbox-group`	Field type	Single yes/no check box or multi-select checklist group.
`values.all`	`AutoFill` source	All values on the item, grouping repeated values into `mean ± SD (n=k)`, for example `d = 343.0 ± 23.5 nm (n=3)`.
`conditions.all`, `results.all`	`AutoFill` source	Every condition or every result on the item listed individually without grouping.
`values.<Symbol>`	`AutoFill` source	The numeric value of the first stored value matching `<Symbol>` (such as `values.H` or `results.COF`).

> **Remember**
>
> Protocol form fields do not compute formulas or create `Derived` value provenance, and `AutoFill` inserts a one-time text snapshot when the protocol is created. Store quantitative results as item **Values** via **Extract Values** or **Add > Derived Value...** (Chapter 9: The four value sources and Derived formulas). Attach the protocol after those values are in place if your template uses `values.all`.

> **Going Deeper: protocol template versioning**
>
> When you edit and save a protocol template that is already used by existing records, APS ELN automatically creates a **new template**

> **version** instead of overwriting the old schema. Existing protocols keep the exact field layout they were completed with, while new protocols use the latest active version. Tick **Show all versions (including inactive)** in the Template Manager to inspect historical versions.

The protocol review workflow

Every protocol moves through four validation states (Figure 8-1):

1. **`Draft`:** the author can edit all fields freely. When the form is complete, change **Validation Status** to `In Review` and click **Save**.
2. **`In Review`:** the protocol is locked against ordinary edits and enters the review queue.
3. **`Approved`:** the reviewer accepts the protocol. Once `Approved`, the protocol is permanently read-only and cannot return to `Draft`.
4. **`Rejected`:** the reviewer rejects the protocol and enters a mandatory rejection reason. The author reads the reason, switches the protocol back to `Draft`, saves, fixes the issue, and submits it to `In Review` again.

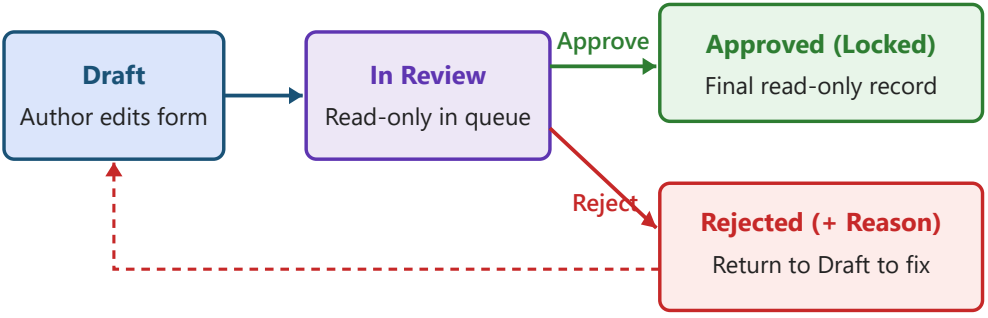


Figure 8-1: The four validation states of a protocol. Submitting a Draft moves it to In Review. Approval locks the protocol permanently; Rejection records a required reason and lets the author return it to Draft.

In ELN

To process protocols waiting in `In Review`, choose **Tools > Protocol Review...** (`Ctrl+Shift+R` in the team client). Select a protocol to read its rendered preview, then click **Approve** or **Reject**. Inside any saved protocol's editor, click **History...** to view the timestamped log of status changes, reviewers, and rejection comments.

In **APS ELN Community**, protocol review is a local self-check by the scientist on their own notebook. Like all human review gates in APS ELN, it must be performed in the Desktop GUI (the MCP server cannot approve or reject protocols).

Submitting protocols from the instrument in a team lab

In a team laboratory where instruments sit in a cleanroom or across the hall from your desk PC, you can complete a protocol right at the machine from your phone or tablet.

In a Team Lab: the Quick Protocol mobile web app

Laboratories running the Commercial **ELN Web API** server provide a mobile web app with three tabs (**Quick Protocol**, **Waiting for me**, and **Projects**). To record a run at an instrument in under a minute:

1. **Sign in** on your phone with your lab username and password (you can register your phone as a trusted web device so future visits sign in automatically).
2. **Find the specimen** by typing a few characters of its sample code (such as `261015A`) into the autocomplete box.
3. **Pick a template:** templates tagged for that instrument via **Tags...** and templates recently used on that specimen appear at the top of the list.

4. **Fill the fields, optionally snap a camera photo, and submit:** the protocol and photo are written directly to the shared MySQL and WebDAV server under that specimen. They appear immediately in the desktop client's **Protocols Awaiting Review** queue (Chapter 3: The Dashboard and keyboard navigation).

Sources

- **APS ELN Community Help:** `Protocols.html` (**Item > Add Protocol**, **Select Template**, **Protocol Editor**, `(No template)` free text, the warning that switching **Template** clears saved form data, the four review states `Draft`, `In Review`, `Rejected`, and `Approved`, **Tools > Protocol Review...**, **History...**, and local human review boundaries); `Templates.html` (**Reference Data > Protocol Templates...**, **+Section**, **+Row**, **+Field**, **Tags...**, **Load Example...**, **Show all versions (including inactive)**, and **Reimport**).
- **APS ELN Commercial ("For Teams") Help:** `Protocols.html` (`Ctrl+Shift+P` for **Add Protocol**, protocol types, the eight field types `text`, `number`, `date`, `select`, `textarea`, `checkbox`, `checkbox-group`, and `richtext`, why calculated formulas belong in **Add > Derived Value...**, `AutoFill` sources `values.all`, `conditions.all`, `results.all`, and `values.<Symbol>`, `Ctrl+Shift+R` for **Protocol Review Queue**, template versioning, and the **Quick Protocol** mobile web app).
- **APS ELN Community repository:** `ELN.Desktop/Assets/Runtime/ProtocolTemplates/` (the 12 categories of shipped JSON/HTML protocol templates).

Cross-references: Chapter 3: The Dashboard and keyboard navigation; Chapter 6: Building and applying Experiment Templates; Chapter 9: The four value sources and Derived formulas.

Values, Units, Formulas, and Provenance

IN THIS CHAPTER

- Recording Conditions and Results with the right Value Type, unit, and tolerance
- Mapping reader keys and converting units automatically during Extract Values
- Computing Derived values from stored inputs with server formulas
- Auditing your items with Check Value Sources to catch drift and changed selections

If you only attach raw files, your notebook is just a file folder. Storing **Values** turns it into a searchable database (Chapter 1: The four links in every laboratory result). Save every deposition pressure, layer thickness, hardness, friction coefficient, and wear rate as a typed value with its unit, tolerance, and source. APS ELN can then average repeated scans in the item tree, plot correlations across samples, and verify that every number matches its raw file.

Conditions, Results, units, and plausible-range checks

Every value stored on a **Procedure** or **Action** has one of two record types:

- **Condition (`EXP_CONDITION`):** an experimental input or setting you chose for the run. Examples include sputtering pressure ($0.267\,\mathrm{Pa}$), target power ($134\,\mathrm{W}$), bias voltage ($8\,\mathrm{V}$), normal force ($F_n = 9.81\,\mathrm{N}$), and cycle count ($N = 1000$).
- **Result (`TEST_RESULT`):** an outcome measured by the instrument or calculated from measurements. Examples include base pressure ($9.10 \times 10^{-5}\,\mathrm{Pa}$), coating step height ($3.58 \times 10^{-7}\,\mathrm{m}$), indentation hardness ($5.92 \pm 0.47\,\mathrm{GPa}$), coefficient of friction (0.528 ± 0.019

), wear rate ($K = 3.05 \times 10^{-7}\,\mathrm{mm^3/(N \cdot mm)}$), and fitted XRR period ($D = 4.94 \times 10^{-9}\,\mathrm{m}$).

Always extract values from attached files when a reader plugin supports your format (Reader mappings, unit factors, and standard deviations). When you need to record a condition or measurement that has no file reader (such as a substrate batch weight or a manual micrometer reading), choose **Item > Add Value**:

1. Select the **Procedure** or **Action** in the **Items** panel and choose **Item > Add Value** (or open **Item > Edit**, switch to the **Values** tab, and click **Add**).
2. Select the **Category** and **Value type**. Each value type displays its symbol, title, and stored unit (from `ValueTypes.csv`, managed in **Reference Data > Value Types...**).
3. Under **Record type**, choose **Condition** or **Result**.
4. Enter the **Value** in the stored unit (or click **Convert...** to convert from another compatible unit of the same physical quantity).
5. Enter a numeric **Tolerance** in the stored unit (with no unit suffix) and a clear **Description**, then click **Save**.

> ⚠ **Warning: value edits inside an existing item dialog save immediately**
>
> When you open an existing **Procedure** or **Action** with **Item > Edit** and add, edit, or delete a row on its **Values** tab, that value change is written to the database immediately. Clicking **Cancel** on the parent procedure/action dialog does not undo value changes already made on the **Values** tab.

Before saving any value, APS ELN checks the number and its tolerance against three plausible-range rules defined on its **Value Type**:

1. **Out of range:** the value lies below the minimum or above the maximum configured for that value type in **Reference Data > Value Types...**.

2. **Tolerance larger than the value:** the tolerance exceeds the magnitude of the value itself (for example, a value of 2.28×10^{-5} with a tolerance of 0.0602).
3. **Tolerance scale mismatch:** a non-zero tolerance is more than the configured tolerance factor (default $1000\times$) smaller than the value (for example, a value of 300 with a tolerance of 0.0001).

If any rule triggers during **Add Value** or interactive **Extract Values**, APS ELN names the issue. You can store the value anyway (**Yes**) or return to the dialog to fix the unit or value type (**No**). Automatic background extraction after **Add File** cannot prompt you, so it skips out-of-range values until you run **Extract Values** interactively on that file.

Reader mappings, unit factors, and standard deviations

Click **Extract Values** in a viewer (Chapter 7: Inside the five core instrument viewers) or right-click multiple files in the tree and choose **Extract values from N files...** (Chapter 6). APS ELN matches each key emitted by the reader plugin to a notebook **Value Type**:

- **Automatic unit conversion:** When the reader's unit and the target Value Type's stored unit measure the same physical quantity, the dialog converts the number automatically. It shows the conversion factor right above **Category**. In Study B (`261014B`), the deposition reader (`aps.deposition`) emits sputtering pressure in `mTorr`, which ELN multiplies by 0.133322 to store `Pa`. It also emits coil current in `mA` and multiplies by 0.001 to store `A`. In `aps.fibps`, wear area in $\mu\mathrm{m}^2$ converts to mm^2 ($\times 10^{-6}$).
- **Ambiguous or offset units are never guessed:** Because the letter `A` means ampere to the general unit parser, an ångström (`A`) key requires an explicit reader mapping rule (`10^-10` to `m`). Offset temperatures ($^{\circ}\mathrm{C}$ to K) are also not auto-converted.
- **Pattern rules with `<Material>`:** In **Reference Data > Value Types... > Reader mappings...**, pattern rules such as `<Material> Thickness` or `Layer <Material>.<Any>` map layer keys for any element or

compound. For `Co Thickness` and `C Thickness` in Study B, the rule places the material name (`Co` or `C`) in the value's **Description** automatically.

- **Automatic `<Key> STD` pairing:** Many readers emit both `<Key>` and `<Key> STD` with the same unit, such as `Hardness` and `Hardness STD` from `aps.mit`, `COF` and `COF STD` from `aps.zrw`, or `Step Height` and `Step Height STD` from `aps.fibps`. APS ELN multiplies the standard deviation by the same unit factor and stores it in the **Tolerance** field of the main value.

> **Tip**
>
> Keep **Remember choices for this reader** ticked at the bottom of the **Extract Values** dialog. When you click **OK**, APS ELN saves your key-to-Value-Type mappings in **Reference Data > Value Types... > Reader mappings...** so future files from that instrument map with one click.

> **Remember**
>
> An item keeps at most one `Reader` value for each key of each attached file. Extracting values from the same file a second time updates the existing `Reader` value in place instead of adding a duplicate row. Deleting an attached file automatically deletes the `Reader` values that came from that file while leaving `Manual` values untouched.

The four value sources and Derived formulas

Every value in APS ELN records how its number entered the notebook (Figure 9-1):

1. **`Reader`:** extracted from an attached file. It records the `file_id`, the file's MD5 hash, the `reader_id` and `reader_version`, the original key name, the emitted unit, and the applied conversion factor. It also stores any viewer analysis snapshot (such as ticked indents in `.mit`, step rectangles in `.fibps`, or the applied cycle interval in `.zrw`).

2. **`Derived`:** calculated by APS ELN from stored input values using a versioned formula (`formula_id`, `formula_version`, and input `value_id` list).
3. **`Manual`:** typed by hand in **Add Value**, or created when a user edits the number or tolerance of an existing `Reader`, `Derived`, or `Planned` value.
4. **`Planned`:** written onto a procedure when you create a deposition from an **Experiment Series** (**Project > Series...**, Chapter 12). It appears in grey italic (`planned`) and cannot be plotted, cited on a hypothesis, or used as an input to a `Derived` formula until you confirm it via **Confirm planned values** (or edit the value) after the run finishes.
5. **`Unknown`:** recorded in an older notebook before value-level source tracking was introduced.

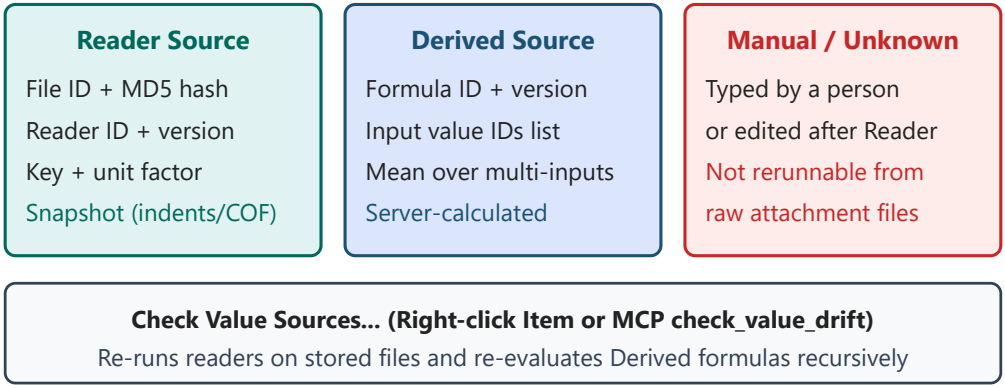


Figure 9-1: The value provenance model in APS ELN. Reader values link to an attached file, reader version, and viewer snapshot. Derived values link to a versioned formula and stored input value IDs. Check Value Sources verifies both automatically.

In ELN

When you select any value on the **Values** tab of **Item > Edit**, the source line at the bottom of the dialog shows its origin (Figure 9-2). For a `Reader` value, the **Go to file** button becomes active: clicking **Go**

> **to file** and confirming **Yes** closes the dialog and opens that exact source file in the **Content Viewer**.

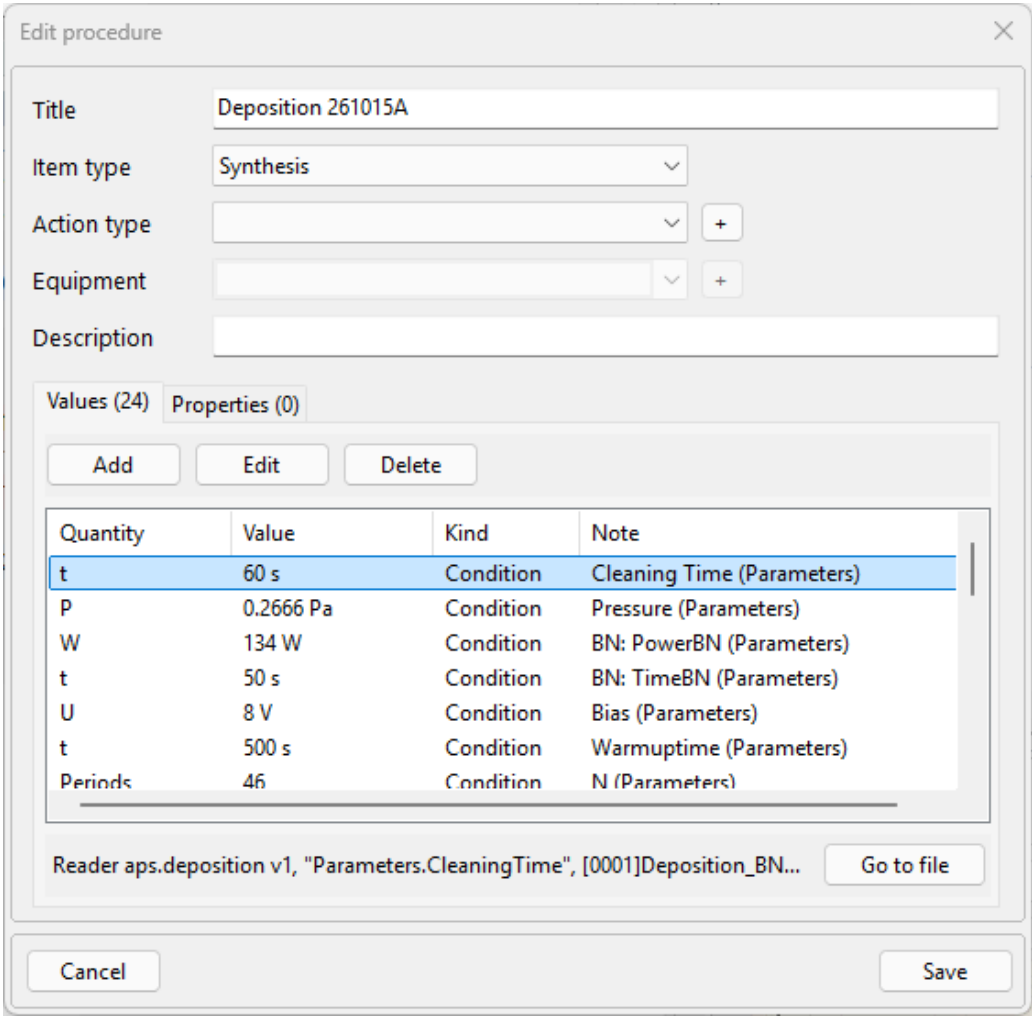


Figure 9-2: The Values tab in Item > Edit on Deposition 261015A. Selecting any value row displays its provenance source (reader ID, version, key name, and source file) at the bottom of the dialog and enables Go to file.

To compute a **Derived** value (as you did for Study A's reciprocating wear rate in Chapter 4: Step 4: Compute the Derived wear rate):

1. Right-click the **Procedure** or **Action** and choose **Add > Derived Value...**.
2. Select the formula (`wear_rate_reciprocating`, version 1: $K = A/(2 \cdot F_n \cdot N)$, returning `Wear rate` in $\mathrm{mm}^3/(\mathrm{N} \cdot \mathrm{mm})$).
3. Select the stored input value(s) for each role (`area` in mm^2, `normal_force` in N, and `cycles`) from the item or its child actions. When you select two or more `Wear area` values with `Ctrl`+click or `Shift`+click, the dialog uses their mean and records every selected `value_id` in the `Derived` source.
4. Click **Save**. You cannot type or alter the calculated number in **Add > Derived Value...**.

> ⚠ **Warning: a Derived value is a stored snapshot, not a live spreadsheet cell**
>
> If you later edit or delete one of the input values that fed a `Derived` value, APS ELN does not silently overwrite the historical `Derived` number. Instead, **Check Value Sources...** flags `input_changed` or `input_missing` so you can inspect the change and create a new `Derived` value if needed.

Checking value sources and understanding drift codes

Before you sign an experiment or cite its numbers in a paper, right-click the root **Procedure** in the **Items** panel and choose **Check Value Sources...**. You can also run `check_value_drift` through MCP in Part 5.

APS ELN re-reads every source file of every `Reader` value in the subtree using the currently installed viewer plugins, and re-calculates every `Derived` value recursively from its inputs. If all values match, a dialog reports how many values were verified with zero differences. If any value differs, a table lists the value ID, item, file, key, stored vs. current numbers, and the diagnostic code (Table 9-1).

Table 9-1: Diagnostic codes reported by **Check Value Sources...** (`check_value_drift`).

STATUS CODE	APPLIES TO	WHAT CHANGED AND HOW TO RESOLVE IT
`value_changed`	`Reader` / `Derived`	Re-running the reader or formula today gives a different number than the stored value. Re-extract or create an updated `Derived` value if the new reader/input is intended.
`reader_version_changed`	`Reader`	The installed plugin has a newer `reader_version` than the version recorded when the value was extracted (for example `aps.mit` v3 vs. v2).
`selection_changed`	`Reader`	The shared evaluation region (`.fibps`) or cycle interval (`.zrw`) on the file was modified after the value was extracted.
`source_changed`	`Reader`	The attachment bytes changed (for example via **Reupload...**) after the value's analysis snapshot was saved.
`file_missing` / `read_failed`	`Reader`	The source attachment record or its file bytes in storage could not be read; run `check_storage` or restore from backup.
`input_changed` / `input_missing`	`Derived`	One of the input values used by the formula now has a different number or was deleted.
`not_fully_rerunnable`	`Derived`	At least one input in the formula chain is `Manual` or `Unknown`, so the chain cannot be verified all the way back to raw files.

Sources

- **APS ELN Community Help:** `Values.html` (`Condition` vs. `Result`, **Item > Add Value**, immediate save warning on the **Values** tab, **Convert...** and **Tools > Unit Converter...**, the three plausible-range and $1000\times$ tolerance-factor checks, **Extract Values** and **Extract values from N files...**, **Reference Data > Value Types... > Reader mappings...**, `<Material>` pattern rules, `<Key> STD` tolerance pairing, the four value sources `Reader`, `Manual`, `Derived`, and `Unknown`, **Add > Derived Value...** with `wear_rate_reciprocating` v1 and multi-area averaging, and **Check Value Sources...** diagnostic codes); `Items.html` (the **Values** tab source line and **Go to file** button).
- **ELN provenance study (`arXiv:2610.27859`, `doi:10.5281/zenodo.23466959`):** Section 5 (design of value-level provenance and drift checking) and `zenodo/demonstration/` (`tribo_study/reports/record_report.md` and `xrr_study/reports/record_report.md` showing `check_value_drift` and `report_value_issues` outputs).

Cross-references: Chapter 1: The four links in every laboratory result; Chapter 4: Step 3: Extract Reader values; Chapter 4: Step 4: Compute the Derived wear rate; Chapter 6: Working on multiple records at once; Chapter 7: Inside the five core instrument viewers; Chapter 8: How protocol templates and AutoFill work.

Principles of Experiment Planning and the Taguchi Method

IN THIS CHAPTER

- Seeing why unplanned trial-and-error runs waste laboratory time and samples
- Connecting a testable scientific hypothesis to a structured experiment series
- Choosing between one-factor sweeps, full factorial grids, and Taguchi orthogonal arrays
- Using level means, Delta effect sizes, and factor ranks to find dominant parameters

In Parts 1 and 2 (Chapter 4 through Chapter 9), you learned how to record a single specimen tree so every stored number traces back to its raw instrument file. Real research rarely stops at one sample. You vary magnetron power, substrate bias, gas pressure, or annealing temperature across a series of depositions to find which parameter controls film thickness, hardness, wear rate, or multilayer period. This chapter explains why you should plan those runs before going to the chamber and how APS ELN uses sweeps, full grids, and **Taguchi orthogonal arrays** (Figure 10-1).

Why unplanned experiments fail in the laboratory

When students start a new coating or treatment study without a written plan, they usually fall into four laboratory traps:

1. **Changing two parameters at once:** Suppose you raise substrate bias from $50\ \mathrm{V}$ to $100\ \mathrm{V}$ and also lower argon pressure from $0.4\ \mathrm{Pa}$ to $0.2\ \mathrm{Pa}$ between two runs. You then cannot tell which change increased the film hardness.

2. **Unrecorded baseline drift:** Fixed conditions such as target-to-substrate distance, base pressure, or substrate cleaning time are easy to omit from a notebook. Without them, day-to-day drift can look like a real physical effect.
3. **Orphan runs:** When you sit down to write your thesis or paper, a project folder full of unlinked `Deposition` procedures does not show which question runs `12` through `17` tested or why run `14` was stopped.
4. **Combinatorial explosion:** A magnetron sputtering process often has four to seven adjustable knobs (power, bias voltage, working pressure, reactive gas flow, substrate temperature, and layer times). Testing 4 factors at 3 levels each by brute force requires $3^4 = 81$ depositions and 81 sets of characterization scans.

APS ELN prevents these traps with two linked planning windows under the **Project** menu:

- **The Hypothesis Tree (`Project > Hypotheses`, Chapter 11):** records *what you expect to happen and why* (`Statement`) and what changed from the parent idea (`Change`). It then links the experimental `Runs`, the cited `Evidence`, and whether the expectation was `Supported`, `Refuted`, or `Inconclusive`.
- **The Experiment Series (`Project > Series...`, Chapter 12):** generates the matrix of planned points and writes target setpoints onto each new procedure as `planned` values. It also tracks which runs are `deposited` or `measured` and plots the results across the series.

> **In ELN**
>
> Both **Project > Hypotheses** and **Project > Series...** open in their own separate windows with their own Windows taskbar buttons. They stay open while you work in the main window, so you can keep your hypothesis or series table side by side with your item tree.

Factors, levels, fixed conditions, and responses

Every structured experiment series in APS ELN is built from four parts:

- **Factors (1 to 7 varied parameters):** the experimental inputs (`Condition` value types) you step across the series, such as `Power`, `Bias`, `Pressure`, or deposition `Time`. When two factors share the same value type (such as the deposition times of two layers), give each factor a **Label** such as `Mo layer` and `Si layer`. APS ELN displays the label in brackets after the value type (`Time (Mo layer), s`).
- **Levels:** the specific numeric setpoints you test for each factor, such as `100; 200; 300` for `Power, W`.
- **Fixed conditions:** the input parameters (`Condition` value types) held constant across every run in the series, such as `Pressure` $= 0.267\,\mathrm{Pa}$ ($2\,\mathrm{mTorr}$) or `Cycles` $= 20$, with an optional tolerance.
- **Responses:** the measured outputs (`Result` value types) extracted on every point, such as step-height `Thickness` (d), `Hardness` (H_{IT}), `Wear rate` (K), or multilayer `Period` (D).

> **Remember**
>
> In APS ELN, a factor or fixed condition is a **Condition** (`EXP_CONDITION` on the **Procedure**). A response is a **Result** (`TEST_RESULT` on the procedure or its child **Actions**). Separating conditions from results during value extraction (Chapter 9) is what allows the series table to assemble itself automatically.

Sweeps, full grids, and Taguchi orthogonal arrays

Before you cut substrates or pump down the chamber, choose the point-generation method that matches the number of factors you are testing (Figure 10-1 and Table 10-1).

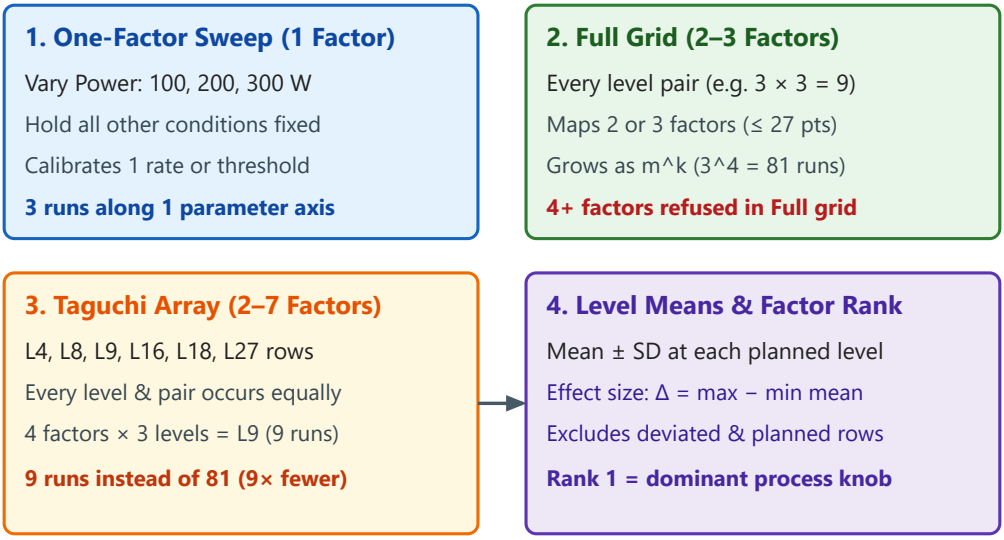


Figure 10-1: Choosing an experiment design in APS ELN. Use a one-factor sweep (1) to calibrate a single knob, a Full Grid (2) to map 2 or 3 interacting factors completely (up to 27 points), and a Taguchi orthogonal array (3) to screen 2 to 7 factors in a balanced fraction of the runs.

1. One-factor sweep (1 factor)

In a **one-factor sweep**, you vary a single factor across several levels (for example, sputtering power at $100\ \mathrm{W}$, $200\ \mathrm{W}$, and $300\ \mathrm{W}$) while holding every other condition fixed. Use a one-factor sweep when you are calibrating a deposition rate, checking thickness linearity against time, or testing a single physical variable.

2. Full factorial grid (`Full grid`, 1 to 3 factors, up to 27 points)

Two or three parameters often interact, such as magnetron `Power` and substrate `Bias` during $\mathrm{BN/DLC}$ deposition. Varying one factor at a time can then miss the best corner of the parameter window. Choosing **Full grid** generates every combination of your factor levels:

- Two factors with 3 levels each produce $3 \times 3 = 9$ plan points.

- Three factors with 3 levels each produce $3 \times 3 \times 3 = 27$ plan points.

A full grid grows exponentially as m^k, where k is the number of factors and m is the number of levels. Four factors at 3 levels would require $3^4 = 81$ depositions. APS ELN therefore limits **Full grid** to at most **3 factors** and **27 points**. When you add a third factor in the plan dialog, APS ELN switches **Point generation** to **Taguchi orthogonal array** automatically.

3. Taguchi orthogonal arrays (`Taguchi orthogonal array`, 2 to 7 factors)

Developed by Genichi Taguchi for engineering process optimization, an **orthogonal array** denoted L_N is a fractional factorial table with N rows (depositions) and k columns (factors). APS ELN includes six standard orthogonal arrays covering 2 to 7 factors (Table 10-1).

Table 10-1: The six Taguchi orthogonal arrays built into APS ELN (`Project > Series...` and MCP `create_series`).

TAGUCHI ARRAY	NUMBER OF FACTORS (K)	LEVELS PER FACTOR	PLANNED RUNS (N)	FULL-GRID RUNS FOR MAX K
L4	2 or 3 factors	2 levels each	4 runs	8 runs (2³)
L8	4 to 7 factors	2 levels each	8 runs	128 runs (2⁷)
L9	2 to 4 factors	3 levels each	9 runs	81 runs (3⁴)
L16	2 to 5 factors	4 levels each	16 runs	1,024 runs (4⁵)

TAGUCHI ARRAY	NUMBER OF FACTORS (K)	LEVELS PER FACTOR	PLANNED RUNS (N)	FULL-GRID RUNS FOR MAX K
L18	2 to 7 factors (mixed)	One 2-level factor (at any position) + 1 to 6 three-level factors	18 runs	1,458 runs (2 × 3⁶)
L27	5 to 7 factors	3 levels each	27 runs	2,187 runs (3⁷)

What makes an array like L9 **orthogonal** is its two-way balance across all columns (Table 10-2):

1. **Equal level frequency:** In the 9 rows of L9, each of the three levels (1, 2, 3) of every factor appears **exactly 3 times**.
2. **Equal pair frequency:** Take any two factors in Table 10-2, such as Power (Factor A) and Bias (Factor B). Every possible ordered pair of levels ((Low, Low), (Low, Med), (Low, High), (Med, Low), …, (High, High)) appears **exactly once** among the 9 runs.

Table 10-2: Worked L9 Taguchi orthogonal array for a 4-factor, 3-level reactive sputtering series (9 runs instead of 81).

RUN (POINT)	FACTOR A: POWER (W)	FACTOR B: BIAS (V)	FACTOR C: PRESSURE (PA)	FACTOR D: N2 FLOW (SCCM)
Point 1 (1, 1, 1, 1)	100 W (Level 1)	40 V (Level 1)	0.2 Pa (Level 1)	2 sccm (Level 1)
Point 2 (1, 2, 2, 2)	100 W (Level 1)	80 V (Level 2)	0.4 Pa (Level 2)	5 sccm (Level 2)
Point 3 (1, 3, 3, 3)	100 W (Level 1)	120 V (Level 3)	0.6 Pa (Level 3)	8 sccm (Level 3)

RUN (POINT)	FACTOR A: POWER (W)	FACTOR B: BIAS (V)	FACTOR C: PRESSURE (PA)	FACTOR D: N2 FLOW (SCCM)
Point 4 (2, 1, 2, 3)	200 W (Level 2)	40 V (Level 1)	0.4 Pa (Level 2)	8 sccm (Level 3)
Point 5 (2, 2, 3, 1)	200 W (Level 2)	80 V (Level 2)	0.6 Pa (Level 3)	2 sccm (Level 1)
Point 6 (2, 3, 1, 2)	200 W (Level 2)	120 V (Level 3)	0.2 Pa (Level 1)	5 sccm (Level 2)
Point 7 (3, 1, 3, 2)	300 W (Level 3)	40 V (Level 1)	0.6 Pa (Level 3)	5 sccm (Level 2)
Point 8 (3, 2, 1, 3)	300 W (Level 3)	80 V (Level 2)	0.2 Pa (Level 1)	8 sccm (Level 3)
Point 9 (3, 3, 2, 1)	300 W (Level 3)	120 V (Level 3)	0.4 Pa (Level 2)	2 sccm (Level 1)

Look at Points 1, 2, and 3 in Table 10-2, where `Power` is held at Level 1 ($100\ \mathrm{W}$). Across those three runs, `Bias` visits Levels 1, 2, and 3; `Pressure` visits Levels 1, 2, and 3; and `N2 flow` visits Levels 1, 2, and 3. The same balance holds for the three runs at $200\ \mathrm{W}$ (Points 4–6) and the three runs at $300\ \mathrm{W}$ (Points 7–9). When you compare the average hardness at $100\ \mathrm{W}$, $200\ \mathrm{W}$, and $300\ \mathrm{W}$, the main effects of the other three factors cancel out cleanly.

> **Tip**
>
> Suppose one factor has 2 levels (such as substrate heater `Off` vs. `On`), while up to 6 other factors have 3 levels each. APS ELN selects the mixed **`L18`** array automatically. You can put the 2-level factor at any position in your factor list.

Reading main effects: level means, Delta, and factor rank

Once you run the depositions in an orthogonal array and extract the response values, open the **Plot** tab of **Project > Series...** (Chapter 12). Tick **Level means (Taguchi)** to evaluate each factor.

For a chosen plan factor X and response Y, APS ELN calculates three quantities:

1. **The level mean ($\bar{Y}_{X=L}$):** For each planned level L of factor X, ELN averages the confirmed response values $Y_1, \ldots, Y_{n_L}$ recorded at that level:

$$\bar{Y}_{X=L} = \frac{1}{n_L} \sum_{i=1}^{n_L} Y_i$$

2. **The level spread (error bar s_L):** When $n_L \geq 2$ runs were measured at level L, the vertical error bar on the plot is the sample standard deviation across those runs:

$$s_L = \sqrt{\frac{1}{n_L - 1} \sum_{i=1}^{n_L} \left(Y_i - \bar{Y}_{X=L} \right)^2}$$

When only $n_L = 1$ run has been measured at level L, the error bar shows that single value's stored tolerance.

3. **The effect size (Δ_X) and factor rank:** Below the result list on the **Plot** tab, APS ELN displays **Delta** (Δ_X). It equals the largest level mean minus the smallest level mean of factor X:

$$\Delta_X = \max_L \left(\bar{Y}_{X=L} \right) - \min_L \left(\bar{Y}_{X=L} \right)$$

Alongside Δ_X, APS ELN shows the factor's **rank** among all k factors in the plan, such as `Delta = 10 nm (rank 1 of 3)` or `Delta = 2.4 GPa (rank 1`

`of 4`). **Rank 1** is the factor with the largest Δ: the parameter that produces the largest change in the measured response across its tested levels. Factors with equal Δ share the same rank.

> ⚠ **Warning: deviated setpoints and unconfirmed planned values are excluded from level means**
>
> Orthogonal cancellation requires each run to sit at its planned factor level within tolerance. When a deposition's stored factor value differs from its planned level beyond tolerance, APS ELN excludes that run from the factor's level means. The plot note then displays `n point(s) differ from the plan and are left out of level means.` Unconfirmed grey italic `planned` values are also excluded until you confirm them.

> ☰ **Going Deeper: when Taguchi screening is enough and when to run a follow-up Full Grid**
>
> Taguchi level-means analysis is a **main-effects screening method**. It assumes that the direct effect of each factor is larger than two-factor interactions (where the effect of Factor A reverses direction depending on Factor B). In addition, APS ELN's **Level means (Taguchi)** plot computes direct response means and Δ ranks rather than logarithmic signal-to-noise (S/N) ratios or an analysis of variance (ANOVA).
>
> This makes a two-stage workflow very effective in practice:
>
> 1. **Stage 1 (Screening with Taguchi `L9` or `L18`):** Propose your initial screening hypothesis (Chapter 11) and run a 9-point `L9` series across 4 candidate parameters (Chapter 12). Use **Level means (Taguchi)** to identify the Rank 1 and Rank 2 factors, and fix the low-impact Rank 3 and Rank 4 factors at their best levels.
> 2. **Stage 2 (Mapping with a 2-factor Full Grid):** Propose a **New follow-up...** hypothesis and create a 2-factor **Full grid** series over just the Rank 1 and Rank 2 factors. You can also export the

> table to OriginLab via **Copy table** or **Save CSV...** to map their response surface in detail.

Sources

- **APS ELN Community Help:** `Series.html` (planning an Experiment Series, 1 to 7 **Factors** with labels and semicolon-separated **Levels**, **Point generation** options **Full grid**, **Taguchi orthogonal array** `L4` / `L8` / `L9` / `L16` / `L18` / `L27`, and **No initial points**, **Fixed conditions**, **Responses**, **Level means (Taguchi)** on the **Plot** tab, **Delta** and factor rank calculation, exclusion of deviated runs and unconfirmed `planned` values, and absence of S/N ratios or ANOVA); `Hypotheses.html` (**Project > Hypotheses** separate window and campaign structure).
- **APS ELN Delphi source:** `ELN.Model/unit_SeriesTaguchi.pas` (`MAX_SERIES_FACTORS = 7`, `MAX_GRID_FACTORS = 3`, `MAX_GRID_POINTS = 27`, `L4`, `L8`, `L9`, `L16`, `L18`, and `L27` orthogonal array definitions, `SelectTaguchiArray`, `BuildTaguchiPoints`, `BuildGridPoints`, `SummarizeLevels`, and `ComputeFactorEffects`).

Cross-references: Chapter 4: Your First Sample Record in Twenty Minutes; Chapter 9: Conditions, Results, units, and plausible-range checks; Chapter 11: The Hypothesis Tree and Evidence; Chapter 12: Experiment Series and Taguchi Arrays in ELN.

The Hypothesis Tree and Evidence

IN THIS CHAPTER

- Opening the standalone Hypotheses window and proposing a root or follow-up hypothesis
- Comparing the Community self-review workflow with Commercial two-gate supervisor review
- Attaching experimental Runs and References, promoting a Reference with Make run, and citing Evidence
- Closing or accepting a conclusion and verifying the cryptographic content seal

Recording procedures, specimens, actions, files, and values (Chapter 6 and Chapter 9) tells you *what* you made and *what numbers* you measured. By itself, the item tree does not explain *why* you changed the bias voltage in a new run or *which* scientific question that sample series tested. The **Hypothesis Tree** ties your research question to the exact runs, files, and values that tested it.

Why connect experiments to a hypothesis tree?

A **hypothesis** in APS ELN sits inside a project and links four things (Figure 11-1):

1. **The Proposal:** what you expect to happen and why (`Statement`), plus what changed compared to the parent hypothesis (`Change`) when you are refining an earlier idea.
2. **The Runs and References:** the experimental **Procedures**, **Specimens**, or **Actions** you ran to test the hypothesis (`Run`), or earlier records from any readable project that provide background context (`Reference`).

3. **The Cited Evidence:** the specific items, attached files (`.mit`, `.zrw`, `.xrcx`), or stored values (H_{IT}, COF, K, D) that support or refute the expectation.
4. **The Conclusion:** what you found (`What we found`) and the outcome (`Supported`, `Refuted`, or `Inconclusive`).

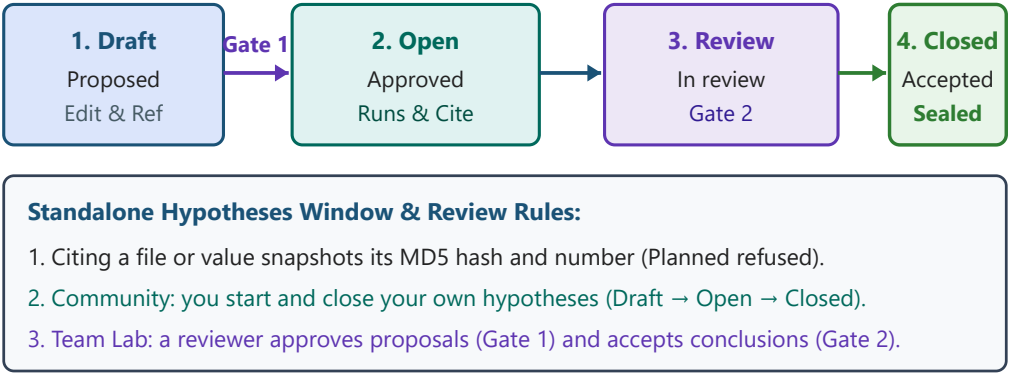


Figure 11-1: The lifecycle of a hypothesis in APS ELN. In Community, you start and close your own hypotheses locally (Draft -> Open -> Closed). In the Team edition, a reviewer approves the proposal at Gate 1 and accepts the conclusion at Gate 2.

A root hypothesis can have **follow-up** hypotheses nested beneath it. Across a multi-month research campaign, the tree shows which ideas were supported, which were refuted, and how each result led to the next experiment (Chapter 10).

Proposing a hypothesis and starting or approving work

Select your active project in the **Projects** panel and choose **Project > Hypotheses** (or click the **Hypotheses** button on the Projects toolbar). The hypotheses open in a **separate top-level window** with its own Windows taskbar button. The window stays open while you work in the main window and remembers its size and position for your next session.

In **APS ELN Community**, you are the only reviewer, so a hypothesis moves **Draft** → **Open** → **Closed** with no separate supervisor approval gate. In the Commercial ("For Teams") edition, a reviewer with the right role approves proposals and accepts conclusions (**Proposed** → **Approved** → **In review** → **Accepted**; see Table 11-1).

Follow these steps to propose and start or approve a hypothesis:

1. Click **New hypothesis...** to create a root hypothesis, or select an existing hypothesis and click **New follow-up...** to propose a child refinement beneath it.
2. Enter a clear **Title** and **Statement - what you expect, and why** (labeled **We expected** on the **Proposal** tab). For a follow-up hypothesis, also fill in **Change - what differs from the parent hypothesis** (labeled **What changes**).
3. Optionally select a **headline quantity** (such as `Wear rate` or `Period`). APS ELN summarizes the count, mean, minimum, and maximum of that value across all attached `Run` items right in the hypothesis tree.
4. Click **OK**. Read-only proposal and conclusion text boxes are tinted with the row's status color so you can tell draft, active, and closed records apart at a glance. While the hypothesis is a `Draft` (`Proposed`), its author can click **Edit...** on the **Proposal** tab to refine the wording and can attach background **Reference** items.
5. In **APS ELN Community**, click **Start** to make the `Draft` **Open** so you can attach experimental runs, or click **Discard** (with an optional reason) to drop it. In the Commercial edition, a reviewer uses the **Your decision** panel to click **Approve** or **Reject** (which requires a comment).

> **In ELN**
>
> Selecting a parent hypothesis row with follow-ups reveals the same three row buttons used in the **Projects** and **Items** panels (Chapter 3: Reading the item tree and its row buttons). Use them to **Collapse all below**, **Expand all below**, or **Focus**.

> **In a Team Lab: who approves proposals and conclusions in a team lab**
>
> In **APS ELN Community**, you start and close your own hypotheses locally inside the Desktop GUI. In the Commercial ("For Teams") edition, an administrator or supervisor configures **Administration > Approval Policy...** (including an optional **Four-eyes** rule that prevents anyone from approving their own hypothesis). They can also tick **Can approve proposals** and **Can accept conclusions** on a project's **Share...** dialog (Chapter 5: Sharing projects in a team laboratory). By default, a student with a `Standard` role submits proposals and conclusions. Their supervisor sees them in the Dashboard's **Waiting for You** list (Chapter 3) and in the mobile web app's **Waiting for me** tab (Chapter 8). Projects that hold hypotheses display a **HYP** badge in the project tree, which turns solid when a decision is waiting on you.

Attaching Runs, promoting References, and citing Evidence

Once a hypothesis is **Open** (Commercial: **Approved**), its status line shows where the laboratory work stands (Figure 11-2 and Table 11-1).

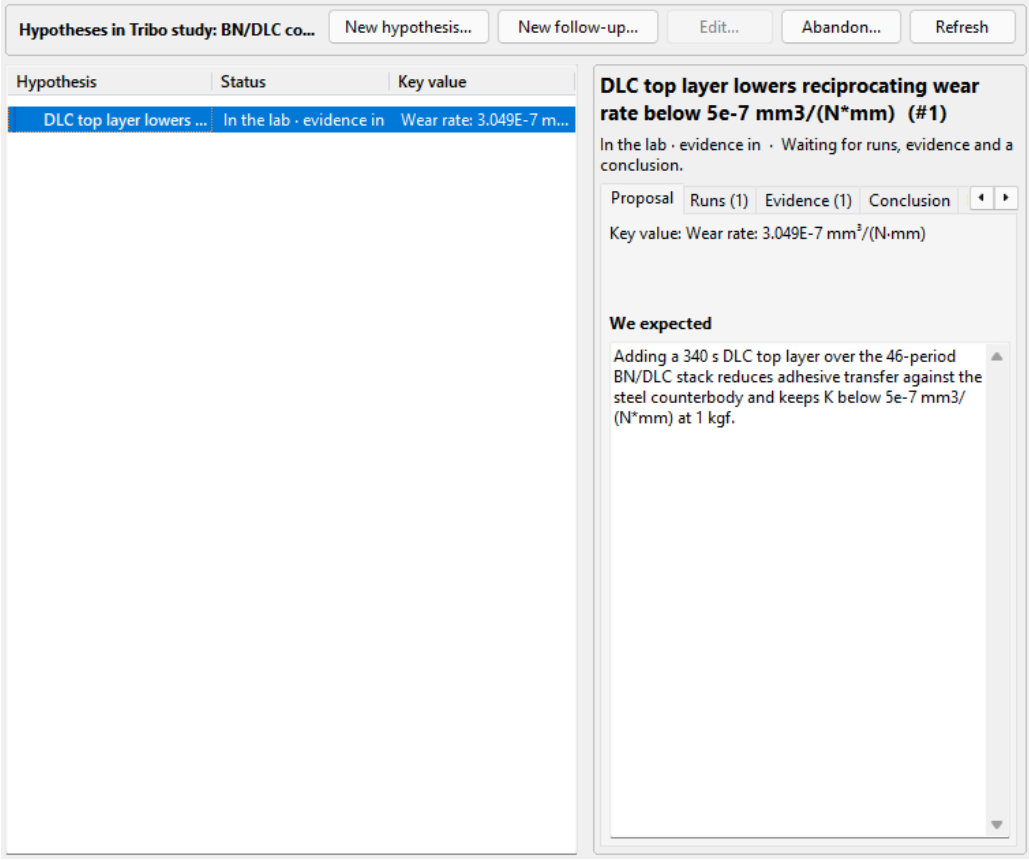


Figure 11-2: The Project > Hypotheses view in APS ELN Desktop. The left pane lists each hypothesis with its derived lab state and headline key-value summary; the right pane organizes the Proposal, attached Runs, cited Evidence, and Conclusion.

To connect your experimental items in the main window to the open **Hypotheses** window:

1. Select the **Procedure**, **Specimen**, or **Action** in the **Items** panel of the main window (for example, `Deposition 261015A` in Study A).
2. Right-click the item in the item tree and choose **Hypothesis > Attach to hypothesis...**. Or switch to the **Hypotheses** window, select the hypothesis, open the **Runs** tab, and click **Attach selected item...**.
3. Choose **Run** (an experimental item in this project made to test the hypothesis) or **Reference** (existing context from any readable project)

and click **OK**. Items attached as runs display a **HYP** badge in the item tree.

4. Suppose you attached an item from the same project as a **Reference** while the hypothesis was still a `Draft`. Once the hypothesis is **Open**, select that row on the **Runs** tab and click **Make run** to promote it to a `Run`.

> **Tip**
>
> If an unexpected result in an existing experiment sparks a new idea, right-click that item in the **Items** panel and choose **Hypothesis > New hypothesis from this item...**. APS ELN opens the new-hypothesis dialog with that item already attached as a `Reference` from the very first draft.

After your measurements and value extractions are complete, cite the specific records that test the hypothesis:

1. In **Project > Hypotheses**, select the approved hypothesis, switch to the **Evidence** tab, and click **Add evidence...**.
2. In the left list of the dialog, select an attached `Run` or `Reference` (or any indented child action beneath a run, such as `Indentation 261015A` or `Friction test 261015A`). You do not need to attach every child action as a separate `Run` to cite its files or values.
3. In the right list, pick what you are citing. You can cite the **item as a whole**, one of its **attached files** (such as `BN-DLC.zrw` or `XRR-fit_Co-C(261014B).xrcx`), or one of its **stored values** ($K = 3.05 \times 10^{-7}\,\mathrm{mm}^3/(\mathrm{N}\cdot\mathrm{mm})$).
4. Type a short note explaining why this evidence matters and click **OK**.

> **Remember**
>
> When you cite a file or a value, APS ELN records the file's MD5 checksum or the value's stored number on the citation. If someone later replaces the file or edits the value, the **Evidence** tab flags that

> row as `changed` (or `missing` if it was deleted). You can see right away if cited evidence changed after the fact.

Concluding, closing or accepting, and checking the content seal

When your runs and cited evidence are in place, you are ready to write the conclusion and seal the record (Table 11-1):

1. Select the hypothesis and open the **Conclusion** tab.
2. In **What we found**, write your summary of the results (you can include Markdown and `[[item:N]]` links).
3. Under **Outcome**, select **Supported**, **Refuted**, or **Inconclusive**. At least one cited piece of evidence is required.
4. **In APS ELN Community:** click **Close with conclusion** to move the `Open` hypothesis to `Closed · Supported`, `Closed · Refuted`, or `Closed · Inconclusive` and seal it in one step. (If an AI agent submitted the conclusion via MCP, the status shows `Open · conclusion ready`; click **Close** to seal it. Clicking **Edit conclusion** reopens a closed hypothesis.)
5. **In the Commercial ("For Teams") edition:** click **Submit conclusion** to move the hypothesis to `In review`. In **Your decision**, the reviewer inspects the runs, evidence, and conclusion text. The reviewer then clicks **Accept** (final seal) or **Return for more work** (requires a comment and returns the hypothesis to `Approved`).

Table 11-1: Hypothesis status names across MCP JSON, Community, and Commercial ("For Teams") editions.

MCP / JSON STATUS	COMMUNITY LABEL	COMMERCIAL ("FOR TEAMS") LABEL	WHAT IS ALLOWED IN THIS STATE
`Proposed`	**Draft**	**Proposed** (`Needs approval`)	Author can edit the proposal and attach `Reference` items. Click **Start** (Community) or **Approve** / **Reject** (Commercial).
`Approved`	**Open**	**Approved** (`no runs yet` / `no evidence yet` / `evidence in`)	Attach `Run` and `Reference` items, promote a reference with **Make run**, and cite items, files, or confirmed values on **Evidence**.
`InReview`	**Open · conclusion ready**	**In review** (`Needs acceptance`)	Conclusion submitted (via MCP in Community, or **Submit conclusion** in Commercial). Click **Close** (Community) or **Accept** / **Return for more work** (Commercial). Cited items are protected from deletion.
`Accepted`	**Closed · Supported** / **Refuted** / **Inconclusive**	**Accepted**	Final **Sealed** record. In Community, **Edit conclusion** returns it to `Open`; in Commercial, any later revision is a **New follow-up...** child hypothesis.
`Rejected` / `Abandoned`	**Discarded** / **Abandoned**	**Rejected** / **Abandoned**	Proposal was discarded or rejected at the first gate, or an open hypothesis was stopped with **Abandon...** and a reason.

> ⚠ **Warning: Planned values from an Experiment Series cannot be cited as evidence**
>
> A grey italic `Planned` value written onto a procedure by **Project > Series...** (Chapter 12) is excluded from the hypothesis headline key-value summary and cannot be cited as evidence (`planned_value`). Likewise, **Close** and **Accept** refuse to seal a hypothesis while any cited value is still `Planned`; confirm the value on its item first.

> ⚠ **Warning: AI agents cannot start, approve, close, or accept a hypothesis**
>
> An AI assistant connected through the MCP server (Part 5) can propose a hypothesis (`propose_hypothesis`), attach runs (`attach_run`), cite evidence (`cite_evidence`), and submit a conclusion (`submit_conclusion`). The MCP server has no tool to start, discard, approve, reject, close, or accept a hypothesis. Those decisions must always be made by a person in the Desktop GUI (or the Team Lab web app).

> **Going Deeper: what the Closed / Accepted content seal covers**
>
> When you click **Close with conclusion** in Community (or **Accept** in the Team edition), APS ELN computes a cryptographic checksum over the hypothesis record. This checksum covers the statement, change description, conclusion text, outcome, attached runs, and cited evidence, and displays **Sealed** on the record. If any underlying record is tampered with or if a cited file or value drifts after sealing, the view warns that the record or its evidence changed. In addition, any item cited by a hypothesis in `InReview` or `Accepted` (`Closed`) is protected from deletion, even when emptying the Recycle Bin (Chapter 5).

Sources

- **APS ELN Community Help:** `Hypotheses.html` (**Project > Hypotheses** separate top-level window with taskbar button and saved geometry, `Draft` -> **Start** / **Discard** -> `Open` -> **Close with conclusion** / **Close** -> `Closed · Supported` / `Refuted` / `Inconclusive`, **Edit conclusion**, status mapping table between MCP JSON, Community, and Commercial, the three row buttons in the hypothesis list, **Runs** tab with **Attach selected item...** and **Make run**, **Evidence** tab refusal of `Planned` values, and **Abandon...**).
- **APS ELN Commercial ("For Teams") Help:** `Experiments.html` (`Proposed` -> **Approve** / **Reject** -> `Approved` -> **Submit conclusion** -> `In review` -> **Accept** / **Return for more work** -> `Accepted`, derived status lines, **HYP** badges in the project and item trees, right-clicking an item for **Hypothesis > Attach to hypothesis...** and **Hypothesis > New hypothesis from this item...**, **Administration > Approval Policy...** and **Four-eyes**, **Sealed** checksums and `changed` / `missing` evidence flags, deletion protection for cited items, and MCP agent boundaries).

Cross-references: Chapter 3: The Dashboard and keyboard navigation; Chapter 5: Sharing projects in a team laboratory; Chapter 5: Deleting containers and using the Recycle Bin; Chapter 8: Submitting protocols from the instrument; Chapter 10: Principles of Experiment Planning and the Taguchi Method; Chapter 12: Experiment Series and Taguchi Arrays in ELN; Chapter 16: Completeness, Audit, and Signatures.

Experiment Series and Taguchi Arrays in ELN

IN THIS CHAPTER

- Planning a 1-to-7-factor Experiment Series in the standalone Series window
- Generating points with Full grid, Taguchi orthogonal arrays (L4 to L27), or manual points
- Making or linking depositions, confirming planned values, and handling failed runs
- Analyzing sweeps and Level means (Taguchi) on the Plot tab and exporting to OriginLab

After choosing your factors, fixed conditions, responses, and design (Chapter 10) and recording your hypothesis (Chapter 11), build the plan in **Project > Series...**. The Series window generates your sweep or Taguchi array and creates or links the deposition procedures. It also tracks measured points, plots raw curves or Taguchi level means, and exports the table to OriginLab (Figure 12-1).

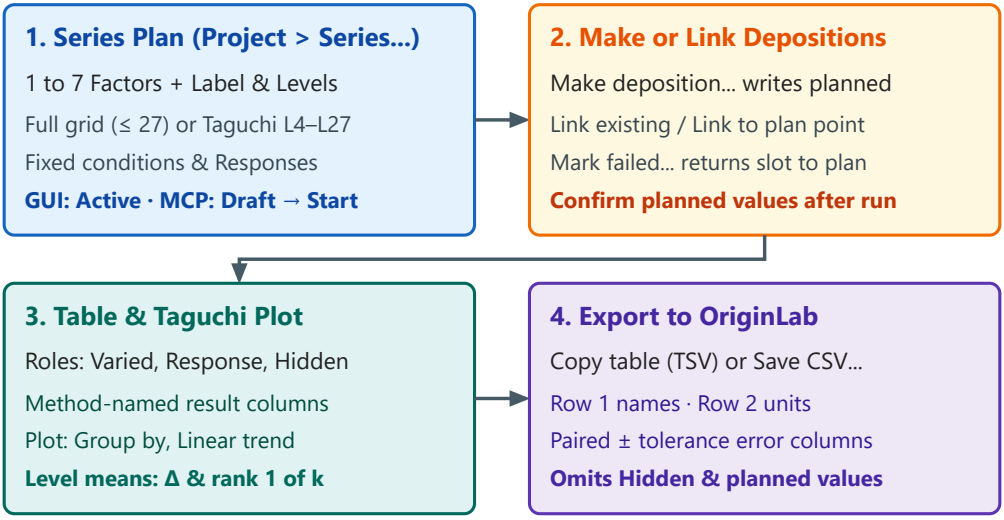


Figure 12-1: The four stages of working with an Experiment Series in APS ELN. Plan the factor levels or Taguchi array (1), make or link depositions and confirm planned values (2), inspect the Table and Taguchi level-means plot (3), and export with paired tolerance columns to OriginLab (4).

Planning a series in the Series window

Select your project in the **Projects** panel and choose **Project > Series...** (or click **Series** on the Projects toolbar). The **Series** view opens in a separate top-level window with its own taskbar button, so it stays open while you work in the main window.

Follow these steps to create a new series plan:

1. Click **New series...** to open the **Series plan** dialog.
2. Under **Series - what it tests and where its depositions go**, enter the **Title** and **Purpose - what this series tests**. Then choose the **Target folder** where new depositions created from this plan will be placed.
3. Under **Factors - parameters you vary (up to 7)**, click **Add...** for each factor (1 to 7):

- Choose a **Category** and a **Value type** (choose **(All)** to list every value type).
- Type an optional **Label** (such as `Mo layer` or `Si layer`) so headers distinguish two factors that share the same value type (`Time (Mo layer), s`).
- Enter optional **Range** minimum and maximum limits; APS ELN refuses any level or point outside that range.
- Enter the factor's semicolon-separated **Levels** (for example, `100; 200; 300`), then click **OK**. The **Levels** column of the factor list displays your entered levels.

4. Under **Point generation**, select how APS ELN builds the plan points:
   - **Full grid:** creates every combination of the factor levels (1 to 3 factors, up to 27 points; a grid of 4 or more factors is refused).
   - **Taguchi orthogonal array:** builds the rows of the matching standard array (`L4`, `L8`, `L9`, `L16`, `L18`, or `L27`; see Chapter 10: Table 10-1). When you add a third factor, APS ELN selects this option automatically.
   - **No initial points:** starts the series with an empty table so you can add points one by one with **Add point...**.
5. Check the summary line under **Point generation**. With 1 or 2 factors, you can type semicolon-separated levels directly in the **Levels** boxes. With 3 to 7 factors, select a factor and click **Edit...** to change its levels. The summary line shows the selected array and point count (such as `Taguchi L9: 9 points.`), or explains why the current level counts do not match an array.
6. Under **Fixed conditions**, click **Add...** for each parameter held constant across the series. Pick its category and value type, enter an optional label, and type its **Value** and optional **Tolerance**.
7. Under **Responses**, click **Add...** for each result value type (`TEST_RESULT`) you will measure on every point, with an optional label (such as `Total stack`).
8. Click **OK**. If any entry has an error, APS ELN explains what is wrong and keeps your entries in the dialog so you can fix them.

> **Tip**
>
> Every number field in the series dialogs (`Range`, `Levels`, `Fixed conditions`, `Tolerance`, and `Add point...`) accepts inline unit conversion. For example, if a pressure value type is stored in Pa, you can type `2 mTorr` directly into the field. APS ELN converts the number to Pa and previews the converted value right below the box. Note that a **Tolerance** cannot use an offset temperature unit such as `°C` on a value type stored in K.

A series created in the Desktop window starts as **Active** immediately. A series created by an AI agent through MCP (`create_series` in Chapter 20) starts as **Draft** so you can review it and click **Start**. While a series has points, **Edit plan...** changes the title, purpose, target folder, factor labels, ranges, fixed conditions, and responses. To add, remove, or change factors, remove all points first. When all runs are finished, click **Close** (author only), or click **Abandon...** to stop work while keeping the history.

Making or linking depositions and confirming planned values

The **Table** tab shows one row for each planned point from the start, and the left list reports progress such as `2/9 measured`. You can populate those rows either by creating new depositions from the table or by linking procedures you already recorded:

1. **Make a new deposition from a planned row:** Double-click a `planned` row (or right-click it and choose **Make deposition...**). APS ELN opens the new-procedure dialog in the target folder with a pre-filled title, item type `Synthesis`, and the action type and equipment of the previous deposition. Click **OK**. APS ELN creates the procedure and writes each factor level and fixed condition onto it as a condition value marked `planned` (shown in grey italic).
2. **Link an existing procedure from the Series table:** Right-click a `planned` row and choose **Link existing deposition...**. Then pick a

top-level procedure from the same project that you recorded before making the plan.

3. **Link one or more procedures from the Item tree:** In the main window's **Items** panel, select one or more top-level procedures, right-click, and choose **Link to plan point...**. The dialog previews each match: `item -> 200 W` (matches a free point within tolerance), `item -> new point 250 W` (no point matches), or the refusal reason (such as `no Power value`). Tick **Add new points for depositions that match no point** if you want unmatched levels added to the plan as `added` points, then click **Link**.
4. **Add an extra point at any time:** Click **Add point...** on the Series toolbar. Enter one level per factor (within range and not duplicating an existing point) and an optional reason. On an `Active` series, ELN immediately opens the new-procedure dialog for that point and marks the row `added` in the **Point** column.

> **⚠ Warning: always confirm planned values after the deposition finishes**
>
> A grey italic `planned` value is a target from the plan, not a confirmed measurement. The **Plot** tab and CSV export ignore `planned` values, **Add > Derived Value...** refuses them as formula inputs, and **Project > Hypotheses** refuses to cite them. When your deposition ran as planned, right-click its row in the Series table (or use the **Item** menu) and choose **Confirm planned values**. If the actual setpoint differed, edit the value on the procedure or extract the real setpoint from your `.depo.json` file; both actions clear the `planned` mark.

When a confirmed factor value or fixed condition differs from the plan by more than tolerance, its table cell shows a warning icon with the tooltip `Differs from the plan (planned value)`. If a deposition attempt fails in the chamber (for example, a substrate breaks or a target arcs), right-click its row and choose **Mark failed...** with a reason. The point returns to `planned` so you can run **Make deposition...** for the next attempt, and ticking **Show failed** on the toolbar displays failed attempts in the table.

Reading the Series Table and setting column roles

Each row and column on the **Table** tab has a specific meaning (Table 12-1):

- **Point and State columns: Point** shows the factor levels with their units (plus `added` for points appended after the initial plan). **State** shows `planned` (no deposition yet), `deposited` (has a deposition, waiting for one or more response measurements), `measured` (all responses present with confirmed values), or `failed`. Point at any **Point** or **State** cell to see the linked procedure title, or double-click any data cell to open the item that holds that value in the main window.
- **Separate columns per measurement method:** A response value often sits on a child measurement action below the deposition, such as film thickness measured by `XRR` or `3D Profiling`. Each measurement action type gets its own column named with the method. APS ELN never averages values across different measurement methods.
- **Ambiguous cells:** Sometimes the same measurement method under one deposition produces more than one value (such as two separate specimens under the same procedure). The cell then shows all values with a warning icon and omits that cell from the plot rather than guessing which one to use.
- **Column roles:** Right-click any data column header to set its role to **Varied**, **Response**, **Shown**, or **Hidden**. `Hidden` columns remain visible in grey in the table for reference, and are left out of the plot and the OriginLab export.

Table 12-1: Statuses, row states, cell warnings, and column roles in **Project > Series...**.

SERIES ELEMENT	VALUES IN PROJECT > SERIES...	BEHAVIOR IN THE TABLE, PLOT, AND ORIGINLAB EXPORT
Series status	`Draft`, `Active`, `Closed`, `Abandoned`	GUI series start `Active`; MCP `create_series` starts `Draft` until you click **Start**. **Close** marks the series complete (author only).
Row state	`planned`, `deposited`, `measured`, `failed`	`planned` has no run; `deposited` has a run missing a response; `measured` has all responses recorded; `failed` is shown when **Show failed** is ticked.
Cell warning icon	`Differs from the plan` or `Several values`	Flags a factor/fixed value outside tolerance (excluded from Taguchi level means) or multiple values from the same method (not plotted).
Column role	`Varied`, `Response`, `Shown`, `Hidden`	Right-click any column header to change its role. `Hidden` columns stay grey in the table and are omitted from the plot and CSV export.
Excluded runs	Status-bar notice + **Unlink excluded**	Depositions moved to the Recycle Bin or another project are omitted from the table; click **Unlink excluded** to remove them from the series.

Plotting sweeps, Taguchi level means, and exporting to OriginLab

Switch to the **Plot** tab in the **Series** window to visualize your results as measurements come in:

1. Under **X (condition)**, choose the condition column for the horizontal axis (defaults to the first `Varied` column).

2. Under **Y (results)**, tick one or more result columns (defaults to every `Response` column). Vertical error bars show the stored tolerance of each result value.
3. For a 1-factor or 2-factor sweep, you can choose **Group by (condition)** to draw a separate curve for each level of a second condition (up to 8 groups). You can also tick **Linear trend** when a single curve is shown.
4. For a Taguchi array or multi-factor plan, tick **Level means (Taguchi)**. This mode is available whenever **X (condition)** is a factor of the plan (**Group by** and **Linear trend** are disabled):
    - For each planned level of X, the chart plots the mean of the confirmed result values at that level (Chapter 10).
    - With 2 or more values at a level, the vertical error bar is the sample standard deviation; with 1 value, it is that value's stored tolerance.
    - Read the note below the **Y (results)** list. It reports **Delta** (largest level mean minus smallest level mean) and the **rank** of the X factor among all plan factors, such as `Delta = 10 nm (rank 1 of 3)`. Switch **X (condition)** through each factor in turn to compare their level-mean curves and ranks.

> **In ELN**
>
> When a plotted row holds unconfirmed `planned` values, the Plot note warns `n point(s) wait for confirmed values`. When a factor setpoint missed its planned level beyond tolerance, the note warns `n point(s) differ from the plan and are left out of level means`.

When you are ready to prepare publication figures or run multi-factor surface fits in OriginLab or Python, use the two export buttons on the Series toolbar:

- **Copy table:** copies the series table to the clipboard as tab-separated text, ready to paste directly into an Origin worksheet.
- **Save CSV…:** writes the same table to a `.csv` file using a period (`.`) as the decimal mark.

In both exports, Row 1 holds the column names and Row 2 holds the units. Every value column with a tolerance is followed by its own `±` column so OriginLab can assign error bars in one click. `Hidden` columns, `planned` rows, and unconfirmed `planned` values are omitted automatically.

Sources

- **APS ELN Community Help:** `Series.html` (**Project > Series...** separate window, **New series...**, 1 to 7 **Factors** with **Label**, **Range**, and semicolon-separated **Levels**, inline unit conversion and offset-unit tolerance rule, **Point generation** modes **Full grid**, **Taguchi orthogonal array** `L4` / `L8` / `L9` / `L16` / `L18` / `L27`, and **No initial points**, **Fixed conditions**, **Responses**, `Draft` vs. `Active` status, **Make deposition...**, **Link existing deposition...**, **Link to plan point...**, **Confirm planned values**, `Differs from the plan` warning, **Add point...**, **Mark failed...** and **Show failed**, **Unlink excluded**, **Table** column roles, **Plot** tab with **Group by**, **Linear trend**, and **Level means (Taguchi)** showing **Delta** and rank, and **Copy table** / **Save CSV...** for OriginLab).
- **APS ELN Delphi source:** `ELN.Desktop/Forms/frm_SeriesWindow.pas` (standalone Series window with taskbar button and saved geometry), `ELN.Desktop/Dialogs/Series/dlg_SeriesPlan.dfm` and `.pas`, `ELN.Desktop/Views/frame_Series.dfm` and `.pas`, and `ELN.Model/unit_SeriesTaguchi.pas`.

Cross-references: Chapter 6: Working on multiple records at once; Chapter 9: The value sources and Derived formulas; Chapter 10: Principles of Experiment Planning and the Taguchi Method; Chapter 11: The Hypothesis Tree and Evidence; Chapter 15: Cross-Project Charts, PDF Reports, and Packages; Chapter 18: Reading Data and Auditing Drift via MCP; Chapter 20: Drafting Trees, Hypotheses, and Wikis.

Project Wiki, Notes, Tags, and Links

IN THIS CHAPTER

- Creating a five-section Project Wiki and editing Markdown pages with inline figures
- Cross-referencing items and specimens with [[item:N]] and [[specimen:CODE]] links
- Categorizing records with colored Tags and discussing results in Comments
- Building typed bidirectional relationships between items with Copy as Link Target

Plenty of project information does not belong inside a single deposition or measurement action. Project goals, literature notes, substrate-cleaning procedures, and cross-sample comparisons apply to the whole project. Individual items also need category badges (`Tags`), discussion threads (`Comments`), or links to the target or substrate batch they came from (`Links`). The **Project Wiki**, **Tags**, **Comments**, and **Typed Links** handle these connections without cluttering your sample tree (Figure 13-1).

The five-section Project Wiki

A **Wiki page** in APS ELN is an item of type **Wiki** whose body is a Markdown (`.md`) file stored in your notebook's attachment folder. Every project Wiki uses the same five folders under `Wiki/` (Figure 13-1):

- **`Wiki / Goals`:** project objectives, grant milestones, and target specifications.
- **`Wiki / Ideas`:** design calculations, brainstorming notes, and preliminary sketches.
- **`Wiki / Literature`:** summaries of published papers, reference parameters, and competing structures.

- **`Wiki / Analysis`:** cross-experiment write-ups, summary tables, and comparative plots.
- **`Wiki / References`:** instrument manuals, calibration certificates, and standard operating procedures (SOPs).

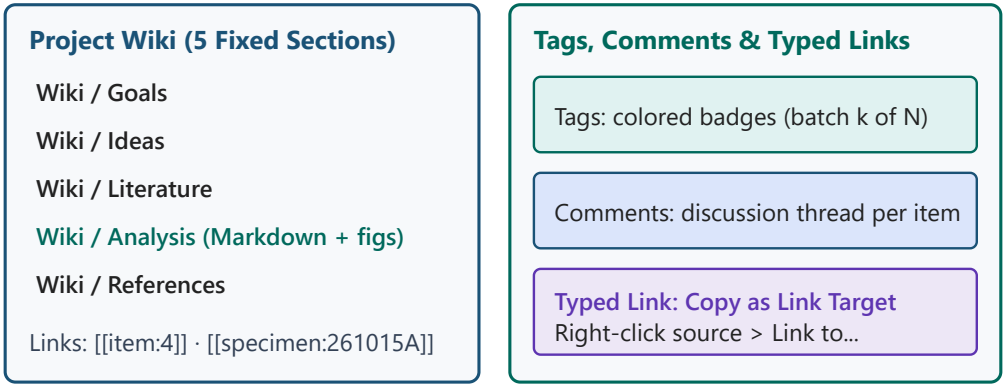


Figure 13-1: Organizing project knowledge and item connections. The five-section Project Wiki (left) stores Markdown write-ups with inline figures and record links. Tags, Comments, and Typed Links (right) classify, discuss, and connect scientific items.

Follow these steps to create and edit a project Wiki:

1. Select your project (or a folder inside it) in the **Projects** panel and choose **Project > New Wiki** (or click **New Topic Wiki** on the toolbar). Confirm **Add a Wiki to ...?**. APS ELN applies the `Wiki` template and creates the `Wiki` folder with its five section subfolders.
2. To add a new page inside one of the five sections, open that section folder (such as `Wiki / Analysis`) and click **New** (`Ins`). Select **Wiki** as the item type, enter the page **Title**, and click **Save**.
3. Single-click the Wiki item in the **Items** panel to read its rendered HTML in the **Markdown** viewer.
4. Click **Edit** in the viewer toolbar to open the **Markdown Editor** (Chapter 7: Working drafts, external edits, and conflict resolution), write your Markdown text, and press `Ctrl+S` (or click **Save**).

To include a figure, diagram, or PDF inside a Wiki page, drag the local file into the open **Markdown Editor** (or copy an image to the clipboard and press `Ctrl+V`). APS ELN uploads the file as an attachment of that Wiki item and inserts the Markdown reference (`![Caption](figure.png)`) automatically.

> **Remember**
>
> To keep the tree clean, a Wiki item hides its attached `.md` body and image files in the **Items** panel and hides the bottom thumbnail strip. In the item tree, **Item > Edit** edits the Wiki item's title and metadata; to edit the page text, always click the **Edit** button inside the **Markdown** viewer.

> **Warning: dropped or pasted Wiki assets upload immediately**
>
> Dragging a file or pressing `Ctrl+V` to paste an image into the **Markdown Editor** uploads that asset to the Wiki item immediately. APS ELN does this so it can generate the Markdown link. Clicking **Cancel** on your text edit later does not delete the uploaded image attachment.

Navigating between Wiki pages and experimental records

Inside any Markdown Wiki page or attachment, you can create live links that jump directly to any item or specimen in your notebook (even across different projects):

- **Short bracket syntax:** write `[[item:4]]` to link to item ID 4, or `[[specimen:261015A]]` to link to the specimen whose title is `261015A`.
- **Standard Markdown URL syntax:** write `[Indentation 261015A](eln://item/4)` or `[Sample 261015A](eln://specimen/261015A)`.

When you click the rendered link in the **Markdown** viewer, APS ELN navigates the **Projects** and **Items** panels straight to that record. You can also jump to any item ID or `eln://` URL at any time by choosing **Navigate > Go to Item...** (`Ctrl+G`).

Classifying items with Tags and Favorites

While folders group items by location, **Tags** let you label items across different folders and projects (for example, `High-Hardness`, `Annealed`, `Thesis-Fig-3`, or `Repeat-Needed`):

1. Right-click an item (or select multiple items with `Ctrl`+click / `Shift`+click and right-click inside the selection) and choose **Tags...**.
2. Tick a tag's check box to assign it to the selected item(s), or clear the check box to remove it. When multiple items are selected, the **Items** column shows `k of N` (how many of the N selected items currently carry that tag). Changes take effect immediately when you click the check box.
3. To create a new tag, click **Add**, enter **Name:**, pick a badge color with **Choose...**, and click **OK**.
4. Click **Close** when finished. Assigned tags appear as colored badges right beside the item title in the **Items** panel.

> ⚠ **Warning: clicking Delete in the Tags dialog removes the tag from the whole notebook**
>
> In the **Tags...** dialog (and in **Reference Data > Tags...**), clicking the **Delete** button deletes that tag definition and strips it from every item in the notebook. To remove a tag from the selected item only, uncheck its box.

To bookmark a project, folder, or item that you visit every day, right-click it and choose **Add to Favorites**. It appears under the **Favorites** branch in the left **Projects** panel for one-click access. Choosing **Remove from Favorites** removes the bookmark without touching the record.

Recording Comments and typed item relationships

APS ELN provides two ways to attach prose to an item and two ways to link items together (Table 13-1):

- **Rich-text Notes (`Item > Add Note`):** formal experimental observations stored as child entries of the item and included in signed item hashes.
- **Comments (`Right-click > Comments...`):** a timestamped discussion thread where you and your supervisor can post questions or review notes. Select a comment you wrote and click **Edit** or **Delete** to revise or remove it; you can only edit your own comments.

Sometimes one scientific item depends on another item outside its parent-child tree branch. For example, specimen `261015A` may be deposited from a specific `B4C/BN Target` item, or a repeat run `261020A` may duplicate `Deposition 261015A`. In those cases, create a **typed item relationship**:

1. Right-click the target item in the **Items** panel and choose **Copy as Link Target**.
2. Right-click the source item and choose **Link to...**.
3. In the **Relationship:** drop-down, pick the relationship phrase (configured in **Reference Data > Link Types...**). Read the preview sentence in the dialog carefully to verify the direction (which item is the source and which is the target).
4. Enter optional **Notes:** and click **Link**.
5. To view, follow, edit, or remove relationships on an item, select the item and choose **Item > Links...** (or double-click an inline link row to jump to the linked item).

Table 13-1: Choosing the right annotation or link tool in APS ELN.

TOOL	HOW TO CREATE IT	BEST USED FOR	PART OF SIGNED ITEM HASH?
Project Wiki page	**Project > New Wiki**, then **New** (`Wiki` item)	Project goals, literature notes, SOPs, and multi-sample summaries with inline figures	Signed only if the Wiki item itself is signed.
Rich-text Note	**Item > Add Note**	Formal experimental observations belonging to one procedure or action	Yes (locked when the parent item is signed).
Comment	Right-click item > **Comments...**	Discussion threads and review feedback with teammates or your supervisor	No (annotation outside the signed payload).
Tag	Right-click item(s) > **Tags...**	Colored category badges (`k of N` batch assignment) for visual scanning and search	No (annotation badge).
Markdown `[[item:N]]` link	Type `[[item:N]]` or `[[specimen:CODE]]` in `.md`	Inline navigation from a Wiki page or Markdown note to an item or specimen	Stored inside the `.md` attachment text.
Typed Item Link	**Copy as Link Target**, then **Link to...**	Bidirectional database relationship between two items (inspected via **Item > Links...**)	No (relationship metadata; read-only in read-only projects).

Sources

- **APS ELN Community Help:** `Wiki.html` (**Project > New Wiki**, the `Wiki` item type and `.md` body attachment, hidden file children and hidden thumbnail strip, clicking **Edit** in the Markdown viewer vs. **Item > Edit**, dragging/pasting images with `Ctrl+V`, and `[[item:N]]` / `[[specimen:CODE]]` navigation); `TagsAndLinks.html` (**Tags...**, batch `k of N` tagging, **Reference Data > Tags...**, the **Delete** vs. uncheck warning, **Comments...**, **Copy as Link Target** and **Link to...**, **Item > Links...**, **Reference Data > Link Types...**, **Add to Favorites**, and signature/read-only boundaries); `FilesAndNotes.html` (**Item > Add Note**).
- **APS ELN Commercial ("For Teams") Help:** `Wiki.html` (the fixed five-section layout `Goals`, `Ideas`, `Literature`, `Analysis`, `References`, **New Topic Wiki**, ETag conflict detection, `eln://item/<id>` and `eln://specimen/<code>` links, and MCP wiki tools).

Cross-references: Chapter 6: Working on multiple records at once; Chapter 7: Working drafts, external edits, and conflict resolution; Chapter 14: Full-Text, Panel, and Semantic Search; Chapter 16: Completeness, Audit, and Signatures.

Full-Text, Panel, and Semantic Search

IN THIS CHAPTER

- Filtering the Projects and Items panels in real time without losing context
- Finding exact words and phrases across a notebook with Navigate > Search
- Setting up and running Semantic Search (Search by meaning) across PDFs and notes
- Using Rerank results, Project/Type/Created filters, and Go to Item

Once your notebook has dozens of depositions, hundreds of files, and multi-page Wiki notes (Chapter 13: The five-section Project Wiki), clicking through folders one by one takes too long. Sometimes you know the exact sample code (`261015A`). Sometimes you remember a phrase from a value note (`masked edge`). Other times you want every record about *delamination after reciprocating wear*, even if the note called it *spallation* or *coating peeling*. APS ELN gives you three search tools for those three jobs (Figure 14-1).

Three ways to find a record in APS ELN

Each search tool in APS ELN covers a different scope (Figure 14-1 and Table 14-1):

1. **Panel filters:** the filter boxes at the top of the **Projects** and **Items** panels narrow the visible tree in real time as you type.
2. **Notebook text search (`Navigate > Search...` in Community, or `Edit > Search...` in Team; `Ctrl+F`):** searches record titles, descriptions, properties, entry titles, attachment titles, value descriptions, and tag names across every readable project.

3. **Semantic search (`Search by meaning` inside `Search...`):** a hybrid search engine (`ELN.Search`) that runs offline on your PC in Community or on your lab's search server in the Team edition. It indexes both structured records and the full text inside attached PDF, Office, Markdown, HTML, JSON, XML, CSV, RTF, and text files, matching by conceptual meaning as well as exact terms.

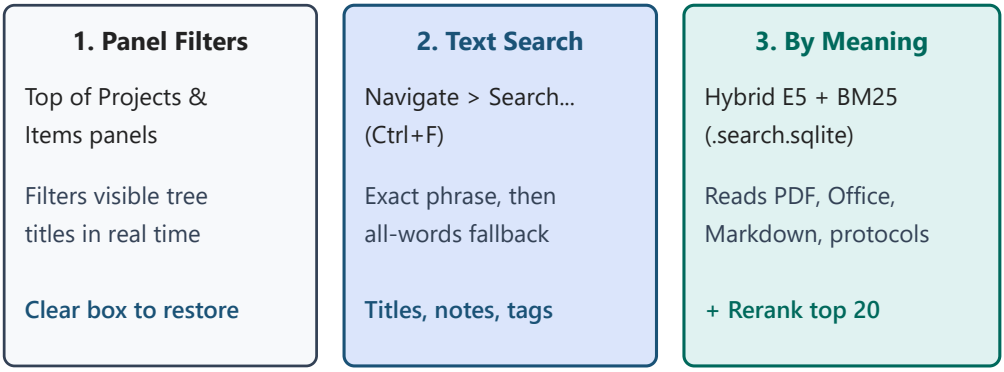


Figure 14-1: The three search tools in APS ELN. Use panel filters (1) to narrow the current tree, Text Search (2) for fast exact-word matches on record metadata, and Search by meaning (3) to search inside PDFs, Office files, Markdown notes, and protocols by concept.

Filtering panels and jumping to a known ID

When you already have a project open and want to spot one sample or one file type quickly, use the filter box at the top of the panel:

- **Projects panel filter:** matches project and folder titles and descriptions while keeping the parent hierarchy visible.
- **Items panel filter (`Filter items...`):** matches item titles and descriptions, note/protocol entry titles, and attached file titles and descriptions inside the currently selected project or folder. Parent rows stay visible so a matching child action never loses its specimen context.

> **Remember**
>
> A panel filter hides non-matching rows temporarily; it never deletes anything. If a project or item seems to have disappeared, clear the text in the panel filter box first. Then click **Show Archived** in the Projects panel (Chapter 5: Archiving finished projects) and check **Tools > Recycle Bin...**.

When a teammate, a Wiki page, or an audit report gives you an exact item number or internal URL, jump straight to it without searching:

1. Choose **Navigate > Go to Item...** in Community, or **View > Go to item...** in the Team client (or press `Ctrl+G`).
2. In **Enter an item ID or eln:// URL:**, type the positive integer item ID (such as `4`) or paste an internal URL (such as `eln://item/4` or `eln://specimen/261015A`).
3. Click **OK**. APS ELN switches to the target project and selects the item in the tree.

Searching notebook text (`Ctrl+F`)

To search across all readable projects at once by word or phrase:

1. Choose **Navigate > Search...** in Community, or **Edit > Search...** in the Team client (or press `Ctrl+F`).
2. Type at least two characters into the search box and press `Enter` (or click **Search**). Clicking the drop-down arrow at the right end of the search box lets you repeat any of your last 20 searches.
3. Inspect the result columns: **ID** (the owning item ID), **Type** (`item`, `entry`, `file`, `value`, or `tag`), **Name**, **Location** (project, folder, and parent item), and **Matched In** (which field matched). In the Team edition, an **Owner** column also appears to the left of **ID** so you can tell teammates' projects apart.
4. Click any result row (or move through the list with the `Up` and `Down` arrow keys). The main window behind the dialog navigates

immediately to that item while keeping the **Search** window open so you can inspect several hits in a row. Press `Esc` to close the dialog.

How standard text search matches your query:

- It first looks for your **entire entered phrase** as a case-insensitive substring.
- If no record contains the exact multi-word phrase, it falls back to searching for the **individual words** (ignoring words shorter than two characters). It then returns items that contain *all* of those words across any of their fields.
- Standard text search does not use wildcard symbols (`*`, `?`) or Boolean operators (`AND`, `OR`). It also does not read inside attachment bytes, such as the body of a PDF or Markdown file. To search inside attachment contents or match synonyms, use **Search by meaning**.

Searching by meaning (Semantic Search) and reranking

Search by meaning uses a multilingual embedding model (`E5`) combined with lexical matching and an optional cross-encoder reranker. It searches both your structured notebook fields and the full text inside your attached documents.

> **In ELN**
>
> To enable semantic search for the first time in **APS ELN Community**, open a notebook and choose **Tools > Semantic Search Setup...**. Click **Download and Install**. APS ELN downloads the ONNX Runtime and embedding model first (~549 MB) and then the optional PDFium extractor and cross-encoder reranker (~117 MB, about 666 MB in total). After that one-time download, semantic search runs 100% offline on your computer; no notebook text is ever sent over the network.

In **APS ELN Community**, the semantic index is stored in a companion file next to your notebook with the extension `.search.sqlite`. Keep that file

protected alongside your `.sqlite` database.

> **In a Team Lab: how Search by meaning works on a shared team server**
>
> In the Commercial ("For Teams") edition, **Search by meaning** runs on your laboratory's shared search server (`ELN.WebAPI`). That server indexes the MySQL database and WebDAV attachments centrally into `notebook.search.sqlite`. You do not download the ~666 MB model bundle on your own workstation, and there is no **Semantic Search Setup...** menu item.
>
> Instead, an administrator (`Admin` or `Super` role) opens **Tools > Settings...**, switches to the **Connection** tab, and enters the **Search server address** (such as `https://nas.lab:8443`). After the next sign-in, your Desktop session authenticates to the search server automatically. The bottom of the **Search** window then shows the server's live index status (`Index complete: N passages · updated HH:MM`), and every search is filtered to the projects you can read.

Once semantic search is available, open **Search...** (`Ctrl+F`) and tick **Search by meaning**:

- **What it reads:** item titles, descriptions, and properties; tags; note and protocol entries; value descriptions; and attachment filenames. It also indexes the full text inside `.pdf` (up to 2,000 pages with PDFium), `.docx`, `.pptx`, `.xlsx`, `.md`, `.html`, `.htm`, `.json`, `.xml`, `.csv`, `.rtf`, and `.txt` files. Each file is indexed up to 64 MB and the first 200,000 characters.
- **Pre-ranking filters:** when **Search by meaning** is ticked, three filters below the search box become active: **Project**, **Type** (item type), and **Created** (first and last UTC creation dates). Because these filters run *before* ranking, narrowing to one project or item type surfaces deep matches that might otherwise rank below top-20 hits from other projects.
- **Rerank results:** when the optional reranking model is installed, the **Rerank results** check box is ticked by default (Figure 14-2). Hybrid

search first retrieves the top 20 candidate items. The cross-encoder model then reads your query alongside each item's best passage and full tree path to re-sort those 20 items in about 0.5 seconds. When the reranker order is shown, the result count above the list ends with **reranked**.

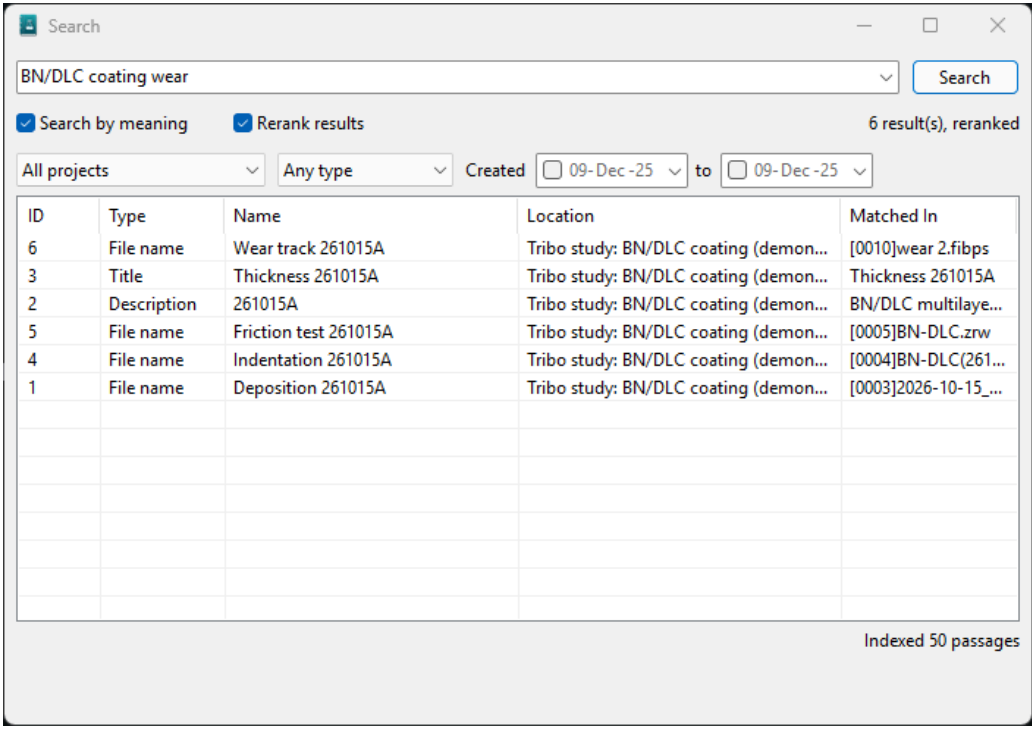


Figure 14-2: The Navigate > Search dialog (Ctrl+F) with Search by meaning and Rerank results active. Pre-ranking Project, Type, and Created filters sit above the results table, and the status bar at the bottom right reports the local passage index count.

> **Tip**
>
> If you type a very short code or two-letter symbol (such as `Co` or `BN`) while **Search by meaning** is active, untick **Rerank results** or switch back to standard text search. Cross-encoder rerankers are trained on

natural-language phrases and can rank a two-letter symbol lower than exact keyword search does.

Table 14-1: Choosing the right search or selection tool.

GOAL	TOOL TO USE	SCOPE AND BEHAVIOR
Spot an item or file inside the currently open project	**Filter items...** box at top of **Items** panel	Real-time title/description filter on the current tree; keeps parent rows for context.
Jump to item `12` or `eln://specimen/261015A`	**Navigate > Go to Item...** (or **View > Go to item...** in Team; `Ctrl+G`)	Direct lookup by integer item ID or specimen code in the active notebook.
Find an exact sample code, tag name, or phrase across all projects	**Navigate > Search...** (or **Edit > Search...** in Team; `Ctrl+F`, standard mode)	Phrase match first, then all-words fallback across titles, descriptions, tags, and value notes.
Find a concept inside PDF papers, `.docx` reports, `.md` Wiki pages, or protocol forms	**Search...** (`Ctrl+F`) with **Search by meaning** ticked	Hybrid vector + keyword search over records and document text, with optional top-20 **Rerank results** and **Project** / **Type** / **Created** filters.
Compare numeric conditions and results across procedures	**Tools > Analysis...** (Chapter 15)	Pairs procedure `Condition` values (*X*) against child action `Result` values (*Y*).

Going Deeper: how the semantic index updates without slowing down your work

When you open a notebook or open the **Search** dialog, `ELN.Search` checks for new or modified items and attachments. It re-extracts text

from an attached file only when the file's byte size or last-write timestamp changes (and verifies the content hash if only the timestamp changed). Encrypted PDFs, legacy binary Office files (`.doc`, `.xls`, `.ppt`), image-only scanned PDFs without a text layer, and files over 64 MB are skipped cleanly without stopping index updates.

Sources

- **APS ELN Community Help:** `Search.html` (**Navigate > Search...**, 2-character minimum, last-20 search history, result columns `ID`, `Type`, `Name`, `Location`, `Matched In`, phrase match with all-words fallback, **Search by meaning**, **Tools > Semantic Search Setup...**, the ~666 MB offline artifact bundle and `.search.sqlite` index, supported document extensions and the 64 MB / 200,000-character / 2,000-page limits, **Rerank results** top-20 re-sorting in ~0.5 s with 1.5 s fallback, **Project** / **Type** / UTC **Created** pre-filters, panel filters, and **Navigate > Go to Item...**).
- **APS ELN Commercial ("For Teams") Desktop and Server source:** `ELN.Desktop/Dialogs/Search/dlg_CommercialSearch.dfm` (**Owner** column in search results), `ELN.Desktop/Dialogs/System/dlg_Settings.dfm` (**Tools > Settings... > Connection > Search server address**), `ELN.Desktop/Units/unit_WebApiSession.pas` (`TSearchServerLink` sign-in), and `ELN.WebAPI/unit_SearchModule.pas` (central `notebook.search.sqlite` index).
- **APS ELN Community README:** `APS-ELN-Community/README.md` (`ELN.Search` architecture: ONNX Runtime, XLM-RoBERTa Unigram tokenizer, E5 embedder, exact vector + passage store, hybrid searcher, and PDFium extractor).

Cross-references: Chapter 3: The Dashboard and keyboard navigation; Chapter 5: Archiving finished projects; Chapter 13: Navigating between Wiki pages and experimental records; Chapter 15: Cross-Project Charts, PDF Reports, and Packages.

Cross-Project Charts, PDF Reports, and Packages

IN THIS CHAPTER

- Plotting experimental Conditions against test Results across a project or folder with Tools > Analysis...
- Understanding how procedure-level Conditions pair with the mean of child Results
- Generating formatted A4 or Letter PDF experiment reports (File > Export Report...)
- Exporting and importing JSON scientific packages (File > Export... and Import scientific package...)

When you plan a structured multi-point sweep or a Taguchi orthogonal array in advance, use **Project > Series...** (Chapter 12: Planning a series in the Series window). When you instead want to compare arbitrary procedures across a project, print a PDF report, or export a JSON package, use **Tools > Analysis...** and **File > Export Report...** (Figure 15-1).

Plotting conditions against results

To plot how an experimental input affects a measured property across a project or folder:

1. Select the project or specimen folder in the **Projects** panel on the left (for example, `Tribology` or its `BN-DLC Series` folder).
2. Choose **Tools > Analysis...**.
3. Under **X Axis**, select the **Condition** value type you varied across runs (such as `Power`, `Bias`, `Pressure`, or `Normal force`).
4. Under **Y Axis**, select the **Result** value type you measured on the samples (such as `Hardness`, `Modulus`, `COF`, `Wear rate`, or `Period`).

5. Tick the procedures you want to include in the list (use **All** or **None** to toggle the whole list), then click **Plot**.
6. Click **Copy Data (CSV)** to copy the paired (X, Y) data rows to the clipboard for OriginLab or Python, or click **Copy Chart** to copy the chart image.

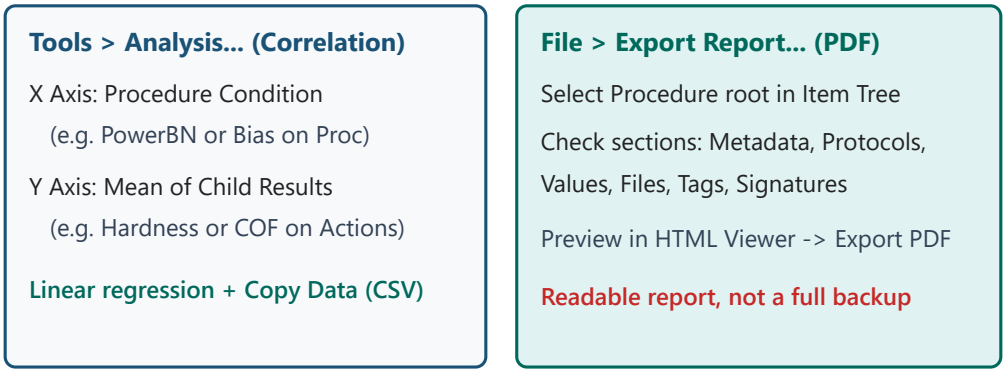


Figure 15-1: Cross-experiment analysis and reporting in APS ELN. Tools > Analysis (left) plots a procedure-level Condition against the mean of matching child Results across selected runs. File > Export Report (right) compiles a Procedure tree into a formatted PDF.

To read the plot accurately, you need to know how **Tools > Analysis...** pairs X and Y for each point:

- **One point per selected Procedure:** each point on the scatter plot represents one root **Procedure** (`Proc`).
- **How X is chosen:** X is the first matching **Condition** recorded directly on that **Procedure** (such as `Parameters.Bias` $= 8\ \mathrm{V}$ on `Deposition 261015A`).
- **How Y is calculated:** Y is the arithmetic mean of all matching **Result** values across three levels under the procedure. It includes direct child actions of the procedure, actions directly below child **Specimens** (such as `Thickness 261015A`, `Indentation 261015A`, and `Friction test 261015A`), and one further action level below those specimen actions (such as `Wear track 261015A` or `XRR fit 261014B`).

- Any procedure that lacks either the selected X condition or at least one matching Y result within those levels is omitted from the plot.

> **Remember**
>
> **Tools > Analysis...** reads the **Condition** from the root **Procedure** and the **Result** from its child actions (down to two levels below a **Specimen**). Always store deposition and synthesis inputs on the **Procedure** and characterization outputs on the child **Actions** (Chapter 6: Designing a clean item hierarchy).

When enough points are plotted, APS ELN draws a linear regression line automatically. Treat that line as a visual guide rather than a statistical proof: the regression does not weight points by their stored tolerances (`STD`). Use **Copy Data (CSV)** when you need weighted fitting or error bars in a publication figure.

Exporting a formatted PDF report

When you need a clean document for a weekly group meeting, a thesis appendix, or a collaborator who does not have your notebook open, generate a PDF report with **File > Export Report...**:

1. In the center **Items** panel, select the **Procedure** root (`Proc`, such as `Deposition 261015A`).
2. Choose **File > Export Report...**.
3. Under **Items to include**, tick the child specimens and actions you want in the report.
4. Tick the **Sections** to include: `Metadata`, `Protocols`, `Values`, `Files`, `Tags`, `Comments`, and `Signatures`. The `Values` section prints both the numeric values and their stored tolerances.
5. Choose the page size (**A4** or **Letter**) and click **Browse...** to set the **Output path**.
6. Click **Preview** to inspect the formatted report in the HTML viewer, then click **Export PDF**.

> ⚠ **Warning: select the Procedure root before opening Export Report**
>
> If you select a **Specimen** or a single **Action** in the item tree before choosing **File > Export Report...**, the **Items to include** checklist offers only that single row without its children. Always select the top-level **Procedure** (`Proc`) when you want a complete multi-action experiment report.

Exporting and importing scientific packages

APS ELN can also export and import structured JSON **scientific packages** (`File > Export...` and `File > Import scientific package...`):

1. To export, select a project and folder, choose **File > Export...**, and tick the **Items to export**. Set the **Export path**, choose whether to include file metadata (**Include files**) and protocol status history (**Include status history**), and click **Export**.
2. To import a package into another notebook, select the destination project and folder, choose **File > Import scientific package...**, browse to the JSON file, inspect the **Package preview**, and click **Import**.

Table 15-1: Comparing PDF reports, JSON scientific packages, and full notebook backups.

OUTPUT METHOD	MENU COMMAND	WHAT IT CONTAINS	CAN IT RESTORE A FULL NOTEBOOK?
PDF Report	**File > Export Report...**	Formatted human-readable document of selected items, protocols, values, tolerances, file lists, and signatures	No (read-only PDF printout).

OUTPUT METHOD	MENU COMMAND	WHAT IT CONTAINS	CAN IT RESTORE A FULL NOTEBOOK?
Scientific Package	**File > Export... / Import scientific package...**	JSON file of top-level item metadata, protocol entries (reset to `Draft` on import), and protocol templates	No (`Include files` writes file metadata only and omits attachment bytes and child hierarchies).
Full Notebook Backup	Close ELN and copy `.sqlite` + `<name> files` folder (Chapter 2)	Every project, item tree, value, provenance source, signature, audit row, and all raw attachment bytes	**Yes** (the only complete backup and restore method).

> ⚠ **Warning: a scientific package is not a notebook backup**
>
> In **File > Export...**, ticking **Include files** writes file metadata into the JSON package while omitting the raw `.fibps`, `.mit`, `.zrw`, or `.xrcx` attachment bytes. Likewise, **File > Import scientific package...** restores only top-level item metadata and protocol forms or templates (always starting imported protocols as `Draft`). Never rely on a scientific package or a PDF report as a backup of your laboratory data.

Sources

- **APS ELN Community Help:** `AnalysisAndReports.html` (**Tools > Analysis...**, **File > Export Report...**, and **File > Export... / File > Import scientific package...**).

Cross-references: Chapter 2: How to back up a local notebook safely; Chapter 6: Designing a clean item hierarchy; Chapter 8: The protocol review workflow; Chapter 9: Conditions, Results, units, and plausible-range checks;

Chapter 12: Experiment Series and Taguchi Arrays in ELN; Chapter 14: Full-Text, Panel, and Semantic Search.

Completeness, Audit, and Signatures

IN THIS CHAPTER

- Defining and running Completeness Rules for each Action Type and Equipment
- Finding missing files and missing values with Data Completeness checks
- Inspecting the notebook Audit Log by date, entity, operation, and user
- Applying and verifying cryptographic Electronic Signatures (SIGNED)

Before you mark an experiment finished or hand it to your supervisor, check three things: every action has its raw file, every value matches its reader output, and the finished record is locked against edits. APS ELN checks each of these in turn (Figure 16-1).

Three layers of record integrity

Three tools handle completeness, provenance, and locking (Figure 16-1):

1. **Completeness Rules (`Tools > Data Completeness...`):** checks whether every **Procedure** and **Action** in a project has the required raw **File Types** and extracted **Value Types** for its **Action Type** and **Equipment**.
2. **Provenance and Audit (`Check Value Sources...` and `Tools > Audit Log...`):** verifies that every `Reader` and `Derived` value still matches its stored file and formula (Chapter 9: Checking value sources and understanding drift codes). It also records the chronological log of every `Create`, `Update`, and `Delete` event in the notebook.

3. **Electronic Signatures (`Signature > Sign Item...`):** computes a cryptographic content hash over a finished item, marks it with a dark-red **SIGNED** badge, and locks it permanently against any further edits.

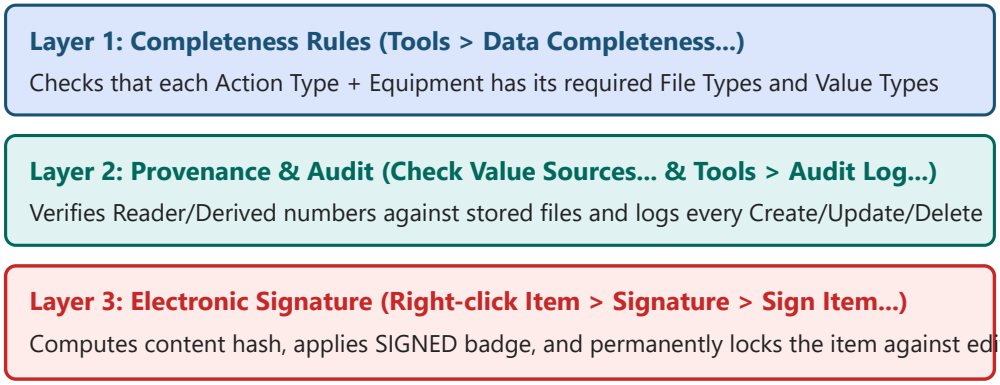


Figure 16-1: The three integrity layers in APS ELN. Run Layer 1 (Completeness) and Layer 2 (Check Value Sources) while the item is still editable, then apply Layer 3 (Electronic Signature) to lock the verified record permanently.

Completeness rules and Data Quality Checks

During a busy day at the instruments, it is easy to create a measurement action (such as a friction test or an XRR scan) and forget to attach the raw file or extract its values. **Completeness Rules** catch those gaps automatically across an entire project.

To define or inspect completeness rules:

1. Choose **Tools > Completeness Rules...**.
2. Click **Add** (or select an existing rule to edit it).
3. Enter a **Title** and **Description**, and select the matching **Action Type** (for example, `Indentation` or `Tribotest`), **Equipment** (optional), and **Severity** (`Error` or `Warning`).

4. In the lower lists, tick the **Required File Types** (such as `.mit` or `.zrw`) and the **Required Value Types** (such as `Hardness` and `Modulus`, or `Fn`, `Cycles`, and `COF`).
5. Tick **Active** and click **Save**.

To run a completeness check on your project before asking your supervisor for review:

1. Select the project or folder in the **Projects** panel.
2. Choose **Tools > Data Completeness...** (or right-click the project/folder and choose **Data Quality Check...** in the team client).
3. Inspect the **Data Quality Check** table, which lists **Category**, **Severity**, **Item**, **Location**, **Rule**, and **Missing** (the exact file type or value type that is absent).
4. Double-click any row (or select it and click **Go To**) to jump straight to the incomplete item in the main window. Attach the missing file or extract the missing value, and re-run **Tools > Data Completeness...** until zero issues remain. You can also click **Export...** to save the report as a CSV file.

Inspecting the Audit Log

Every time a person or an MCP agent creates, updates, or deletes a record in APS ELN, the notebook writes an entry to its **Audit Log**:

1. Choose **Tools > Audit Log...**.
2. Set the optional filters at the top of the dialog: **From** and **To** dates, **Entity** (such as `Item`, `File`, `Value`, `Protocol`, or `Hypothesis`), **Op** (`Create`, `Update`, or `Delete`), and **User**.
3. Click **Refresh** to inspect the **Timestamp**, **Entity Type**, **Entity ID**, **Operation**, **User**, and **Description** of each event.

> **Tip**
>
> When you delete a stored value (or when a batch reader update replaces old values), the audit log preserves the old record. It keeps

the old number, tolerance, value type, source, and description in the item's audit trail. You can always check what was removed and when.

Signing an item and verifying signature integrity

Once an item passes **Tools > Data Completeness...** and **Check Value Sources...**, you can apply an **Electronic Signature** to lock it permanently:

1. In the **Items** panel, right-click the **Procedure**, **Specimen**, or **Action** itself (not an attached file or protocol row) and choose **Signature > Sign Item...**.
2. Check the **Item Title** shown in the dialog.
3. Select the signature **Meaning**: `Authored`, `Reviewed`, `Approved`, or `Witnessed`.
4. Enter an optional **Comment** and type your **Password** (the signing password you set when creating your local Community notebook in Chapter 2, or your personal account password in a team lab).
5. Click **Sign**. A dark-red **SIGNED** badge appears beside the item in the tree.

> ⚠ **Warning: signing is permanent and cannot be undone**
>
> Signing an item permanently locks that item against any further edits: you cannot edit its fields, attach or replace files, save working drafts, add or edit values, or delete the item. APS ELN provides no "unsign" command. Always run **Check Value Sources...** and **Tools > Data Completeness...** *before* you sign.

To inspect a signature and verify that the item's database record still matches its signed hash:

1. Right-click the signed item and choose **Signature > View Signature...** (or double-click the signed badge).
2. Read the **Signed By**, **Date**, **Meaning**, **Comment**, and **Content Hash** fields.

3. Click **Verify Now** (or right-click the item in the tree and choose **Signature > Verify Integrity**). APS ELN recomputes the hash over the item's committed fields, note and protocol entries, file titles and MD5 hashes, and numeric values and tolerances. It then reports whether the hash is `VERIFIED`.

Table 16-1: Comparing Protocol Approval, Hypothesis Acceptance, and Item Signatures.

INTEGRITY MECHANISM	WHAT IT LOCKS OR SEALS	HOW IT IS APPLIED	CAN AN MCP AGENT APPLY IT?
Protocol Approval (Chapter 8)	One attached protocol form (`Approved` state)	**Tools > Protocol Review...** > **Approve**	No (human GUI gate only).
Hypothesis Acceptance Seal (Chapter 11)	A hypothesis statement, conclusion, runs, and cited evidence (`Sealed` badge)	**Project > Hypotheses** > **Close with conclusion** (Community) or **Accept** (Team)	No (human GUI gate only).
Item Electronic Signature	An item's properties, notes, protocols, file hashes, and values (`SIGNED` badge)	Right-click item > **Signature > Sign Item...** + password	No (requires interactive user password).

Going Deeper: what Verify Integrity checks and what check_storage checks

Signature > Verify Integrity checks the committed database records of that single item (including the stored MD5 hashes of its attached files), and it is not recursive over child items. To verify that every physical attachment file on disk or WebDAV still matches its recorded MD5 across a whole project, pair signature verification with the MCP `check_storage` tool (Chapter 18). Items signed in older versions before

> versioned canonical hashes existed display **Legacy signature; integrity cannot be verified** while remaining locked.

Sources

- **APS ELN Community Help:** `Integrity.html` (**Signature > Sign Item...**, meanings `Authored`, `Reviewed`, `Approved`, and `Witnessed`, permanent lock warning, **Signature > View Signature...**, **Verify Now** and **Signature > Verify Integrity**, what the content hash covers, legacy signature message, **Tools > Audit Log...**, **Tools > Completeness Rules...**, **Tools > Data Completeness...**, and recycle safety).
- **APS ELN Commercial ("For Teams") Help:** `Signatures.html` (the dark-red **SIGNED** badge, signature verification, and regulatory context); `Projects.html` (**Data Quality Check...** on the project context menu).
- **ELN provenance study (`arXiv:2610.27859`):** Sections 2.4 and 4 (audit log verification and completeness gaps on empty placeholder actions).

Cross-references: Chapter 2: Creating a local notebook in Community; Chapter 5: Deleting containers and using the Recycle Bin; Chapter 8: The protocol review workflow; Chapter 9: Checking value sources and understanding drift codes; Chapter 11: Concluding, closing or accepting, and checking the content seal.

Connecting an AI Agent to Your Notebook

IN THIS CHAPTER

- Understanding how the ELN Model Context Protocol (MCP) server works
- Launching ELNCommunityMCP.exe with --notebook, --upload-root, and --plugins
- Registering ELNMCPServer.exe with DSH or OMP in a team laboratory
- Knowing the security, upload-root, and human-review boundaries enforced by the server

Instead of clicking through menus for every repetitive check, you can connect an AI assistant such as DeepSeek Harness (DSH) or OMP to APS ELN through the **Model Context Protocol (MCP)** server. The assistant can search across projects, inspect time-series logs, and audit every stored value against its raw file in seconds. It can also fit an X-ray reflectivity curve (Chapter 19) and build structured experiment trees (Chapter 20), all under the server's built-in safety boundaries (Figure 17-1).

What the ELN MCP server does

Both editions of APS ELN include a command-line MCP host executable installed right beside the Desktop program (Figure 17-1):

- **`ELNCommunityMCP.exe` (APS ELN Community):** opens a local format 6 `.sqlite` notebook and its attachment folder on your computer.
- **`ELNMCPServer.exe` (APS ELN "For Teams"):** connects to your shared laboratory MySQL and WebDAV server using your workstation's configuration.

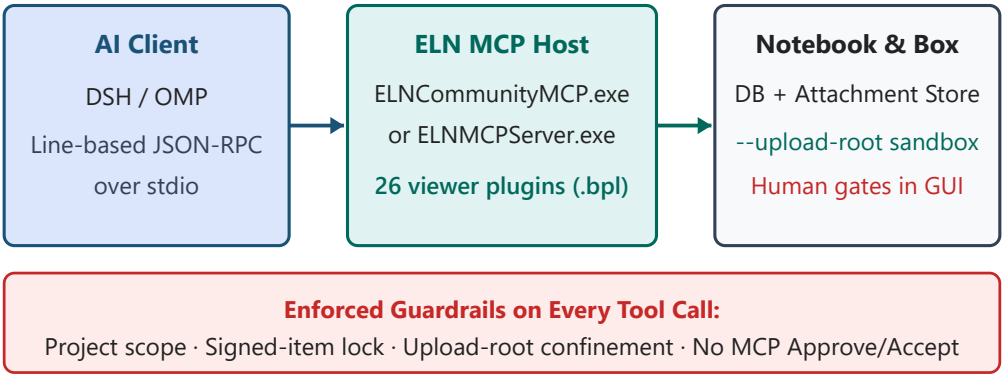


Figure 17-1: How an AI assistant connects to APS ELN. The client launches the MCP host over standard input/output (stdio). The host loads the same 26 viewer plugins (.bpl) and enforces project scope, signed-item locks, upload-root confinement, and human review gates.

When an AI client starts the MCP host, they communicate over standard input and standard output (`stdio`) using one UTF-8 JSON-RPC 2.0 message per line. The MCP host loads the same scientific domain model and the same 26 `.bpl` viewer plugins as the Desktop GUI. Calling `extract_file_key_values` from an agent therefore runs the exact same reader code (`aps.deposition`, `aps.fibps`, `aps.mit`, `aps.zrw`, `aps.xrcx`) as clicking **Extract Values** on screen (Chapter 9: Reader mappings, unit factors, and standard deviations).

Table 17-1: The nine tool categories exposed by the ELN MCP server.

CATEGORY	REPRESENTATIVE MCP TOOLS	CHAPTER
Navigation	`list_projects`, `list_folders`, `list_items`, `get_item`, `search`, `search_semantic`	18
Values & Series	`get_values`, `query_values`, `list_value_types`, `correlate_values`, `list_series`, `get_series_table`, `create_series`, `update_series_plan`	18, 20

CATEGORY	REPRESENTATIVE MCP TOOLS	CHAPTER
Files & Columns	`list_files`, `extract_file_data`, `extract_file_key_values`, `list_file_columns`, `get_file_columns`, `download_file`	18, 19
Checks (Read-only)	`check_storage`, `check_value_drift`, `report_value_issues`	18, 19
Metadata	`get_audit_log`, `get_comments`, `get_item_links`, `get_protocols`, `get_signatures`, `list_action_types`, `list_audit_rules`, `list_equipment`, `list_file_types`	18
Write (Items & Files)	`create_project`, `create_folder`, `create_item`, `update_item`, `move_item`, `reparent_item`, `upload_file`, `rename_file`, `move_file`, `delete_file`	19, 20, 21
Write (Values)	`add_value`, `add_values`, `add_derived_value`, `update_value`, `remove_value`, `remove_values`	19, 20, 21
Wiki & Tags	`wiki_create_page`, `wiki_get_page`, `wiki_update_page`, `create_tag`, `assign_tag`, `unassign_tag`, `list_tags`	20, 21
Hypotheses	`propose_hypothesis`, `update_hypothesis`, `attach_run`, `detach_run`, `cite_evidence`, `remove_evidence`, `submit_conclusion`, `abandon_hypothesis`, `get_hypothesis`, `get_hypothesis_tree`, `list_hypotheses`, `find_hypotheses_for_item`	20

Preparing a local session in APS ELN Community

In **APS ELN Community**, the installer uses a per-user folder (`%LocalAppData%\Programs\ELN Community`, which expands to `C:\Users\<you>\AppData\Local\Programs\ELN Community`) rather than `C:\Program`

`Files`, so you can install and update it on a student or university PC without Windows Administrator privileges. It places `ELNCommunityMCP.exe` in that folder right next to `ELNCommunity.exe`. Follow these steps to prepare a local MCP session safely:

1. **Create your notebook in the Desktop program first, then close the Desktop.** A local Community notebook uses a single-user SQLite file. Close `ELNCommunity.exe` before starting an MCP session that writes to that same `.sqlite` file.
2. **Create a dedicated upload/staging folder (`--upload-root`).** Create an ordinary local folder (such as `C:\ELN\Inbox`) with no directory junctions or symbolic links. Place only the raw measurement files you want the agent to upload (or the files exported by `download_file`) inside this folder.
3. **Configure your MCP client with absolute paths:**

```
ELNCommunityMCP.exe --notebook "C:\ELN\notebook.sqlite" --
upload-root "C:\ELN\Inbox" --plugins "C:\Users\
<you>\AppData\Local\Programs\ELN Community\Plugins"
```

If your `.sqlite` notebook already remembers its attachment folder (Chapter 2: Creating a local notebook in Community), `ELNCommunityMCP.exe` uses that saved folder automatically. If not, pass `--storage "C:\ELN\notebook files"` as well.

> **⚠ Warning: never pass --config to ELNCommunityMCP.exe**
>
> `--config` is a Desktop-only flag (`ELNCommunity.exe`). `ELNCommunityMCP.exe` takes `--notebook`, `--upload-root`, `--plugins`, and optional `--storage`, and it only opens an existing notebook; it never creates or migrates a database.

Registering with DSH or OMP in a team laboratory

In a team laboratory running the Commercial edition, the installer places `ELNMCPServer.exe` in `C:\Program Files\E-LabNotebook3\`.

In a Team Lab: registering ELNMCPServer.exe with DSH or OMP and clicking eln:// links

`ELNMCPServer.exe` reads your server connection settings, credentials, WebDAV URL, and upload root directly from the Windows Registry configured by the Desktop client. You can register it in either **OMP** or **DeepSeek Harness (DSH)**.

In OMP (`~/.omp/agent/mcp.json`): add the `eln` entry under `"mcpServers"`:

```
{
  "mcpServers": {
    "eln": {
      "type": "stdio",
      "command": "C:\\Program Files\\E-
LabNotebook3\\ELNMCPServer.exe",
      "enabled": true
    }
  }
}
```

In DeepSeek Harness (`$DSH_HOME/profiles/web/cordis.patch.yml`): add the `eln` server to the `mcp-manager` configuration:

```
- id: mcp-manager
  config:
    servers:
      eln:
        transport: stdio
        command: 'C:\Program Files\E-
```

```
LabNotebook3\ELNMCPServer.exe'
        enabled: true
```

(If you upgraded from an older 32-bit build in `C:\Program Files (x86)\E-LabNotebook3\`, update the `command` path to `C:\Program Files\E-LabNotebook3\ELNMCPServer.exe`.)

Every MCP tool that returns an item (`list_items`, `get_item`, `create_item`, `update_item`) includes a clickable `url` field such as `eln://item/42`. Because the Desktop client registers the `eln://` protocol handler in Windows on startup, clicking `eln://item/42` in a Markdown-rendering chat window focuses the Desktop application and jumps straight to item 42.

> **Tip**
>
> In the Commercial Desktop client, right-click any project, folder, or item in the tree and choose **Copy MCP Link**. Paste that reference directly into your chat prompt so your AI assistant knows the exact project or item ID without searching.

Security, upload-root, and review boundaries

An AI agent can make fast, persistent changes to a notebook, so the ELN MCP server enforces five hard boundaries inside its own code (Figure 17-1):

1. **Project and account scope:** Every tool call reloads the active user principal and checks project permissions. Attempting to read or write an item or file in a project you cannot access returns the same `out of scope` error as a non-existent ID. An agent cannot probe private projects.
2. **Upload-root file confinement (`--upload-root` / `MCPUploadRoot`):** Both `upload_file` (reading a local file into ELN) and `download_file` (copying an attached file out of ELN) are strictly confined to `--upload-root`. The server refuses any path outside that folder, any `..`

traversal, any Windows device prefix (`\\?\`, `\\.\`), any alternate data stream, or any directory junction/symlink along the path. Never set `--upload-root` to `C:\` or your entire user profile.

3. **Signed-item and cited-item locks:** The MCP server refuses to edit, add values to, replace files on, or delete any item carrying an electronic signature (`SIGNED`, Chapter 16). It also refuses to delete files or values cited by a hypothesis in `InReview` or `Accepted` (Chapter 11).
4. **Read-only session mode:** A read-only MCP session hides all write tools from `tools/list` and refuses any call that modifies data (`Tool "name" is not available in a read-only session.`). All read and audit tools (`list_projects`, `get_item`, `get_values`, `search`, `check_value_drift`, `check_storage`) remain available.
5. **No MCP tools for human review gates:** Calling `approve_hypothesis` or `accept_conclusion` returns `tool-not-found`. Approving protocols (Chapter 8), starting or closing/accepting hypotheses (Chapter 11), starting a `Draft` series or confirming `planned` values (Chapter 12), and signing items (Chapter 16) can only be performed by a human in the GUI.

Going Deeper: line-based JSON-RPC stdio initialization

When an MCP client launches `ELNCommunityMCP.exe` or `ELNMCPServer.exe`, it sends an `initialize` request line, followed by `notifications/initialized`. It then calls `tools/list` to read the JSON Schema of every registered tool before making any `tools/call` request:

```
{"jsonrpc":"2.0","id":1,"method":"initialize","params":
{}}
{"jsonrpc":"2.0","id":2,"method":"tools/list","params":
{}}
{"jsonrpc":"2.0","id":3,"method":"tools/call","params":
{"name":"list_projects","arguments":{}}}
```

Diagnostic messages from viewer packages go to `stderr` so `stdout` carries pure JSON-RPC lines, and closing `stdin` terminates the server

> cleanly.

Sources

- **APS ELN Community Help:** `MCP.html` (preparing a local session, closing the Desktop first, `--notebook`, `--upload-root`, `--plugins`, and `--storage` flags, default install path `%LocalAppData%\Programs\ELN Community`, line-based JSON-RPC 2.0 stdio transport, the 70 registered tools, read-only session mode, and write/review boundaries).
- **APS ELN Commercial ("For Teams") Help:** `AIIntegration.html` (`ELNMCPServer.exe`, the nine tool categories, Windows Registry configuration sharing, 64-bit path migration, and clickable `eln://item/N` URLs); `Projects.html` (**Copy MCP Link**).
- **Download tool specification:** `APS-ELN-Community/docs/note-download-file-mcp-tool.md` (`CheckUploadPathAllowed` confinement rules shared by `upload_file` and `download_file`).
- **ELN provenance study (`arXiv:2610.27859`):** Section 2.4 (MCP architecture and agent operations).

Cross-references: Chapter 2: Creating a local notebook in Community; Chapter 8: The protocol review workflow; Chapter 9: Reader mappings, unit factors, and standard deviations; Chapter 11: The Hypothesis Tree and Evidence; Chapter 16: Signing an item and verifying signature integrity; Chapter 18: Reading Data and Auditing Drift via MCP; Chapter 19: Study B: XRR Fitting with Provenance; Chapter 20: Drafting Trees, Hypotheses, and Wikis.

Reading Data and Auditing Drift via MCP

IN THIS CHAPTER

- Browsing projects, items, values, and experiment series ($list_{series}$, $get_{seriestable}$) with MCP read tools
- Running hybrid Semantic Search ($search_{semantic}$) with numeric value filters
- Inspecting long time-series logs with $list_{filecolumns}$ and $get_{filecolumns}$
- Auditing an entire project with $check_{storage}$, $check_{valuedrift}$, and $report_{valueissues}$

The safest way to start using an AI assistant with APS ELN (Chapter 17: What the ELN MCP server does) is through its read-only tools. In one prompt, your assistant can list all specimens in a folder, filter semantic search hits by numeric value ranges, summarize a 10,000-row sputtering log, and check every file hash and value across a project (Figure 18-1).

Navigating and querying values

When you ask an AI assistant a question about your notebook, it uses nine structured read tools to navigate the hierarchy, inspect values, and read experiment series (Table 18-1):

- **`list_projects` and `list_folders`:** return your readable projects (with item counts, owner, and dates) and the folder tree (`folder_id` and parent references) inside a project.
- **`list_items` and `get_item`:** `list_items` filters a project by `folder_id`, `item_type` (`Synthesis`, `Specimen`, `Measurement`, `Testing`), or `level` (`Proc`, `Item`, `Action`). `get_item` returns one item's full details: its children, attached files (with `file_id`, `extension`, and MD5 `hash`), recorded values, note/protocol entries, and tags.

- **`get_values` and `query_values`:** `get_values` returns all values on one item grouped into experimental conditions (`EXP_CONDITION`) and test results (`TEST_RESULT`). It also includes `ancestor_ids` (from the root procedure down to the direct parent) and each value's `source` object (`Reader`, `Derived`, `Manual`, `Planned`, or `Unknown`). `query_values` searches values across a project by `value_type` (title or symbol), `rec_type`, and item creation date (`date_from`, `date_to`).
- **`correlate_values`:** pairs condition values and result values across items in a project (the MCP counterpart of **Tools > Analysis...** in Chapter 15: Plotting conditions against results).
- **`list_series` and `get_series_table`:** `list_series` returns the experiment series of a project (`id`, `title`, `purpose`, `status`, and `member_count`). `get_series_table` returns the full series `plan` (`factors`, `fixed`, `responses`, and `target_folder_id`), every column with its `role` (`Shown`, `Hidden`, `Varied`, or `Response`), and one row per point (`slot_id`, `state`, `origin`, `cell_marks` with `planned_value` and `deviation`, and cell `hits` with `planned: true` when unconfirmed).

For example, when you ask about the friction and wear results on sample `261015A`, you just name the sample:

> **You:** What friction coefficient and wear rate did we get for sample 261015A, and how was the wear rate calculated?
>
> **Assistant:** On `Friction test 261015A` (`eln://item/5`, under `Deposition 261015A` → specimen `261015A`), the test ran at a normal force of $9.81\ \mathrm{N}$ for 1000 cycles (`BN-DLC.zrw`):
>
> - **Friction coefficient (μ):** 0.528 ± 0.019 (extracted by `aps.zrw` v2 from `BN-DLC.zrw` over the automatic plateau, cycles 88–999).
> - **Wear rate (K):** $3.05 \times 10^{-7}\ \mathrm{mm^3/(N \cdot mm)}$ (`Derived` via `wear_rate_reciprocating` v1). It was calculated from the mean of the three profilometer wear-track areas on `Wear track 261015A` ($0.00620\ \mathrm{mm^2}$ from `wear 1.fibps`, $0.00553\ \mathrm{mm^2}$ from `wear 2.fibps`, and $0.00621\ \mathrm{mm^2}$ from `wear 3.fibps`;

mean $0.00598\ \mathrm{mm}^2$) divided by $2 \times 9.81\ \mathrm{N} \times 1000\ \mathrm{cycles}$.

Searching by text and meaning (`search` and `search_semantic`)

The MCP server exposes both search engines from Chapter 14: Three ways to find a record in APS ELN:

- **`search`:** runs full-text search across items, protocols, files, values, and tags (with optional `project_id`). Every file match returns its `file_id`, `full_name` (preserving compound extensions such as `.depo.json` and `.process.log`), and `extension`.
- **`search_semantic`:** runs hybrid semantic search (`query`, optional `top_k` from 1 to 50, and optional `project_id`) over record fields and attachment text (PDF, Office, Markdown, HTML, JSON, XML, CSV, RTF, and text). When the cross-encoder reranker is installed, it re-sorts the top 20 item groups (`reranked: true`).

Unlike the Desktop Search dialog (which offers only project, item type, and creation date filters), `search_semantic` in MCP also accepts **structured numeric value filters** before ranking:

- **`folder_id`, `item_types`, `levels`, and `action_type_id`:** restrict the search to a specific folder subtree, item types (such as `["Measurement", "Testing"]`), levels (`["Proc", "Item", "Action"]`), or action type.
- **`date_field`, `date_from`, and `date_to`:** filter by `created` (default) or `modified` ISO 8601 dates.
- **`values`:** an array of numeric range filters, where each entry specifies `value_type` (symbol or title), `rec_type` (`EXP_CONDITION` or `TEST_RESULT`), `min`, `max`, and `unit` (the unit of your filter limits). The server converts `min` and `max` into the stored unit of the value type and echoes the converted limits in `filters_applied` alongside `requested_unit`.

Because the assistant translates your plain-English question into those filter fields, you can combine topic and number limits naturally:

> **You:** Which of our Co/C multilayer mirrors have a fitted period around 50 Å (say between 45 and 55 Å)?
>
> **Assistant:** One sample matches (`Period` test result between 45 and $55\ \text{Å}$, converted to 4.5×10^{-9}–$5.5 \times 10^{-9}\ \text{m}$):
>
> - **Specimen `261014B` (`XRR fit 261014B`, `eln://item/5`):** fitted in `XRR-fit_Co-C(261014B).xrcx` (`aps.xrcx` v2) with **Period** $= 49.40\ \text{Å}$ ($4.940 \times 10^{-9}\ \text{m}$), **Co Thickness** $= 20.83\ \text{Å}$, **C Thickness** $= 28.57\ \text{Å}$, and $\chi^2 = 0.943$.

Remember

If `search_semantic` is called on a large notebook while the indexer is still reading newly attached PDFs, the response includes `"index": {"complete": false, "pending_passages": ..., "files_pending": ...}`. Call `search_semantic` again after a moment to let the background index catch up.

Reading tabular instrument logs (`list_file_columns` and `get_file_columns`)

Sputtering logs (`.tsv`, `.log`) and tribometer curves often contain tens of thousands of time-series rows. Dumping an entire 8 MB `.tsv` file into an AI prompt would overflow the model's context window. Instead, APS ELN provides a two-step columnar tool pair:

1. **Discover the columns (`list_file_columns`):** Pass `file_id`. The server uses the file's viewer plugin to return `total_rows`, `time_span`, and the list of columns (`index`, `name`, `units`, and `dtype`).

2. **Fetch summary statistics or downsampled series (`get_file_columns`):** Pass `file_id`, the `columns` you want (by index or column name), and one of three modes:
    - `mode: "stats"`: returns the count, minimum, maximum, mean, and standard deviation of each requested column.
    - `mode: "downsample"` (the default): bins the series into `max_rows` (default `500`) min/max buckets so the agent sees every spike and drop in at most 1,000 points.
    - `mode: "raw"`: returns unbinned rows, optionally sliced to `row_range: [start, end]`.

When you ask whether a deposition run was stable, the assistant finds the `.tsv` log on that sample and checks the column statistics without reading all 17,171 rows into chat:

> **You:** Was the argon flow and magnetron current stable during the deposition of 261014B?
>
> **Assistant:** Yes. In the sputtering log `2026-10-14_11-56-09_StudyB_CoC_Co42s_C108s_N20_2mT.tsv` on `Deposition 261014B` ($17,171$ rows from `11:56:09` to `12:56:35`, $3626.4\,\mathrm{s}$ total):
>
> - **Argon flow (`MFC_Ar1`):** held steady at its $20\,\mathrm{sccm}$ setpoint throughout sputtering (mean $19.63 \pm 2.07\,\mathrm{sccm}$ across $16,983$ active rows, max $20.06\,\mathrm{sccm}$; $0\,\mathrm{sccm}$ only during initial pump-down and final venting).
> - **Co DC current (`PSU_DC4001A_I`):** locked at $50.00 \pm 0.00\,\mathrm{mA}$ across all $17,171$ rows.
> - **Pulsed-DC channel (`PSU_G3F_C_I`):** $48.99 \pm 7.02\,\mathrm{mA}$ across $16,827$ active rows (peak $50.10\,\mathrm{mA}$ at `11:57:07`). Note that the `.tsv` column header labels the current unit as `A`, whereas the logged numbers (50) are in mA (matching `currentCo` $= 0.05\,\mathrm{A}$ in `Deposition_2026-10-14_11-56-09.depo.json`).

> **Tip**
>
> Always ask your agent to call `list_file_columns` and `get_file_columns` with `mode: "stats"` or `"downsample"` first. Request `mode: "raw"` over a narrow `row_range` only when inspecting a specific event such as an arc or pressure spike.

Running the three project-wide health checks

To check an entire project or item subtree in one pass, have your assistant run these three read-only tools (Figure 18-1 and Table 18-1):

1. **`check_storage`:** give `project_id`, `item_id` (which checks the item and all of its descendants), or `file_ids`. The server checks that every attachment's physical file exists in storage and compares its MD5 hash against the hash recorded at upload time. It returns counts of `ok`, `missing`, `content_mismatch`, `no_hash`, and `error`.
2. **`check_value_drift`:** give `project_id` or `item_id`. The server re-runs the current reader plugin on the source file of every `Reader` value (replaying any saved `.fibps` pixel regions or `.zrw` cycle interval from `analysis_snapshot`). It also recalculates every `Derived` value recursively from its inputs (Chapter 9: Table 9-1). In Study A (`261015A`), it returns `{"checked": 24, "unchanged": 24, "drifted": []}`; in Study B (`261014B`), it returns `{"checked": 22, "unchanged": 22, "drifted": []}`.
3. **`report_value_issues`:** give `project_id` or `item_id`. Scans all stored values against their value type's plausible range and tolerance factor ($1000\times$), flagging `below_range`, `above_range`, `tolerance_exceeds_value`, and `tolerance_scale_mismatch`.

   You: Before I sign sample 261015A, check that all files and values are intact and nothing has drifted.

   Assistant: Sample `261015A` (`Deposition 261015A`, `eln://item/1`, and all five child items) is ready to sign:

1. **Files (`check_storage`):** all **11 attachments are `ok`** (0 missing, 0 MD5 mismatches across `.depo.json`, `.run.json`, `.tsv`, `BN-DLC(261015A).mit`, `BN-DLC.zrw`, and the six `.fibps` scans).
2. **Value drift (`check_value_drift`):** all **24 values are `unchanged`** (0 drifted). Re-running `aps.deposition` v1 (12 values), `aps.fibps` v3 (6 values, including the saved pixel rectangles on `1.fibps` and `2.fibps`), `aps.mit` v2 (2 values), `aps.zrw` v2 (3 values), and the `Derived` formula `wear_rate_reciprocating` v1 reproduced every stored number.
3. **Ranges and tolerances (`report_value_issues`): 0 issues** found (`tolerance_factor = 1000`).

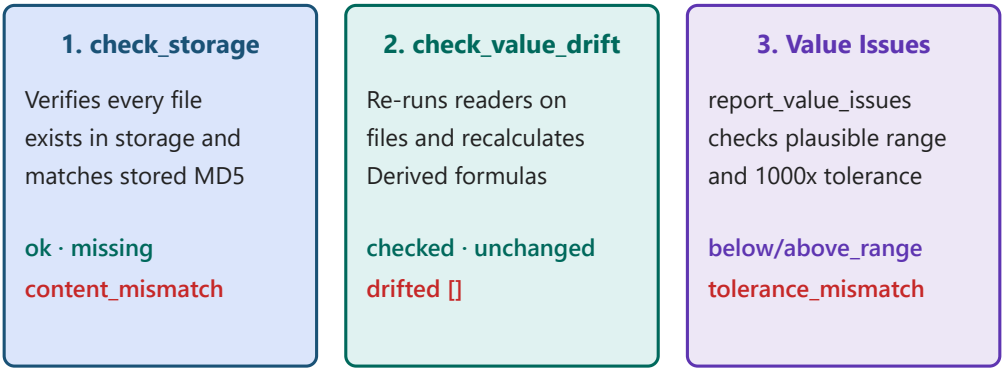


Figure 18-1: The three read-only MCP audit tools. Running $\text{check}_{\text{storage}}$ (1), $\text{check}_{\text{valuedrift}}$ (2), and $\text{report}_{\text{valueissues}}$ (3) verifies every file hash, reader extraction, derived formula, and unit range in a project.

Table 18-1: Read-only MCP inspection and audit tools.

MCP TOOL	KEY ARGUMENTS	WHAT IT RETURNS
`get_item` / `get_values`	`item_id`	Item metadata, children, files, and values with `ancestor_ids` and full `source` provenance.

MCP TOOL	KEY ARGUMENTS	WHAT IT RETURNS
`list_series` / `get_series_table`	`project_id` / `series_id`, `include_failed`	Experiment series list and full series table (`plan`, column `roles`, row `states`, `cell_marks`, and value hits).
`search_semantic`	`query`, `project_id`, `folder_id`, `item_types`, `values`	Hybrid semantic + keyword matches across records and document text, with pre-filters and top-20 `reranked` order.
`list_file_columns` / `get_file_columns`	`file_id`, `columns`, `mode` (`stats`, `downsample`, `raw`)	Column metadata and summary stats or min/max-binned series from tabular logs (`.tsv`, `.log`, `.dat`).
`check_storage`	`project_id`, `item_id`, or `file_ids`	MD5 and presence check for stored attachments (`ok`, `missing`, `content_mismatch`, `no_hash`, `error`).
`check_value_drift`	`project_id` or `item_id`	Re-runs readers and `Derived` formulas; returns `checked`, `unchanged`, and any `drifted` rows with status codes.
`report_value_issues`	`project_id` or `item_id`	Lists stored values failing `below_range`, `above_range`, `tolerance_exceeds_value`, or `tolerance_scale_mismatch`.

> ⚠ **Warning: scope check_value_drift and check_storage to item_id in legacy projects**
>
> If you add a new sample to an older project that contains historical records from before file hashes or reader tracking were enforced, `project_id` checks will include those old items. Pass `item_id` (your new root **Procedure** or **Action**) to verify your new subtree cleanly.

Sources

- **APS ELN Community Help:** `MCP.html` (`search_semantic` indexing and filters `folder_id`, `item_types`, `levels`, `action_type_id`, `date_field`, `date_from`, `date_to`, and `values` with `requested_unit` and `filters_applied`, `list_file_columns` and `get_file_columns` modes `stats`, `downsample`, and `raw`, `get_values` and `query_values` with `ancestor_ids`, `list_series` and `get_series_table`, and `check_value_drift`, `check_storage`, and `report_value_issues`).
- **APS ELN Commercial ("For Teams") Help:** `AIIntegration.html` (navigation, value, file, metadata, and check tools).
- **Zenodo demonstration deposit (`doi:10.5281/zenodo.23466959`):** `tribo_study/reports/record_report.md` and `xrr_study/reports/record_report.md` (verbatim outputs of `check_value_drift`, `check_storage`, and `report_value_issues`, and why Part 3 of Study B scopes `check_value_drift` and `check_storage` to `item_id = 12`).

Cross-references: Chapter 9: Checking value sources and understanding drift codes; Chapter 12: Reading the Series Table and setting column roles; Chapter 14: Searching by meaning and reranking; Chapter 15: Plotting conditions against results; Chapter 17: Connecting an AI Agent to Your Notebook; Chapter 19: Study B: XRR Fitting with Provenance.

Study B: XRR Fitting with Provenance

IN THIS CHAPTER

- Understanding why agent-written records must store Reader values instead of prose numbers
- Building the Study B (261014B Co/C mirror) tree and extracting 18 deposition values
- Staging the .xrdml scan with $\text{download}_{\text{file}}$ and fitting it in X-Ray Calc 3
- Uploading the accepted .xrcx fit and storing its Reader values with zero drift

In Chapter 4, you recorded **Study A (`261015A`)** by hand in the Desktop window. Here, an AI agent records and fits **Study B (`261014B`)**, a 20-period cobalt/carbon (Co/C) multilayer X-ray mirror on soda-lime glass. It uses `ELNCommunityMCP.exe` and the X-Ray Calc 3 MCP server (`XRC_MCP.exe`). Every raw file and call log for this walkthrough is in the book's demonstration bundle (`/MasteringELN/data/study-b/`).

Why agent-written records need the Value Route

If you ask an AI agent to fit an X-ray reflectivity curve without strict rules, it usually does this:

- It attaches the input files, runs the fit, and writes a Markdown note naming the source file, the job ID, and the fitted thicknesses.
- It leaves **0 test result values** on the action. Because the fitted period, layer thicknesses, and χ^2 sit inside a text note instead of `Reader`-sourced **Values**, **Tools > Analysis...** (Chapter 15) and `check_value_drift` (Chapter 18) cannot plot or check any of them.

To prevent this, Study B enforces a strict **Value Route**: the agent is forbidden to type numbers by hand (`Manual` source) or leave results in prose only. Every stored number must come from calling `extract_file_key_values` on an attached file and passing the extracted fields verbatim to `add_values` with `source_kind: "Reader"` (Figure 19-1).

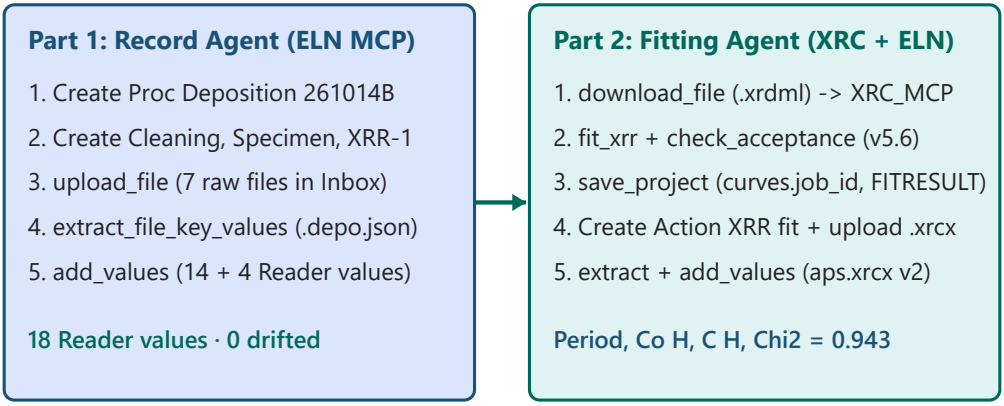


Figure 19-1: The two-agent workflow for Study B (sample 261014B, Co/C multilayer on glass). The Record Agent (left) builds the deposition and measurement tree and extracts 18 Reader values. The Fitting Agent (right) fits the .xrdml curve in X-Ray Calc 3, uploads the .xrcx fit file, and extracts 4 Reader values.

Part 1: The Record Agent builds Study B (261014B)

In Part 1 of Study B (`record_agent.md`), the Inbox (`--upload-root`) holds 7 raw files from the sputtering system and the X-ray diffractometer (Table 19-1). The Record Agent executes these steps through `ELNCommunityMCP.exe`:

1. **Verify catalog IDs:** Calls `list_file_types` (verifying `.json` is `19`, `.tsv` is `38`, and `.xrdml` is `10`) and `list_value_types`.
2. **Create the container and item tree:** Calls `create_project` (`"Study B: Co/C multilayer (demonstration)"`), `create_folder` (`"261014B"`), and `create_item` four times to build:
   - Item `1` (`Proc`, `Synthesis`): `Deposition 261014B`
   - Item `2` (`Action`, `Treatment`, parent `1`): `Cleaning 261014B`

    - Item 3 (`Item`, `Specimen`, parent 1): `261014B` (description: `"Co/C multilayer, N = 20, on glass"`)
    - Item 4 (`Action`, `Measurement`, parent 3): `XRR-1 261014B`
3. **Upload the 7 raw files:** Calls `upload_file` to attach the 3 deposition files to Item 1, the 3 plasma-cleaning files to Item 2, and `XRR-1_Co-C(261014B).xrdml` to Item 4.
4. **Extract and store the 18 `Reader` values (`aps.deposition` v1):** Calls `extract_file_key_values` on `Deposition_2026-10-14_11-56-09.depo.json` (`file_id = 1`) and `Cleaning_(261014B)_2026-10-14_11-16-41.depo.json` (`file_id = 4`). Following the **Value Route**, it selects every key that has `is_key: true` and a mapped `value_type_id`, and calls `add_values` once per item:
    - **14 values on `Deposition 261014B`:** 12 conditions (`Deposition Conditions.P` $= 0.267\,\mathrm{Pa}$ converted from $2\,\mathrm{mTorr}$, `Pbase` $= 0\,\mathrm{Pa}$, `ArFlow` $= 20\,\mathrm{sccm}$, `N` $= 20$, `Layer Co.currentCo` $= 0.05\,\mathrm{A}$ from $50\,\mathrm{mA}$, `freqCo` $= 40000\,\mathrm{Hz}$, `dutyCo` $= 70\%$, `timeCo` $= 42\,\mathrm{s}$, `Layer C.currentC` $= 0.3\,\mathrm{A}$ from $300\,\mathrm{mA}$, `freqC` $= 50000\,\mathrm{Hz}$, `dutyC` $= 60\%$, `timeC` $= 108\,\mathrm{s}$). It also stores 2 results (`Duration` $= 3626.42\,\mathrm{s}$, `BasePressure` $= 5.10 \times 10^{-5}\,\mathrm{Pa}$).
    - **4 values on `Cleaning 261014B`:** `Parameters.CleaningTime` $= 60\,\mathrm{s}$, `Parameters.P` $= 0.200\,\mathrm{Pa}$, `Duration` $= 138.71\,\mathrm{s}$, and `BasePressure` $= 1.14 \times 10^{-4}\,\mathrm{Pa}$.
5. **Run the three audit checks:** Calls `check_value_drift` (`18 checked, 18 unchanged`), `check_storage` (`7 ok`), and `report_value_issues` (`count: 0`).

> **Remember**
>
> Three domain facts in Study B show why you should always extract from `.depo.json` with `is_key: true` rather than guessing from logs. First, `timeCo` ($42\,\mathrm{s}$) and `timeC` ($108\,\mathrm{s}$) are sputtering times rather than layer thicknesses. Second, the `Mean.*` keys in `.depo.json` average over the entire run window including pump-down, so they are not deposition setpoints. Third, the `.tsv` units header labels current

columns as `A` when the numbers are in `mA`. Attach the `.tsv` for time-series plots, and extract values only from `.depo.json`.

> **In a Team Lab: practicing Study B in your Practice group on the laboratory server**
>
> When you complete Study B for your laboratory practice assignment, first create the project `Study B Practice` inside your personal `Practice` group in the Desktop window (**Project > New Group** -> `Practice`, then right-click `Practice` -> **New Project...** -> `Study B Practice`). Unpack the 7 raw files from `Study-B-261014B-raw.zip` into your `--upload-root` folder. Next, instruct your AI agent (`ELNMCPServer.exe` and `XRC_MCP.exe`) to use your existing `Study B Practice` project and build folder `261014B` and the four items of Part 1. Finally, have it fit `XRR-1_Co-C(261014B).xrdml` in `XRC_MCP.exe` (Part 2) and upload the accepted `.xrcx` file and its 4 `Reader` values under `XRR fit 261014B`.

Part 2: Fitting Agent records `XRR fit 261014B`

Once the raw scan `XRR-1_Co-C(261014B).xrdml` (`file_id = 7`) is in the notebook, you give the Fitting Agent (`fitting_agent.md`) this prompt to fit the curve and record the accepted fit back into APS ELN (Figure 19-1 and Table 19-1):

> **You:** Fit the XRR curve on `XRR-1 261014B` (`XRR-1_Co-C(261014B).xrdml`, Co/C multilayer with $N = 20$ on soda-lime glass) using the xrr-fitting skill and `csmic-empyrean` preset. Once a candidate passes `check_acceptance`, save `XRR-fit_Co-C(261014B).xrcx` from `curves.job_id` (so `[FITRESULT]` and χ^2 are stored), create child Computing action `XRR fit 261014B` under `XRR-1 261014B`, attach the `.xrcx` file, extract and store `Period`, `Co Thickness`, `C Thickness`, and `Chi2` via the Value Route (`source_kind: "Reader"`), and run the storage, drift, and value-issue checks.

The Fitting Agent executes these six steps:

1. **Stage the `.xrdml` file out of ELN (`download_file`):** When an attachment exists only inside ELN storage, an agent calls `download_file` with `file_id` and a `dest_path` inside the allowed `--upload-root`. The server verifies project read access, copies the bytes out, checks the MD5 hash against `stored_hash`, and returns `storage_status: "ok"` without altering notebook state.
2. **Assess and fit the curve in X-Ray Calc 3 (`XRC_MCP.exe`):** Following the 13-step laboratory fitting procedure in the xrr-fitting skill (*Mastering XRR Fitting*), the agent calls `assess_xrr`. It then builds the starting model (`SodaLimeGlass` substrate / $[\mathrm{C/Co}] \times 20$ / surface C contamination layer). It scans resolution ($\Delta\theta = 0.014°$) and runs 5-seed Periodic and Polynomial order 1 fits (`fit_xrr`, `job_wait`, `job_result`, verified by `check_acceptance.py`). Because the period drift across the 20 periods ($\Delta D = -0.336\ \text{Å}$) exceeds twice the 5-seed spread ($0.064\ \text{Å}$), the Polynomial order 1 fit (`fit-20261016-153456-ea29`, $\chi^2 = 0.943$) is accepted.
3. **Save `.xrcx` with `curves.job_id` so `[FITRESULT]` is written:** The agent calls `save_project` passing `curves.job_id = "fit-20261016-153456-ea29"` and *without* a manual `structure` override, writing `XRR-fit_Co-C(261014B).xrcx` (MD5 `106D0EFC42C3B0A200FFD9BE087BB1A5`) with `"fit_record": "FITRESULT"`.
4. **Create `XRR fit 261014B` and upload the `.xrcx` file:** Back in `ELNCommunityMCP.exe`, the agent calls `create_item` to create child action `XRR fit 261014B` (`Item 5`, `level: "Action"`, `item_type: "Computing"`, `parent_id: 4` under `XRR-1 261014B`). It then calls `upload_file` (`file_type_id: 1`) to attach `XRR-fit_Co-C(261014B).xrcx` (`file_id = 8`).
5. **Extract and store the 4 fit values (`aps.xrcx` v2):** The agent calls `extract_file_key_values` on `file_id = 8` and calls `add_values` on `XRR fit 261014B` to store the four `Reader`-sourced results (Table 19-1).
6. **Verify zero drift and wait for human acceptance:** Finally, the agent runs `check_value_drift` (`22 checked, 22 unchanged`), `check_storage` (`8 ok`), and `report_value_issues` (`count: 0`). It then

stops for the human owner to inspect and accept the fit in the Desktop window (Figure 19-2). Once accepted, it copies the fit into the production laboratory notebook (`Item 12` under `XRD` in Project 3 `Co/C`).

Assistant: Fitted `XRR-1_Co-C(261014B).xrdml` and recorded child action `XRR fit 261014B` (`eln://item/5` under `XRR-1 261014B`):

- **Accepted XRR fit:** Polynomial order 1 job `fit-20261016-153456-ea29` ($\chi^2 = 0.943$, `check_acceptance` verdict `accepted`; period drift $\Delta D = -0.336$ Å across the 20 periods vs. 5-seed spread 0.064 Å). Saved as `XRR-fit_Co-C(261014B).xrcx` (`file_id = 8`, MD5 `106D0EFC42C3B0A200FFD9BE087BB1A5`, `"fit_record": "FITRESULT"`).
- **Stored `Reader` values (`aps.xrcx` v2, `file_id = 8`):** **Period** $= 4.940 \times 10^{-9}$ m (49.40 Å), **Co Thickness** $= 2.083 \times 10^{-9}$ m (20.83 Å), **C Thickness** $= 2.857 \times 10^{-9}$ m (28.57 Å), and **Chi2** $= 0.943$.
- **Audit checks:** `check_value_drift` (`22 checked, 22 unchanged`), `check_storage` (`8 / 8 ok`), and `report_value_issues` (`0 issues`). Ready for your inspection in Desktop before copying to Project 3 (`Co/C`).

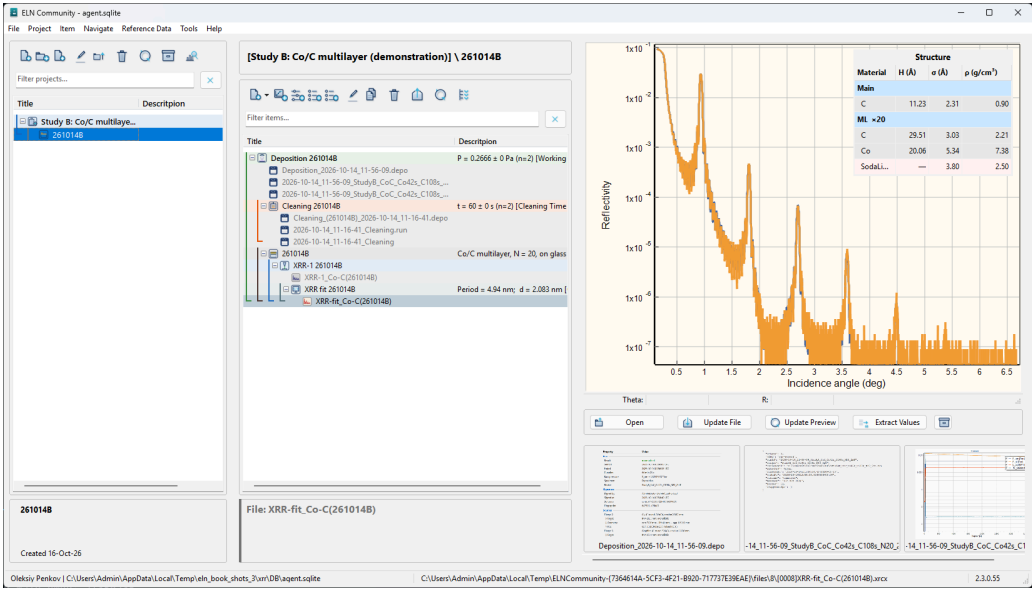


Figure 19-2: The completed Study B (261014B) notebook in APS ELN Community after Part 2. Selecting XRR-fit$_{Co}$-C(261014B).xrcx under child action XRR fit 261014B displays the fitted reflectivity curve and layer structure alongside the extracted Period, Co Thickness, C Thickness, and Chi2 values.

Table 19-1: The items, files, and extracted fit values of Study B (`261014B`).

ITEM ID & TITLE	PARENT	ATTACHED FILE(S)	STORED READER VALUES
1 **Proc:** `Deposition 261014B`	Folder `261014B`	`.depo.json`, `.run.json`, `.tsv` (3 files)	12 conditions + 2 results (`aps.deposition` v1)
2 **Action:** `Cleaning 261014B`	`Deposition 261014B`	`.depo.json`, `.run.json`, `.tsv` (3 files)	2 conditions + 2 results (`aps.deposition` v1)
3 **Specimen:** `261014B`	`Deposition 261014B`	None (`Co/C multilayer, N = 20, on glass`)	0

ITEM ID & TITLE	PARENT	ATTACHED FILE(S)	STORED READER VALUES
4 **Action:** `XRR-1 261014B`	`261014B`	`XRR-1_Co-C(261014B).xrdml` (1 file)	0 (raw reflectivity scan)
5 **Action:** `XRR fit 261014B`	`XRR-1 261014B`	`XRR-fit_Co-C(261014B).xrcx` (1 file)	`Period` = 4.940 × 10^{-9} m (49.40 Å), `Co Thickness` = 2.083 × 10^{-9} m (20.83 Å), `C Thickness` = 2.857 × 10^{-9} m (28.57 Å), `Chi2` = 0.943 (`aps.xrcx` v2)

> **Warning: why save_project must use curves.job_id without a typed structure**
>
> When `XRC_MCP.exe` saves an `.xrcx` file from a finished fit job via `curves.job_id`, it writes a `[FITRESULT]` block inside the archive that stores the fit χ^2. If an agent instead passes a manually typed `structure` JSON to `save_project`, the file is written with `[LASTFIT]` and no valid χ^2, so `aps.xrcx` cannot extract `Chi2`. Never type `Chi2` by hand (`Manual`) to hide a missing `[FITRESULT]`.

> **Going Deeper: how aps.xrcx v2 names main-stack versus surface-layer keys**
>
> An X-Ray Calc 3 model often has a thin surface contamination stack (`N = 1`, stack 1) above the main periodic stack (`[Co/C] x 20`, stack 2). In that case, the `aps.xrcx` reader (version 2) identifies the stack with the largest repeat count ($N > 1$) as the **main stack**. Its layers receive the bare key names `Co Thickness` and `C Thickness` (averaged over the 20 periods for a polynomial profile). Their sum is emitted as `Period` ($49.40\ \text{Å}$, converted by factor 10^{-10} to $4.940 \times 10^{-9}\ \text{m}$). Layers belonging to other stacks receive a stack suffix such as `C Thickness (stack 1)` ($11.23\ \text{Å}$ for the surface layer) so they can never be confused with the multilayer period layers.

Sources

- **Study B agent instructions and verification reports:** `zenodo/demonstration/xrr_study/agent_instructions/record_agent.md`, `fitting_agent.md`, and `reports/record_report.md` in the ELN data deposit (doi:10.5281/zenodo.23466959). Gives the exact MCP commands, the Value Route rules, the 5 items, 8 files, 22 `Reader` values, the X-Ray Calc 3 fit job `fit-20261016-153456-ea29`, and the production laboratory notebook step (`Item 12` in Project 3 `Co/C`).
- **XRR fitting skill (`xrr-fitting`):** github.com/OleksiyPenkov/xrr-fitting-skill (doi:10.5281/zenodo.22933081). Gives the 13-step fitting procedure, `preset.schema.json`, `validate_preset.py`, and `check_acceptance.py`.
- **ELN provenance study (`arXiv:2610.27859`):** Sections 4.5 and 5.8 (the Value Route and complete provenance in Study A and Study B).
- **MCP `download_file` specification:** `APS-ELN-Community/docs/note-download-file-mcp-tool.md` (`file_id`, `dest_path` confinement under `UploadRoot`, `overwrite`, and `storage_status`).
- **APS ELN Community Help:** `Values.html` (`aps.xrcx` version 2 main-stack averaging, `Period`, `<Material> Thickness`, stack index suffixes, and `Chi2`).

Cross-references: Chapter 1: Two running examples: Study A and Study B; Chapter 9: Reader mappings, unit factors, and standard deviations; Chapter 17: Security, upload-root, and review boundaries; Chapter 18: Running the three project-wide health checks; Chapter 20: Drafting Trees, Hypotheses, and Wikis.

Drafting Trees, Hypotheses, and Wikis

IN THIS CHAPTER

- Passing viewer selections (selection) and analysis$_{snapshot}$ in Study A (261015A)
- Computing Derived values (add$_{derivedvalue}$) and editing values safely via MCP
- Drafting Project Wiki pages (wiki$_{createpage}$) and experiment series (create$_{series}$, update$_{seriesplan}$)
- Proposing hypotheses and citing evidence via MCP while keeping approvals and series execution in the GUI

In Chapter 19, every extracted value in Study B (`261014B`) came straight from a file's default reader output. Sometimes an agent also needs to extract a step height from specific pixel rectangles on a `.fibps` map, or compute a `Derived` wear rate from three wear-track scans. You can also ask it to update a value, write a Project Wiki page, draft a Taguchi series, or prepare a hypothesis while leaving approvals to you in the GUI (Figure 20-1).

Passing viewer selections and Derived inputs

When the Record Agent built **Study A (`261015A`)** through `ELNCommunityMCP.exe` (`tribo_study/reports/record_report.md`), it had to handle two things beyond plain key extraction: viewer evaluation selections and multi-input `Derived` formulas.

First, on `Thickness 261015A` (`eln://item/3`), scan `3.fibps` had a clean automatic step, while `1.fibps` and `2.fibps` needed the standard left and right pixel rectangles (`rect_a` $= [69, 80, 208, 403]$ and `rect_b` $= [347, 80, 486, 403]$ on the 556×484 pixel map). In MCP, `extract_file_key_values` accepts an optional `selection` parameter (`rect_a`, `rect_b`, `pix_w`, and `pix_h`) for that call. When `selection` is passed,

`aps.fibps` (version 3) computes `Step Height` and `Step Height STD` from those exact rectangles and returns an `analysis_snapshot` object with `"mode": "regions"` and the rectangle coordinates. Similarly, on `BN-DLC.zrw` (`file_id = 5`), `aps.zrw` returns an `analysis_snapshot` recording the plateau cycle interval and file hash, and on `BN-DLC(261015A).mit` (`file_id = 4`), `aps.mit` returns an `analysis_snapshot` recording the 9 evaluated indents.

The agent passes that `analysis_snapshot` object **unchanged** into `add_values`. Because the snapshot is stored on the value, running `check_value_drift` later replays those exact rectangles, indents, or cycles without opening the GUI.

Second, after storing the three `Wear Area` values (`value_id = 21, 22, 23` on `Wear track 261015A`) and `Fn` (`value_id = 15`) and `Cycles` (`value_id = 16`) on `Friction test 261015A` (`eln://item/5`), the agent calls `add_derived_value` with `formula_id: "wear_rate_reciprocating"` and `inputs: {"area": [21, 22, 23], "normal_force": 15, "cycles": 16}`. The server averages the three `Wear Area` inputs, calculates $K = A/(2 \cdot F_n \cdot N) = 3.05 \times 10^{-7}\,\mathrm{mm}^3/(\mathrm{N}\cdot\mathrm{mm})$, and stores `value_id = 24` with source kind `Derived` and all five input value IDs:

> **You:** Extract the step heights from `1.fibps`, `2.fibps`, and `3.fibps` on `Thickness 261015A` (using the standard Desktop left/right pixel regions whenever automatic step detection returns no step), then extract the three wear areas on `Wear track 261015A` and calculate the derived reciprocating wear rate on `Friction test 261015A`.
>
> **Assistant:** Stored all three step heights, the three wear areas, and the derived wear rate for sample `261015A`:
>
> - **Step heights on `Thickness 261015A` (`eln://item/3`, `aps.fibps` v3):**
> - `1.fibps` (`file_id = 6`, `mode: "regions"`, rectangles $[69, 80, 208, 403]$ and $[347, 80, 486, 403]$ on 556×484 px): $3.58 \times$

$10^{-7} \pm 1.90 \times 10^{-7}\,\mathrm{m}$ ($358.3 \pm 189.7\,\mathrm{nm}$, `value_id = 18`).
    - `2.fibps` (`file_id = 7`, `mode: "regions"`, same standard rectangles): $3.16 \times 10^{-7} \pm 1.09 \times 10^{-7}\,\mathrm{m}$ ($316.0 \pm 109.3\,\mathrm{nm}$, `value_id = 19`).
    - `3.fibps` (`file_id = 8`, automatic step): $3.55 \times 10^{-7} \pm 6.71 \times 10^{-8}\,\mathrm{m}$ ($354.8 \pm 67.1\,\mathrm{nm}$, `value_id = 20`).
- **Wear areas on `Wear track 261015A` (`eln://item/6`, `aps.fibps` v3):** $0.00620 \pm 0.00013\,\mathrm{mm}^2$ (`wear 1.fibps`, `value_id = 21`), $0.00553 \pm 0.00057\,\mathrm{mm}^2$ (`wear 2.fibps`, `value_id = 22`), and $0.00621 \pm 0.00014\,\mathrm{mm}^2$ (`wear 3.fibps`, `value_id = 23`).
- **Derived wear rate on `Friction test 261015A` (`eln://item/5`):** `add_derived_value` (`wear_rate_reciprocating` v1, inputs `area: [21, 22, 23]`, `normal_force: 15`, `cycles: 16`) stored **Wear rate** $= 3.05 \times 10^{-7}\,\mathrm{mm}^3/(\mathrm{N}\cdot\mathrm{mm})$ (`value_id = 24`).

> **Remember**
>
> Never pass a `value` field to `add_derived_value`. The server calculates the number from the stored input values and refuses any call that tries to supply the result number from the client.

Updating and removing values and files safely

When you extract values interactively in the Desktop viewer (Chapter 9: Reader mappings, unit factors, and standard deviations), re-extracting from the same file updates the existing `Reader` value in place. The MCP write tools behave differently by design:

> ⚠ **Warning: add_value and add_values in MCP always insert new rows**
>
> Calling `add_value` or `add_values` through MCP always creates a new value record; it never overwrites an existing `Reader` value from the same file and key. If you need to update an existing value in place (for example, after a reader version upgrade) or remove obsolete values, use `update_value`, `remove_value`, or `remove_values` instead of calling `add_values` a second time.

APS ELN provides four dedicated MCP tools for maintaining existing values and files (Table 20-1):

- **`update_value`:** takes `value_id` and updates one or more of `value`, `tolerance`, `description`, `rec_type`, `value_type_id`, and (for a `Reader` value) `reader_version` and `analysis_snapshot`. It preserves the old fields in the audit log (`entity_type: "Value"`) and returns them in `previous`, alongside any `dependent_derived_value_ids` that use this value as an input. (The number of a `Derived` value cannot be edited with `update_value`; call `add_derived_value` to compute a new one.)
- **`remove_value` and `remove_values`:** hard-delete one value or up to 200 values in an all-or-nothing transaction. They record an optional `reason` along with the old value, tolerance, type, and source in the owning item's audit log.
- **`delete_file`:** permanently deletes an attached file and all `Reader` values extracted from it (leaving `Manual` and `Unknown` values intact). By default, `dry_run` is `true`, so your first call previews `values_to_delete` without deleting anything; to execute the deletion, pass `dry_run: false` and `confirm_hash` (the file's MD5 hash from `list_files`).

All four tools refuse to touch signed items, deleted items, or records attached to or cited by a hypothesis in `InReview` or `Accepted`.

> **You:** We re-extracted `Hardness` and `Modulus` on `Indentation 261015A` after upgrading `aps.mit` to version 2. Update the two stored values in place so their `reader_version` and 9-indent

analysis_snapshot are current without creating duplicate rows.

Assistant: Updated both values in place on Indentation 261015A (eln://item/4) via update_value (previous state preserved in the Value audit log; dependent_derived_value_ids: []):

- **Hardness (value_id = 13):** $5.92 \pm 0.47\ \mathrm{GPa}$ (Reader aps.mit v2, file_id = 4, mode: "automatic" over 9 indents).
- **Modulus (value_id = 14):** $127.17 \pm 11.06\ \mathrm{GPa}$ (Reader aps.mit v2, file_id = 4, mode: "automatic" over 9 indents).

Drafting and updating Wiki pages

You can also use an AI assistant to turn raw project data and literature notes into Project Wiki pages (Chapter 13: The five-section Project Wiki):

- **wiki_create_page:** takes project_id, section (Goals, Ideas, Literature, Analysis, or References), title, body_md, and optional attachments (paths inside --upload-root). It idempotently creates the Wiki/ and Wiki/<section>/ folders if they do not exist yet, creates the Wiki item, and writes the .md body and any attachments.
- **wiki_get_page:** downloads the page's .md body and returns it inline as UTF-8 body_md along with item metadata and non-Markdown attachment records.
- **wiki_update_page:** replaces body_md in place and optionally uploads additional attachments.

Both wiki_create_page and wiki_update_page are **all-or-nothing**. They validate every file type and upload-root path before writing; if any step fails, they roll back the new records and restore the previous .md body text automatically.

You: Create a Wiki page in the `Analysis` section of the Study A project summarizing the mechanical and tribological properties of sample 261015A with clickable links to each measurement item.

Assistant: Created Wiki page **`BN/DLC 261015A Tribology Summary`** (`eln://item/7` under `Wiki / Analysis` in `Tribo study: BN/DLC coating (demonstration)`) with live record links:

- **Coating & deposition (`[[item:1]]`, `[[specimen:261015A]]`):** 46-period $\mathrm{BN/DLC}$ multilayer on $\mathrm{Si_3N_4}$ ($P = 0.267\,\mathrm{Pa}$, bias $8\,\mathrm{V}$, top DLC layer $340\,\mathrm{s}$), step height $316\text{–}358\,\mathrm{nm}$ (`[[item:3]]`).
- **Mechanical properties (`[[item:4]]`):** $H_{\mathrm{IT}} = 5.92 \pm 0.47\,\mathrm{GPa}$ and $E_{\mathrm{IT}} = 127.17 \pm 11.06\,\mathrm{GPa}$ (`BN-DLC(261015A).mit`, 9 indents).
- **Friction & wear (`[[item:5]]`, `[[item:6]]`):** $\mu = 0.528 \pm 0.019$ at $F_n = 9.81\,\mathrm{N}$ over 1000 cycles (`BN-DLC.zrw`), mean wear area $0.00598\,\mathrm{mm^2}$, and reciprocating wear rate $K = 3.05 \times 10^{-7}\,\mathrm{mm^3/(N \cdot mm)}$.

Drafting experiment series and preparing hypotheses

Two high-level planning features in APS ELN follow the same human-in-the-loop pattern: **Experiment Series** (Chapter 12: Planning a series in the Series window) and **Hypotheses** (Chapter 11). Your AI assistant can draft the plan or prepare the evidence through MCP. Starting the series, linking depositions, confirming planned values, and closing or approving hypotheses stay with you in the Desktop GUI (Figure 20-1 and Table 20-1).

For an **Experiment Series**, MCP provides two write tools:

- **`create_series`:** takes `project_id`, `title`, and optional `purpose`, `factors` (1 to 7 objects with `value_type_id`, `label`, `range_min`, `range_max`, and `levels`), `fixed`, `responses`, `design` (`"grid"`, `"taguchi"`, or `"manual"`), `points`, and `target_folder_id`. When you pass `levels` on every factor without specifying `design`, the server uses `"grid"` for 1 or 2 factors and `"taguchi"` for 3 to 7 factors (Chapter 10). It creates the series in **`Draft`** status and returns `series_id`, `status`, `point_count`, and `taguchi_array` (such as `"L9"`). You then review the plan in **Project > Series...** and click **Start** when ready.
- **`update_series_plan`:** takes `series_id` and updates `purpose`, `factors`, `fixed`, `responses`, `target_folder_id`, `design`, or `add_points` (with an optional `reason`). While a series is `Draft`, any plan field can be updated; once you click **Start** in the Desktop GUI (`Active`), MCP can only update `purpose` and append new points. Starting a series, making or linking depositions, and confirming `planned` values are always done by a person in the Desktop GUI.

For a **Hypothesis**, MCP always uses the stored JSON status names (`Proposed`, `Approved`, `InReview`, `Accepted`, `Rejected`, `Abandoned`), while the Desktop window shows edition-specific labels (Chapter 11: Table 11-1). An agent can prepare every step between the two human decisions:

1. **Propose (`propose_hypothesis` and `update_hypothesis`):** the agent creates a root or child hypothesis (`statement` and `change`) in `Proposed` (Community: `Draft`; Team: `Proposed`).
2. **Wait for the first human decision in the GUI:** in Community, click **Start** in the **Hypotheses** window to make the draft `Open`; in Team mode, a reviewer clicks **Approve**.
3. **Attach runs and cite evidence (`attach_run` and `cite_evidence`):** once the hypothesis is `Approved` (`Open`), the agent attaches `Run` or `Reference` items and cites key items, files (`file_id`), or values (`value_id`). The server refuses a `Run` from another project (`[run_other_project]`), refuses an item outside an attached run (`[not_citable]`), and refuses an unconfirmed `Planned` value (`[planned_value]`). Valid citations snapshot the file's MD5 hash or the value's stored number and bump the hypothesis `version`.

4. **Submit the conclusion (`submit_conclusion`):** the agent calls `get_hypothesis` to read the current `version`, then calls `submit_conclusion` with the Markdown `conclusion` (including `[[item:N]]` links), `outcome` (`Supported`, `Refuted`, or `Inconclusive`), and `version`. The status becomes `InReview` (Community: `Open · conclusion ready`; Team: `In review`).
5. **Wait for the final human decision in the GUI:** inspect the cited curves and values in the **Hypotheses** window, then click **Close** (Community) or **Accept** (Team) to seal the conclusion.

   You: I approved the hypothesis on sample 261015A in Desktop. Attach `Deposition 261015A` as its experimental run, cite the indentation hardness, steady-state COF, and derived wear rate as evidence, and submit the conclusion as `Supported` for my final review.

   Assistant: Attached the run, recorded the three snapshot-hashed citations, and submitted the conclusion on hypothesis **`DLC top-layer wear protection on 46-period BN/DLC`** (`hypothesis_id = 1`, now in `InReview` / `Needs acceptance`, `version = 5`):

   - **Attached Run:** `Deposition 261015A` (`eln://item/1`, role `Run`).
   - **Cited Evidence:** `Hardness` $= 5.92 \pm 0.47\,\mathrm{GPa}$ (`value_id = 13` on `[[item:4]]`), `COF` $= 0.528 \pm 0.019$ (`value_id = 17` on `[[item:5]]`), and `Wear rate` $= 3.05 \times 10^{-7}\,\mathrm{mm^3/(N \cdot mm)}$ (`value_id = 24` on `[[item:5]]`).
   - **Submitted Conclusion (`Supported`):** Ready for you to click **Close** (Community) or **Accept** (Team) in **Project > Hypotheses**.

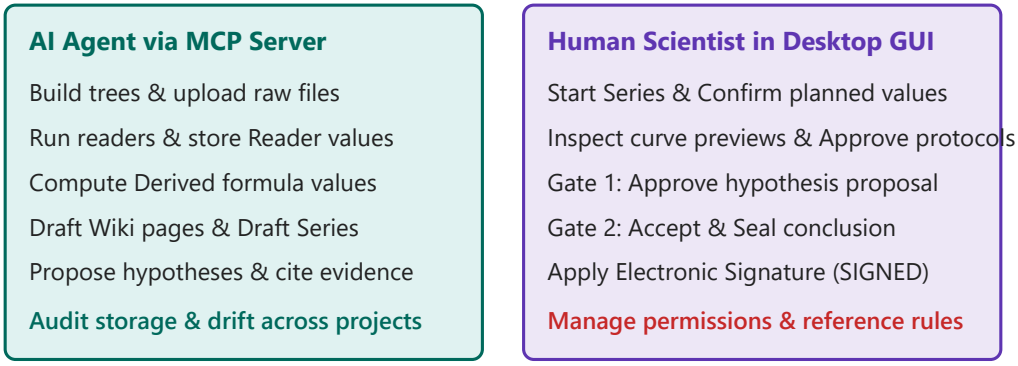


Figure 20-1: The division of labor between an AI agent and a human scientist in APS ELN. The agent handles mechanical tree creation, reader extraction, derived calculations, Wiki drafting, evidence citation, and drift audits. The human scientist owns every visual inspection, review gate, and signature.

Table 20-1: Key MCP write tools for values, files, Wiki pages, experiment series, and hypotheses.

MCP TOOL	KEY ARGUMENTS	SAFETY AND PROVENANCE BEHAVIOR
`add_value` / `add_values`	`item_id`, `value`, `value_type_id`, `source_kind`, `source_file_id`, `reader_id`, `reader_version`, `key_name`, `reader_units`, `conversion_factor`, `analysis_snapshot`	All-or-nothing batch up to 200 values; enforces plausible range, 1000× tolerance check, and same-branch source file ownership.
`add_derived_value`	`item_id`, `formula_id`, `inputs` (`role -> value_id` or `value_id[]`)	Server computes formula result (e.g. `wear_rate_reciprocating` v1) and records all input `value_ids`.

MCP TOOL	KEY ARGUMENTS	SAFETY AND PROVENANCE BEHAVIOR
`update_value` / `remove_values`	`value_id` / `value_ids[]`, `reason`	Updates in place or deletes up to 200 values; logs old values in audit trail and lists `dependent_derived_value_ids`.
`delete_file`	`file_id`, `dry_run` (default `true`), `confirm_hash`	Previews `values_to_delete` on dry run; deletes file and its `Reader` values in one transaction when confirmed.
`wiki_create_page` / `wiki_update_page`	`project_id`, `section`, `title`, `body_md`, `attachments`	Enforces the five Wiki sections (`Goals`, `Ideas`, `Literature`, `Analysis`, `References`) with atomic rollback on error.
`create_series` / `update_series_plan`	`project_id`, `title`, `factors` (with `levels`), `design` (`grid`, `taguchi`, `manual`), `fixed`, `responses`, `points` / `series_id`, `add_points`	Creates a `Draft` series (returning `point_count` and `taguchi_array`) or adds points; starting the series, linking depositions, and confirming `planned` values stay in GUI.
`propose_hypothesis` / `attach_run` / `cite_evidence` / `submit_conclusion`	`project_id`, `hypothesis_id`, `item_id`, `file_id`, `value_id`, `outcome`, `version`	Prepares proposals, runs, snapshot-hashed citations, and conclusions; optimistic `version` check prevents lost updates; human gates stay in GUI.

Sources

- **APS ELN Community Help:** `MCP.html` (`extract_file_key_values` with temporary `selection` JSON and `analysis_snapshot` for `.fibps` and `.zrw`, `add_value` and `add_values` validation, `add_derived_value` with `wear_rate_reciprocating` and multi-area array inputs, `update_value`, `remove_value`, `remove_values`, `delete_file` with `dry_run` and `confirm_hash`, `find_hypotheses_for_item`, `create_series` and `update_series_plan`, `wiki_create_page` and `wiki_update_page` atomic rollback, and write/review boundaries); `Series.html` (`Draft` vs. `Active` series workflow).
- **APS ELN Commercial ("For Teams") Help:** `AIIntegration.html` (source field validation rules, why MCP `add_value` always inserts a new row rather than updating an earlier `Reader` value, and hypothesis tool rules); `Wiki.html` (`wiki_create_page`, `wiki_get_page`, `wiki_update_page`).
- **Study A demonstration report (`doi:10.5281/zenodo.23466959`):** `tribo_study/reports/record_report.md` (how `1.fibps` and `2.fibps` use `selection` rectangles `[69,80,208,403]` and `[347,80,486,403]` on `556 x 484` pixels and how `value_id = 24` is computed from `area: [21, 22, 23]`, `normal_force: 15`, `cycles: 16`).

Cross-references: Chapter 4: Step 4: Compute the Derived wear rate; Chapter 9: The four value sources and Derived formulas; Chapter 11: The Hypothesis Tree and Evidence; Chapter 12: Experiment Series and Taguchi Arrays in ELN; Chapter 13: The five-section Project Wiki; Chapter 17: Security, upload-root, and review boundaries; Chapter 19: Study B: XRR Fitting with Provenance.

Importing Legacy Folders with an AI Agent

IN THIS CHAPTER

- Turning an instrument-first or date-first folder archive into a specimen tree
- Having an agent scan folders, deduplicate files by MD5, and draft a migration plan
- Separating Reader values extracted from raw files and Manual conditions from Word or Excel logs
- Walking through a real migration from D:\ResearchProjects\TiNiZr\ into APS ELN

In Chapter 19 and Chapter 20, your AI agent recorded new samples (`261014B` and `261015A`) right after the measurements finished. Every laboratory also has years of older work sitting on shared drives or external disks, such as `D:\ResearchProjects\`. Those folders were created before the laboratory used APS ELN. An AI agent connected through MCP (Chapter 17) can migrate a legacy folder into a structured, provenance-verified ELN project in minutes. To keep the import clean, follow a four-phase workflow that puts a human review gate before any database write (Figure 21-1).

Why legacy folders break direct import

APS ELN organizes every record around the physical sample: a **Procedure** produces a **Specimen**, each measurement is a child **Action**, the native raw file sits on that action, and **Values** point back to the file (Chapter 1: The four links in every laboratory result). Legacy drives like `D:\ResearchProjects\` grew in the opposite direction:

1. **Instrument-first folders:** A project folder such as `D:\ResearchProjects\TiNiZr\` splits measurements of the same specimen across separate subfolders. It puts nanoindentation in

`UNHT\`, 3D laser profilometry in `Wear\`, tribometry in `COF\`, scanning electron microscopy in `SEM\`, and transmission electron microscopy in `TEM #1\` through `TEM#3\`. Broader archive trees like `D:\ResearchProjects\_Experiments\` put ten years of unrelated samples under `UNHT\`, `LAXRD\`, `3DProfiler\`, `Sputtering\`, and `Tribotest\`.

2. **Date-first folders:** Inside `_Experiments\UNHT\2016-07-18\` or `TriboTester Development\2026-04-30\`, folders are named after the calendar date of the instrument session rather than the specimen code.
3. **Conditions hidden inside collaborator documents:** Deposition and annealing conditions sit in Word documents (`Specimens_List_2_(10-15).docx`, `C60+Nitrogen\specimens.docx`) or Excel workbooks (`Deposition log (Co-C) 2015-12-18.xlsx`). Collaborators often wrote these notes in Russian or Ukrainian using free-form tables and paragraphs.
4. **Duplicates, print exports, and cross-sample compilations:** Byte-identical copies appear under slightly different names (`TiNiZr_9B#1.mit` and `TiNiZr_09B#1.mit`, both $11.3\,\mathrm{MB}$) or inside backup subfolders (`Set 1\UNHT\`). In `Wear\`, native Keyence confocal height maps (`12-5N-5k.vk4`) sit next to exported print reports (`12-5N-5k.rpt`) and counterbody ball scans (`Ball-12-5N-5k.vk4`). At the folder root, OriginLab projects (`ALL_COF.opj`, `H&WR_vs_Temp.opju`) and slide decks (`compilation.pptx`) combine 20 specimens in a single file.

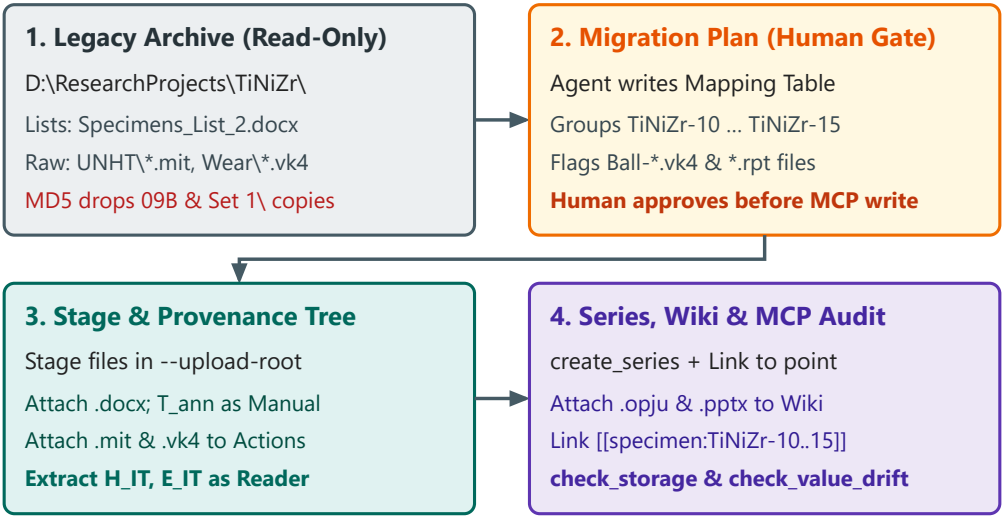


Figure 21-1: The four-phase workflow for migrating a legacy folder (`D:\ResearchProjects\TiNiZr\`) into APS ELN with an AI agent. Phases 1 and 2 are read-only on disk and stop at a mandatory human review gate before Phase 3 stages files into `--upload-root` and writes to the notebook.

If you tell an agent to "import everything in `D:\ResearchProjects\TiNiZr\`", it will create duplicate specimens for `9B` and `09B`. It will also attach 20-sample `.opju` plots to a single specimen and guess which load belongs to `12-3.vk4`. Instead, use the mapping rules in Table 21-1 across four distinct phases.

Table 21-1: Common legacy folder artifacts in `D:\ResearchProjects` and how an agent maps them to APS ELN.

LEGACY ARTIFACT IN D:\RESEARCHPROJECTS\	EXAMPLE ON DISK	WHERE IT BELONGS IN APS ELN	VALUE PROVENANCE RULE
Collaborator Word or Excel specimen list	`TiNiZr\Specimens_List_2_(10-15).docx`, `C60+Nitrogen\specimens.docx`, `Sputtering\Exeriment logs\*.xlsx`	Attach to the parent **Procedure** (`Deposition <Batch>`) so every specimen in the batch shares the source document.	Store parsed conditions (annealing temperature, substrate, ion energy) as `Manual` values whose **Description** cites the attached `.docx` or `.xlsx` `file_id`.
Native raw measurement file with a reader	`UNHT\TiNiZr_10#1.mit`, `Wear\12-5N-5k.vk4`, `COF\6-1.raw`, `LAXRD\CoC(160708).xrcx`	Attach to the specimen's child **Action** (`Indentation`, `3D Profiling`, `Tribotest`, `XRR fit`).	Run `extract_file_key_values` (`aps.mit`, `aps.vk4`, `aps.tribo`, `aps.xrcx`) and store results via `add_values` with `Reader` provenance.
Byte-identical duplicate file or backup folder	`TiNiZr_9B#1.mit` vs. `TiNiZr_09B#1.mit` (same MD5); `TiNiZr\Set 1\UNHT\` mirror	Stage and upload **one** canonical copy; list the skipped duplicate paths in the migration report.	Never upload or extract from both copies, or the item tree will average each measurement twice (`n=2`).

LEGACY ARTIFACT IN D:\RESEARCHPROJECTS\	EXAMPLE ON DISK	WHERE IT BELONGS IN APS ELN	VALUE PROVENANCE RULE
Exported instrument print report beside raw file	`Wear\12-5N-5k.rpt` (29.4 MB) sitting beside `12-5N-5k.vk4` (36.3 MB)	Skip `.rpt` when the native `.vk4` height map is present (or attach only if you need the legacy printout).	Only `.vk4` opens in the **VK4** viewer (`aps.vk4`); `.rpt` uses the fallback file browser and has no reader.
Cross-sample summary plot or slide deck	`H&WR_vs_Temp.opju`, `ALL_COF.opj`, `compilation.pptx`, `Thermal_Conductivity_#2.pdf`	Attach to a **Project Wiki** page under `Analysis` (`wiki_create_page`) that links to `[[specimen:CODE]]`.	Never attach a multi-specimen compilation under one arbitrary specimen.

Phases 1 and 2: Reconnaissance and the migration plan

The first two phases are **100% read-only**. The agent inspects the legacy folder on disk without modifying a single file in `D:\ResearchProjects\` and without calling any MCP write tool.

1. **Phase 1: Read-only inventory and MD5 deduplication.** Point your agent (DSH or OMP) at the legacy project folder, such as `D:\ResearchProjects\TiNiZr\`. Ask it to list all subfolders, compute MD5 hashes for every file, and read any `.docx`, `.xlsx`, `.md`, or `.txt` specimen lists. In `TiNiZr\`, the MD5 pass catches that `UNHT\TiNiZr_9B#1.mit` and `UNHT\TiNiZr_09B#1.mit` are byte-for-

byte identical. It also shows that `Set 1\UNHT\` and `Set 1\Wear\` are earlier folder copies of files already sitting in `UNHT\` and `Wear\`.

2. **Phase 2: The human-approved Migration Plan.** Ask the agent to match the specimen numbers in the `.docx` lists against the filenames in `UNHT\`, `Wear\`, `COF\`, `SEM\`, and `TEM #1\–TEM#3\`. Have it output a structured **Migration Plan table** before touching APS ELN.

The Migration Plan table must separate **confident matches** from **questions for you**. For example, in `D:\ResearchProjects\TiNiZr\Wear\`, sample `#12` has `12-1.vk4`, `12-3.vk4`, `12-3N-5k.vk4`, `12-5N-1k.vk4`, `12-5N-5k.vk4`, `Ball-12-3N-5k.vk4`, and `Ball-12-5N-5k.vk4`. An agent can read the load ($3\ \mathrm{N}$ vs. $5\ \mathrm{N}$) and cycle count (1000 vs. 5000 cycles) from `12-3N-5k.vk4` and `12-5N-5k.vk4`. However, it must ask you whether `Ball-12-5N-5k.vk4` (the counterbody ball scar) belongs on the same wear action as `12-5N-5k.vk4`. It must also ask which test conditions correspond to `12-1.vk4` and `12-3.vk4`:

> **You:** Scan `D:\ResearchProjects\TiNiZr\` in read-only mode. Read `Specimens_List_2_(10-15).docx`, match samples `#10` through `#15` to their raw files in `UNHT\`, `Wear\`, and `COF\`, check for duplicate files by MD5, and show me a Migration Plan table before calling any ELN write tools.
>
> **Assistant:** Scanned `D:\ResearchProjects\TiNiZr\` (read-only) and read `Specimens_List_2_(10-15).docx`.
>
> - **Batch context (`Specimens_List_2_(10-15).docx`):** Single-layer magnetron-sputtered TiNiZr coatings, nominal thickness $2.5\ \mu\mathrm{m}$, deposited under identical conditions. Samples `#13` (unannealed) and `#14` ($600\ ^\circ\mathrm{C}$) sit on $\mathrm{Al_2O_3}$ substrates ($\varnothing 15 \times 1\ \mathrm{mm}$). Samples `#10` ($500\ ^\circ\mathrm{C}$), `#11` ($550\ ^\circ\mathrm{C}$), `#12` ($750\ ^\circ\mathrm{C}$, over-annealed Laves phase + Zr solid solution), and `#15` ($200\ ^\circ\mathrm{C}$, amorphous check) sit on $\mathrm{Cr18Ni10Ti}$ (`X18H10T`) stainless steel ($2.25\ \mathrm{mm}$ thick).
> - **Duplicates and exports to skip:** `Set 1\UNHT\` and `Set 1\Wear\` duplicate the root `UNHT\` and `Wear\` folders by MD5. In `Wear\`, `12-3N-5k.rpt`, `12-5N-5k.rpt`, and `15-`

`1.rpt` are exported print reports that sit beside native `.vk4` maps (`12-3N-5k.vk4`, `12-5N-5k.vk4`).

- **Questions before I stage or upload anything:**
  1. For sample `#12`, should `Ball-12-3N-5k.vk4` and `Ball-12-5N-5k.vk4` be attached to the same `Wear track` actions as `12-3N-5k.vk4` and `12-5N-5k.vk4`?
  2. Sample `#15` has `15-2.vk4` in `Wear\` and `15-1.rpt` (with no `15-1.vk4`). Should I attach `15-2.vk4` alone or also keep `15-1.rpt` for completeness?

> ⚠ **Warning: Never let an agent write to ELN before you approve the migration plan**
>
> Calling `create_item` and `upload_file` on a legacy folder before reviewing the mapping table creates orphan specimens, duplicate `Reader` values, and misfiled scans. Always require a read-only Migration Plan table first and answer the agent's questions before Phase 3 begins.

Phase 3: Staging files and separating Reader from Manual provenance

Once you approve the Migration Plan (Table 21-2), the agent executes Phase 3.

Table 21-2: Approved migration plan for `D:\ResearchProjects\TiNiZr` Batch 2 (samples `#10`–`#15` from `Specimens_List_2_(10-15).docx`).

ELN SPECIMEN	SUBSTRATE & ANNEALING (FROM .DOCX)	MATCHED RAW FILES TO ATTACH	VALUES TO STORE IN APS ELN
`TiNiZr-10`	Cr18Ni10Ti steel (2.25 mm); annealed at 500 °C	`UNHT\TiNiZr_10#1.mit`; `TEM #2\10\`, `TEM#3\10\`	`Annealing`: *T* = 500 °C (`Manual`, cites `.docx`). `Indentation`: *H*_{IT}, *E*_{IT} (`Reader`, `aps.mit`).
`TiNiZr-11`	Cr18Ni10Ti steel (2.25 mm); annealed at 550 °C	`UNHT\TiNiZr_11#1.mit`	`Annealing`: *T* = 550 °C (`Manual`, cites `.docx`). `Indentation`: *H*_{IT}, *E*_{IT} (`Reader`, `aps.mit`).
`TiNiZr-12`	Cr18Ni10Ti steel (2.25 mm); annealed at 750 °C (Laves phase)	`UNHT\TiNiZr_12#1.mit`; `Wear\12-3N-5k.vk4`, `Ball-12-3N-5k.vk4`, `12-5N-1k.vk4`, `12-5N-5k.vk4`, `Ball-12-5N-5k.vk4`, `12-1.vk4`, `12-3.vk4`	`Annealing`: *T* = 750 °C (`Manual`, cites `.docx`). `Indentation`: *H*_{IT}, *E*_{IT} (`Reader`, `aps.mit`).

ELN SPECIMEN	SUBSTRATE & ANNEALING (FROM .DOCX)	MATCHED RAW FILES TO ATTACH	VALUES TO STORE IN APS ELN
`TiNiZr-13`	Al₂O₃ (Ø15 × 1 mm); as-deposited (unannealed)	`Wear\13-SS ball.vk4`, `13-SS ball-3k-1N.vk4`	Substrate recorded in Specimen properties; `.vk4` maps attached to `Wear track` actions (`aps.vk4`).
`TiNiZr-14`	Al₂O₃ (Ø15 × 1 mm); annealed at 600 °C	`Wear\14-SS ball.vk4`, `14-SS ball-3k-1N.vk4`, `14-SiN ball.vk4`, `14-SiN ball-3k-1N.vk4`; `TEM #1\14\`	`Annealing`: *T* = 600 °C (`Manual`, cites `.docx`). `.vk4` maps attached to `SS ball` and `SiN ball` wear actions.
`TiNiZr-15`	Cr18Ni10Ti steel (2.25 mm); annealed at 200 °C (amorphous)	`UNHT\TiNiZr_15#1.mit`; `Wear\15-2.vk4`, `15-1.rpt`	`Annealing`: *T* = 200 °C (`Manual`, cites `.docx`). `Indentation`: *H*_{IT}, *E*_{IT} (`Reader`, `aps.mit`).

Phase 3 follows three strict rules so the original archive stays untouched and every stored value carries honest provenance:

1. **Stage approved files into `--upload-root` without modifying `D:\ResearchProjects\`.** As covered in Chapter 17: Security, upload-root, and review boundaries, `upload_file` only reads paths inside your configured `--upload-root` directory. Never point `--upload-root`

at the entire `D:\` drive or rename files inside `D:\ResearchProjects\`. Instead, have the agent copy only the approved, deduplicated files into a clean staging subfolder inside `--upload-root` (for example `C:\ELN-Upload\TiNiZr-Batch2\`).

2. **Attach the collaborator `.docx` or `.xlsx` log to the parent Procedure and record its conditions as `Manual`.** The agent creates project `TiNiZr`, folder `Batch 2 (Samples 10-15)`, and procedure `Deposition TiNiZr Batch 2` (Proc). It uploads `Specimens_List_2_(10-15).docx` to `Deposition TiNiZr Batch 2` (`file_id = 1`). Next, it records the nominal coating thickness ($2.5 \times 10^{-6}\,\mathrm{m}$) on `Deposition TiNiZr Batch 2` and the annealing temperature ($500\,^{\circ}\mathrm{C}$, $550\,^{\circ}\mathrm{C}$, $750\,^{\circ}\mathrm{C}$, $600\,^{\circ}\mathrm{C}$, or $200\,^{\circ}\mathrm{C}$) on each specimen's `Annealing` action. It calls `add_value` with `source_kind: "Manual"` and a **Description** that names the attached Word document: `"From attached Specimens_List_2_(10-15).docx (file_id = 1)"`.
3. **Extract raw instrument measurements through the Value Route (`Reader` provenance).** For `TiNiZr_10#1.mit`, `TiNiZr_11#1.mit`, `TiNiZr_12#1.mit`, and `TiNiZr_15#1.mit`, the agent uploads each `.mit` file to its `Indentation` action. It then runs `extract_file_key_values` (`aps.mit`) and passes the returned `Reader` provenance fields verbatim to `add_values` (Chapter 19: Why agent-written records need the Value Route).

> **Remember**
>
> Never let an agent disguise a number read from a `.docx`, `.xlsx`, or `.pdf` prose table as a `Reader` value. `Reader` provenance is reserved for numbers extracted by a registered viewer plugin via `extract_file_key_values`. Historical conditions transcribed from legacy documents must be stored as `Manual` values whose **Description** cites the attached document.

> **Tip**
>
> Many collaborator documents in `D:\ResearchProjects\` are written in Russian or Ukrainian (`Specimens_List_2_(10-15).docx`, `C60+Nitrogen\Описание_образцов50819.docx`, `Old Data (Jeju)\таблица.docx`). Always attach the original `.docx` file unchanged, and ask the agent to include an English summary in the **Procedure** or **Specimen** description so both languages are searchable.

Phase 4: Linking an Experiment Series, filing compilations, and auditing

Once the individual specimen trees are built (`Deposition TiNiZr-10` through `Deposition TiNiZr-15`), you can connect the whole batch into an **Experiment Series** (Chapter 12: Planning a series in the Series window) so APS ELN compares and plots the annealing sweep natively:

1. Ask your agent to call `create_series` (Chapter 20) on the `TiNiZr` project with factor `Temperature` (`label: "Annealing"`, `levels: [200, 500, 550, 600, 750]`), fixed condition `Thickness` (2.5×10^{-6} m), and responses `Hardness` and `Modulus`.
2. In the Desktop GUI, open **Project > Series...**, review the `Draft` series, and click **Start** to make it `Active`.
3. In the center **Items** panel, select the imported top-level procedures, right-click, and choose **Link to plan point...**. APS ELN matches each procedure to its temperature point and populates the series **Table** and **Plot** tabs automatically.

Next, handle the cross-sample analysis files sitting at the root of `D:\ResearchProjects\TiNiZr\`: `H&WR_vs_Temp.opju`, `WR.opju`, `Nanoindentation.opj`, `ALL_COF.opj`, `Wear_Rate.mcdx`, `compilation.pptx`, and `Thermal_Conductivity_#2.pdf`. Each of these files compares many specimens across annealing temperatures.

Do not attach a multi-specimen `.opju` or `.pptx` file under one arbitrary specimen. Instead, ask your agent to call `wiki_create_page` (Chapter 20:

Drafting and updating Wiki pages) to create a page under the `Analysis` section of the **Project Wiki** (for example, `title: "Legacy TiNiZr Compilations and Summary Plots"`):

- In `body_md`, the agent writes a table linking every imported specimen (`[[specimen:TiNiZr-10]]` through `[[specimen:TiNiZr-15]]`; Chapter 13) and explains what each legacy compilation file contains.
- In `attachments`, the agent passes the staged paths of `H&WR_vs_Temp.opju`, `WR.opju`, `compilation.pptx`, and `Thermal_Conductivity_#2.pdf`. In APS ELN, `.opju` files display their embedded OriginLab graph previews directly in the **OPJU** viewer, while `.docx`, `.pptx`, and `.pdf` attachments are indexed for **Search by meaning** (`search_semantic` in Chapter 14 and Chapter 18).

> **Going Deeper: Legacy .opj files versus modern .opju files**
>
> OriginLab switched from the older binary `.opj` format (`ALL_COF.opj`, `Nanoindentation.opj`) to the Unicode `.opju` format (`H&WR_vs_Temp.opju`, `WR.opju`) in Origin 2018. APS ELN's built-in **OPJU** viewer plugin renders embedded graph thumbnails from `.opju` files directly in the **Content Viewer**. Older `.opj` files are still stored with full MD5 integrity and can be opened in Origin via working drafts (Chapter 7), or re-saved as `.opju` if you want inline graph previews.

Finally, have the agent run the three project-wide MCP audit calls from Chapter 18: Running the three project-wide health checks on the newly imported `TiNiZr` project:

1. **`check_storage (project_id)`:** confirms that every staged `.docx`, `.mit`, `.vk4`, `.opju`, and `.pptx` file sits in ELN attachment storage with an MD5 hash matching upload time (`missing: 0`, `content_mismatch: 0`).
2. **`check_value_drift (project_id)`:** re-runs `aps.mit` on every imported `.mit` file and confirms `drifted_count: 0` across all extracted `Reader` values (while reporting the documented `.docx` annealing temperatures under `skipped_manual_or_unknown`).

3. **`report_value_issues (project_id)`:** checks that every converted unit (such as $2.5 \times 10^{-6}\ \mathrm{m}$ for the $2.5\ \mu\mathrm{m}$ coating thickness) and every tolerance sits inside its value type's plausible range (`issue_count: 0`).

> **In a Team Lab**
>
> When migrating a shared legacy archive into a team laboratory server via `ELNMCPServer.exe`, create the project first and use **Share...** in the Desktop client (Chapter 5: Sharing projects in a team laboratory). That way, the collaborators who made the original samples can inspect the imported tree and verify the viewer selections.

Sources

- **APS ELN Community Help:** `MCP.html` (`--upload-root` path confinement, `create_project`, `create_folder`, `create_item`, `upload_file`, `extract_file_key_values`, `add_values`, `add_value`, `create_series`, `wiki_create_page`, `check_storage`, `check_value_drift`, and `report_value_issues`); `Series.html` (**Project > Series...** and **Link to plan point...**); `FilesAndNotes.html` (the 26 viewer plugins, including **MIT** `.mit`, **VK4** `.vk4`, **Tribo** `.raw`, **OPJU** `.opju`, **OLE** `.docx/.xlsx/.pptx`, and **PDF** `.pdf`); `Values.html` (`Reader` vs. `Manual` value provenance); `Search.html` (multilingual ONNX indexing of `.docx`, `.pptx`, `.xlsx`, and `.pdf` attachments).
- **APS ELN Commercial ("For Teams") Help:** `AIIntegration.html` (source validation rules and why `Reader` source fields must come from `extract_file_key_values`); `Wiki.html` (`wiki_create_page` and `[[specimen:CODE]]` links).
- **Legacy research archive example:** `D:\ResearchProjects\TiNiZr\` (`Specimens_List_2_(10-15).docx`, `Specimens_List_3_(16-22).docx`, `Specimens_List_4_(26-28).docx`, `UNHT\*.mit`, `Wear\*.vk4` and `*.rpt`, `COF\*.raw`, `Set 1\`, `H&WR_vs_Temp.opju`, and `compilation.pptx`).

Cross-references: Chapter 1: The four links in every laboratory result; Chapter 7: How file attachment and extension routing work; Chapter 9: The

four value sources and Derived formulas; Chapter 12: Experiment Series and Taguchi Arrays in ELN; Chapter 13: The five-section Project Wiki; Chapter 17: Security, upload-root, and review boundaries; Chapter 18: Running the three project-wide health checks; Chapter 19: Why agent-written records need the Value Route; Chapter 20: Drafting and updating Wiki pages; Chapter 24: Ten Things an Agent Can and Cannot Do.

Ten Mistakes That Break Your Data Trail

IN THIS CHAPTER

- Ten real laboratory mistakes that silently break provenance
- How each mistake shows up in APS ELN
- Which chapter and check prevent each one

A notebook can look full on screen and still fail the moment a thesis examiner or journal reviewer asks you to trace a single number in your thesis back to its raw instrument curve. The ten mistakes below come directly from daily laboratory work and student onboarding for Study A (`261015A`) and Study B (`261014B`).

1. Leaving an action empty

A student creates a `Friction test` or `Thickness` action in the item tree before going to the instrument, runs the measurement, leaves the file on a USB drive, and forgets to attach it. When it is time to review the project, the action row is still an empty placeholder. Build the whole tree at the start of the experiment so every planned measurement is visible. Then run **Tools > Data Completeness...** (Chapter 16: Completeness rules and Data Quality Checks) before you finish the day so no action stays empty.

---

2. Attaching a screenshot instead of raw data

Saving a `.png` or `.jpg` screenshot of an X-Ray Calc fit window or a nanoindenter screen gives a picture that no data file reader can parse. You cannot extract `Reader` values from a screenshot, you cannot re-evaluate its cycle interval or indent selection, and **Check Value Sources...** cannot verify a single number against it. Always attach the native raw file (`.depo.json`,

`.tsv`, `.fibps`, `.mit`, `.zrw`, `.xrdml`, `.xrcx`; Chapter 7: Inside the five core instrument viewers).

3. Putting result numbers in a file name

Naming a file `BN-DLC-351.9nm.fibps` or `wear-4.3E13nm3.fibps` puts an unverified number into a filename string. If you later adjust the step-height rectangles in the **FIBPS** viewer or fix a calibration factor, the stored **Value** updates while the filename still shows the old number. Name every file after the sample code, the measurement, and the position (`1.fibps`, `wear 1.fibps`, `BN-DLC(261015A).mit`), and store results as typed **Values** (Chapter 1: Four rules that keep your data trail unbroken).

4. Typing a number that a reader can extract

When a student reads `Hardness = 5.92 GPa` off the **MIT** viewer and types `5.92` into **Item > Add Value** by hand, APS ELN records the value with source kind `Manual` instead of `Reader`. That value has no link to `BN-DLC(261015A).mit`, no `reader_version`, and no record of which indents were ticked. Use **Extract Values** (or `extract_file_key_values` + `add_values` in MCP) whenever a reader plugin supports your file (Chapter 9: Reader mappings, unit factors, and standard deviations).

5. Typing a value before attaching its file

If you type a number into the notebook right after a measurement and plan to upload the raw file later, that file often stays on the instrument PC forever. Make the order strict: create the **Action**, attach the **File** (Chapter 4: Step 2), and extract the **Values** from that attached file (Chapter 4: Step 3).

6. Hiding results inside prose notes

Writing fitted thicknesses, χ^2, or hardness averages inside a Markdown text note leaves zero result **Values** on the action. Prose notes are invisible to **Tools > Analysis...** (Chapter 15), invisible to `query_values` and `correlate_values` (Chapter 18), and invisible to drift checks. Write your narrative in notes and Wiki pages, and always store the numbers themselves as **Values**.

7. Putting a fit under the wrong sample

An analysis project file (such as `XRR-fit_Co-C(261014B).xrcx` or a `Wear track` action) belongs to the specific measurement action it analyzed. If you attach a fit of sample `261014B` under sample `261014A`, **Tools > Analysis...** will pair `261014B`'s period with `261014A`'s deposition conditions. Always create `XRR fit <Code>` as a child action of `XRR-1 <Code>` (Chapter 6: Designing a clean item hierarchy) and include the sample code in both the action title and the `.xrcx` filename.

8. Mixing angstroms, nanometers, and meters

X-ray multilayer thicknesses are quoted in ångströms ($\mathrm{Å}$), while optical profiler step heights are displayed in nanometers (nm) or micrometers ($\mu\mathrm{m}$). General `Thickness` and `Period` value types in the database store meters (m). Meanwhile, the `.tsv` sputtering log in Study B labels current columns as `A` when the numbers are `mA`. Never convert units in your head and type the result. Let **Extract Values** apply the verified reader mapping factor (`10^-10` for $\mathrm{Å} \rightarrow \mathrm{m}$, `0.001` for $\mathrm{mA} \rightarrow \mathrm{A}$), and check the conversion line in the dialog (Chapter 9).

9. Renaming storage files in Windows Explorer

Your notebook database tracks every attached file by its relative storage path and MD5 checksum. If you open the `<name> files` attachment directory in Windows Explorer and rename, move, or edit a file directly on disk, `check_storage` will report `missing` or `content_mismatch`. Always rename, replace, or move attachments inside APS ELN using **Edit**, **Reupload...**, or **Move to...** (Chapter 7: Replacing, moving, exporting, and viewing files).

10. Backing up the database without its files

Copying `notebook.sqlite` onto a backup drive without copying its `<name> files` attachment folder saves your item titles and numbers while leaving every raw curve and image behind. Worse, exported PDF reports and JSON scientific packages (`File > Export...`) do not contain raw attachment bytes either (Chapter 15: Table 15-1). Always close APS ELN and copy the `.sqlite` file and its entire attachment folder together (Chapter 2: How to back up a local notebook safely).

Sources

- **The ELN provenance study:** *Value-level data provenance in an electronic laboratory notebook used by humans and language-model agents: An experience report* (arXiv:2610.27859). Section 4.1 (empty placeholder actions and screenshots in place of raw XRR data); Section 4.2 (values entered before source files, manual typing without reader links, and unit/scale errors); Section 4.3 (misplaced analysis files); Section 4.5 (agent records with prose results and 0 result values).
- **Study A and Study B checklists and instructions:** `review/demo_A_student_checklist.md` and `zenodo/demonstration/xrr_study/agent_instructions/record_age`

`nt.md` (keeping results out of filenames, building the full tree at the start, and `.tsv` current unit label warning).

- **APS ELN Community Help:** `Notebooks.html` (backing up `.sqlite` and the attachment folder together); `FilesAndNotes.html` (never editing storage files directly on disk); `AnalysisAndReports.html` (scientific packages do not back up attachment bytes).

Cross-references: Chapter 1; Chapter 2; Chapter 4; Chapter 6; Chapter 7; Chapter 9; Chapter 15; Chapter 16; Chapter 19.

Ten Checks Before You Sign or Submit

IN THIS CHAPTER

- Ten checks to run before signing an item or submitting to your supervisor
- Verifying files, viewer intervals, provenance badges, and drift
- Ensuring a colleague can trace every number back to its raw file

Electronic signatures (`SIGNED`) and protocol or hypothesis approvals lock your record permanently (Chapter 16: Signing an item and verifying signature integrity). Complete these ten checks before you click **Sign** or ask your supervisor to review your experiment.

1. Every action has its raw files attached

Expand your **Procedure** row in the **Items** panel with **Expand all below** (Chapter 3: Reading the item tree and its row buttons). Every **Procedure** and **Action** in the tree should have its native instrument or analysis files attached beneath it, and no placeholder action should sit empty. For a deposition run using `.depo.json`, verify that its paired `.tsv` log file is attached to the same item so time-series charts work.

2. Every file opens in its built-in viewer

Click through the attached files in the **Items** panel and check the **Content Viewer** on the right (Chapter 7: Inside the five core instrument viewers). Confirm that `.depo.json` opens in the **Deposition** viewer (rather than plain JSON), `.fibps` shows its height map, and `.mit` plots the load-displacement curves. Also check that `.zrw` plots the friction curve and `.xrcx` shows both the reflectivity curves and the **Structure** table. If you edited any attachment

in an external program or the **Markdown Editor**, verify that you saved your working draft (**Tools > Save Attachment Working Draft**).

3. Shared regions and intervals are applied

In the **FIBPS** viewer, check that the step-height rectangles **A** and **B** sit on flat, clean areas of the coated and uncoated levels. For wear scans, check that the automatic **Wear Track** groove boundaries bracket the wear scar cleanly. In the **MIT** viewer, check that unphysical indents are unticked in **Records:**. In the **ZRW** viewer, check that the plateau cycle interval is set and that you clicked **Apply** so the shared selection is saved with the file (Chapter 7: Table 7-1).

4. Headline values carry Reader or Derived sources

Open each **Procedure** and **Action** with **Item > Edit** and inspect its **Values** tab (Chapter 9: The four value sources and Derived formulas). Every measurement that has a file reader should show a `Reader` source naming the reader ID, version, key, and file. Every formula output (such as reciprocating wear rate K) should show a `Derived` source naming the formula ID, version, and input values. If a value shows `Manual`, verify that it is a quantity with no file reader and that its **Description** explains how it was measured.

5. Repeated measurements show mean and SD

An action often holds multiple measurements of the same quantity, such as the three step-height scans `1.fibps–3.fibps` or the three wear-track scans `wear 1.fibps–wear 3.fibps` in Study A. Check the summary on the item tree row. Because all three values share the same Value Type, Record Type (`Result`), and Description, the tree row displays their mean, sample standard deviation, and count: for example, `d = 343.0 ± 23.5 nm (n=3)` (Chapter 3).

If `(n=6)` appears when you attached only 3 files, check for duplicate manual entries.

6. Sample codes match across items and files

Check that the sample code in the **Specimen** title (such as `261015A` or `261014B`) matches the code in the parent **Procedure** title. It should also match every child **Action** title and the attached filenames (Chapter 4: Table 4-1). Consistent sample codes allow **Navigate > Search...** (`Ctrl+F`) and `eln://specimen/<CODE>` links to locate every record of that specimen without ambiguity.

7. Required protocol forms are completed

If your procedure or action uses a structured **Protocol** (Chapter 8: Adding and completing a protocol on an item), click the protocol row and read its rendered HTML in the **Content Viewer**. If the template uses `AutoFill` fields such as `values.all`, verify that the protocol was created *after* you extracted the item's values so the summary snapshot is populated. Then move **Validation Status** from `Draft` to `In Review` when ready.

8. Data Completeness reports zero missing items

Select your project or folder and run **Tools > Data Completeness...** (or right-click the project and choose **Data Quality Check...** in the team client; Chapter 16: Completeness rules and Data Quality Checks). Confirm that no action fails a required file-type or value-type rule, and run `report_value_issues` via MCP (Chapter 18) to confirm zero out-of-range values or tolerance-scale mismatches.

9. Check Value Sources reports zero drift

Right-click the root **Procedure** of your experiment and choose **Check Value Sources...** (or run `check_value_drift` and `check_storage` via MCP; Chapter 9: Checking value sources and understanding drift codes). Confirm that every attachment's MD5 checksum is `ok` and that every `Reader` and `Derived` value reports `unchanged` with zero `value_changed`, `selection_changed`, or `input_changed` warnings.

10. Hypothesis runs and citations are current

If your experiment tests a hypothesis (Chapter 11: Attaching Runs, promoting References, and citing Evidence), open **Project > Hypotheses**, select the hypothesis, and check its **Runs** and **Evidence** tabs. Confirm that the procedure or specimen is attached as a `Run` and that the final files and values are cited on the **Evidence** tab (with any `Planned` series values confirmed first). Verify that no cited row shows a `changed` or `missing` warning before you click **Close with conclusion** (Community) or **Submit conclusion** (Team).

Sources

- **APS ELN Community Help:** `Items.html` (**Expand all below** and the **Values** tab **Go to file** button); `FilesAndNotes.html` (compound extensions and **Tools > Save Attachment Working Draft**); `Values.html` (repeated-value tree summaries, `Reader` and `Derived` sources, and **Check Value Sources...**); `Protocols.html` (`Draft` to `In Review`); `Hypotheses.html` (**Runs**, **Evidence**, and **Submit conclusion**); `Integrity.html` (**Tools > Data Completeness...** and **Signature > Sign Item...**).
- **Study A and Study B verification reports (`doi:10.5281/zenodo.23466959`):** `tribo_study/reports/record_report.md`, `tribo_study/reports/content_check.md`, and

`xrr_study/reports/record_report.md` (the exact checks run at the end of Study A and Study B: `check_value_drift`, `check_storage`, and `report_value_issues`).

Cross-references: Chapter 3; Chapter 4; Chapter 7; Chapter 8; Chapter 9; Chapter 11; Chapter 16; Chapter 18.

Ten Things an Agent Can and Cannot Do

IN THIS CHAPTER

- Five jobs an AI agent can run for you through the MCP server
- Five safety and review boundaries an agent can never cross
- How to split the work between your agent and yourself

An AI assistant connected through `ELNCommunityMCP.exe` or `ELNMCPServer.exe` (Chapter 17: What the ELN MCP server does) can save you hours of repetitive clicking. To use it safely, you need to know which jobs belong to the agent and which decisions belong to you. Items 1 through 5 below are what an agent can do for you; items 6 through 10 are what it can never do.

1. Can: Build item trees and attach raw files

Give an agent a folder of raw files inside `--upload-root` and a checklist (such as `record_agent.md` in Study A and Study B, or an approved Migration Plan table for a legacy archive in Chapter 21). It calls `create_project`, `create_folder`, `create_item`, and `upload_file` to construct the full `Procedure -> Specimen -> Action` hierarchy. It then attaches every `.depo.json`, `.tsv`, `.fibps`, `.mit`, `.vk4`, `.zrw`, `.raw`, and `.xrdml` file with verified MD5 hashes in seconds (Chapter 19: Part 1).

2. Can: Extract Reader values with snapshots

An agent calls `extract_file_key_values` and forwards the mapped fields (`converted_value`, `value_type_id`, `tolerance`, `source_file_id`, `reader_id`, `reader_version`, `key_name`, `reader_units`, `conversion_factor`, and `analysis_snapshot`) verbatim into `add_values`. In a single all-or-nothing transaction, it stores up to 200 `Reader` values (Chapter 19: Why agent-

written records need the Value Route). For `.fibps` maps, it can also pass pixel rectangle coordinates in `selection` so the step height and its `regions` snapshot are recorded reproducibly (Chapter 20).

3. Can: Compute Derived values from formulas

An agent never needs to calculate a wear rate in its own prompt and type the result as a `Manual` number. Instead, it calls `add_derived_value` with `formula_id` (such as `wear_rate_reciprocating`) and the stored input `value_ids` (including an array of multiple `Wear Area` value IDs). The ELN server computes the formula result and records the full dependency chain (Chapter 20: Passing viewer selections and Derived inputs).

4. Can: Audit storage, ranges, and drift

Three read-only calls (`check_storage`, `check_value_drift`, and `report_value_issues`) let an agent audit an entire project (Chapter 18: Running the three project-wide health checks). It checks every attachment's MD5 checksum on disk, re-runs every data file reader and `Derived` formula, and flags any value or tolerance outside its plausible range. It can also search across PDFs, Office files, and notes with `search_semantic` and summarize long `.tsv` logs with `get_file_columns`.

5. Can: Stage curves to external fitting tools

Using `download_file`, an agent exports a raw measurement file (such as `XRR-1_Co-C(261014B).xrdml`) from ELN storage into `--upload-root` and passes it to a domain-specific tool server such as `XRC_MCP.exe` (X-Ray Calc 3). After running the multi-seed fitting procedure, it uploads the resulting `.xrcx` fit project back to a child action and extracts the fitted parameters as `Reader` values (Chapter 19: Part 2).

6. Cannot: Approve or accept a hypothesis

An agent can draft a Full Grid or Taguchi experiment series (`create_series`), propose a hypothesis (`propose_hypothesis`), attach runs (`attach_run`), cite evidence (`cite_evidence`), and submit a conclusion (`submit_conclusion`). It can never start a `Draft` series, confirm `planned` values, start or approve a hypothesis proposal, discard or reject a proposal, close or accept a conclusion, or approve a protocol. Those tools do not exist in the MCP server; every scientific approval gate requires a human decision in the GUI (Chapter 11, Chapter 12, and Chapter 20).

7. Cannot: Sign an item or edit signed data

The MCP server exposes `get_signatures` to read existing electronic signatures, but has no tool to apply a signature. Signing requires a human scientist to right-click the item in the Desktop GUI and enter their password (Chapter 16: Signing an item and verifying signature integrity). Once an item is signed (`SIGNED`), the MCP server refuses every write, file replacement, value edit, or deletion on that item.

8. Cannot: Read or write outside its scope

The MCP host reloads the active user principal before every tool call and enforces the exact same project permissions as the Desktop client (Chapter 5 and Chapter 17). A missing project or item ID and an inaccessible project or item ID return the same error message, so an agent cannot even discover the existence of private projects outside its account scope.

9. Cannot: Access files outside upload-root

Both `upload_file` and `download_file` check the resolved file path against the configured `--upload-root` (`MCPUploadRoot`) before touching the file system. Any path outside that folder, any `..` parent traversal, and any

directory junction or symbolic link is rejected immediately (Chapter 17: Security, upload-root, and review boundaries).

10. Cannot: Judge physical sample quality

An agent can verify that a `.fibps` file has a valid MD5 hash, that `aps.fibps` v3 returns a step height of $3.58 \times 10^{-7}\,\mathrm{m}$, and that $\chi^2 = 0.943$ passes the XRR acceptance rules. However, it cannot see physical mistakes in the laboratory. It does not know if you dropped the steel coupon on the floor, if the masked step edge peeled during lift-off, or if the tribometer counterbody ball was rusty. Preparing the sample, inspecting the physical surface, and owning the scientific conclusion are always your job.

Sources

- **APS ELN Community Help:** `MCP.html` (the 70 registered tools, `extract_file_key_values`, `add_values`, `add_derived_value`, `check_storage`, `check_value_drift`, `report_value_issues`, `search_semantic`, `get_file_columns`, and the write/review boundaries: local principal scope, signed-item locks, `--upload-root` confinement, and no MCP tools for hypothesis approval or conclusion acceptance).
- **APS ELN Commercial ("For Teams") Help:** `AIIntegration.html` (tool categories and validation rules); `Experiments.html` (Section *Agents*: what AI agents can and cannot do on the Hypothesis Tree).
- **MCP `download_file` specification:** `APS-ELN-Community/docs/note-download-file-mcp-tool.md` (`CheckUploadPathAllowed` path confinement and junction/symlink refusal).
- **Study A and Study B demonstration deposit (`doi:10.5281/zenodo.23466959`):** `tribo_study/` and `xrr_study/` (demonstrating items 1–5 and stopping before human acceptance).

Cross-references: Chapter 5; Chapter 11; Chapter 12; Chapter 16; Chapter 17; Chapter 18; Chapter 19; Chapter 20; Chapter 21.

Toolbar, Menu, and Tree Icons Reference

IN THIS CHAPTER

- Icons and row buttons in the Projects and Items panels
- Scientific item-type icons and tree status badges
- Main menu and administration icons

This appendix lists the toolbar controls, item-type icons, tree row buttons, status badges, and menu icons displayed in the APS ELN Desktop window (Chapter 3: The three-panel workspace). Both light-theme and dark-theme variants are supplied in the runtime icon assets.

Projects and Items panel toolbars

Table A-1 lists the toolbar buttons at the top of the **Projects** panel (left) and the **Items** panel (center).

Table A-1: Toolbar buttons in the Projects and Items panels.

PANEL	BUTTON NAME	FUNCTION AND MENU EQUIVALENT
Projects	**New Project / Add Folder**	Creates a new project (**Project > New Project**) or a folder inside the selected project (**New Folder**).
Projects	**New Topic Wiki**	Seeds the five-section `Wiki/` folder hierarchy inside the selected project (**Project > New Wiki**).
Projects	**Hypotheses**	Opens the standalone Hypothesis Tree window for the selected project (**Project > Hypotheses**, Chapter 11).

PANEL	BUTTON NAME	FUNCTION AND MENU EQUIVALENT
Projects	**Series**	Opens the standalone Experiment Series window for the selected project (**Project > Series...**, Chapter 12).
Projects	**Move Up**	Moves the selected folder one level higher toward the project root (**Move Folder Up**).
Projects	**Show Archived**	Toggles display of archived projects in the tree (`Ctrl+Shift+A` in the Team client).
Items	**New**	Opens creation choices (**Procedure**, **Specimen**, **Action**, or **Wiki**) for the current selection.
Items	**Edit**	Opens the property editor for the selected item (**Item > Edit**, `F2` in the Team client).
Items	**Delete**	Moves the selected item tree to the Recycle Bin, or permanently deletes a selected file or note (**Item > Delete**).
Items	**Add File**	Opens the file browser to attach one or more files (**Item > Add File**).
Items	**Add Note**	Creates a rich-text note entry on the selected item (**Item > Add Note**).
Items	**Add Protocol**	Attaches a structured protocol form from a template (**Item > Add Protocol**).
Items	**Add Value**	Opens the value entry dialog on a procedure or action (**Item > Add Value**).
Items	**Duplicate**	Duplicates the selected item into a chosen project or folder (**Item > Duplicate**).
Items	**Export**	Exports the selected item tree as a JSON scientific package (**File > Export...**).
Items	**Refresh**	Reloads the item tree from the database (`F5` in the Team client).

PANEL	BUTTON NAME	FUNCTION AND MENU EQUIVALENT
Items	**Expand / Collapse**	Expands or collapses all branches of the current item tree.

Item type icons in the tree

Every row in the **Projects** and **Items** panels carries a color-coded icon that identifies its container or scientific item type (Table A-2).

Table A-2: Container and item-type icons in the project and item trees.

ICON NAME	HIERARCHY LEVEL	SCIENTIFIC ROLE IN THE NOTEBOOK
Project	Project container	Top-level container for a research topic, grant, or material system (Chapter 5).
Folder	Folder container	Subfolder organizing procedures and specimens inside a project.
Synthesis	`Proc` or `Action`	Material synthesis or thin-film deposition run (such as `Deposition 261015A` or `Deposition 261014B`).
Treatment	`Proc` or `Action`	Pre-treatment or post-processing step (such as plasma cleaning `Cleaning 261014B` or annealing).
Specimen	`Item` (**Specimen**)	Physical sample, coated coupon, or substrate piece produced or studied in a procedure.
Model	`Item` (**Specimen**)	Reference structure or computational model under investigation.
Characterization	`Action`	Structural, optical, or surface characterization step (such as `Thickness 261015A` or `Wear track 261015A`).

ICON NAME	HIERARCHY LEVEL	SCIENTIFIC ROLE IN THE NOTEBOOK
Measurement	`Action`	Quantitative physical measurement (such as `XRR-1 261014B`).
Testing	`Action`	Mechanical, tribological, or functional test (such as `Indentation 261015A` or `Friction test 261015A`).
Computing	`Action`	Data fitting, simulation, or numerical modeling step (such as `XRR fit 261014B`).
Wiki	`Item`	Markdown knowledge-base page inside one of the five project Wiki sections (Chapter 13).
Protocol	Attached entry	Structured protocol form attached to an item (Chapter 8).
Note / Other	Attached entry or `Item`	Rich-text note entry attached to an item, or an uncategorized general item.

Tree row buttons and status badges

Selecting a row with children in the **Projects** panel, the **Items** panel, or the **Hypotheses** list reveals three action buttons at the right end of that row. They appear alongside any status or tag badges (Table A-3).

Table A-3: Inline row buttons and tree badges.

BUTTON OR BADGE	WHERE IT APPEARS	MEANING AND BEHAVIOR
Collapse all below	Right edge of selected parent row	Collapses every nested level under the selected row.

BUTTON OR BADGE	WHERE IT APPEARS	MEANING AND BEHAVIOR
Expand all below	Right edge of selected parent row	Expands every nested level under the selected row.
Focus	Right edge of selected parent row	Tooltip: **Focus: collapse everything else**. Collapses all other branches and scrolls the selected row into view.
SIGNED badge	Beside item title in Items panel	Dark-red rounded badge indicating that the item carries a cryptographic electronic signature and is permanently locked (Chapter 16).
HYP badge	Beside project or item title	Indicates hypotheses in a project (solid when a review decision is pending in Team mode) or an item attached as a hypothesis `Run` (Chapter 11).
Tag badge(s)	Beside item title in Items panel	Colored label(s) assigned via right-click > **Tags...** (Chapter 13).

Main menu and administration icons

Table A-4 summarizes the navigation, analysis, integrity, and reference-data icons used across the main menu bar (**File**, **Project**, **Item**, **Navigate**, **Reference Data**, **Tools**, and **Help**).

Table A-4: Navigation, analysis, integrity, and reference-data menu icons.

MENU COMMAND / ICON	MENU LOCATION	PURPOSE
Dashboard	**Navigate > Dashboard** (Community) or **Edit > Dashboard** (Team)	Opens the overview of recent items, project counts, and pending protocol/hypothesis reviews.
Search...	**Navigate > Search...** (Community) or **Edit > Search...** (Team; `Ctrl+F`)	Opens the full-text and semantic (**Search by meaning**) search dialog (Chapter 14).
Analysis...	**Tools > Analysis...**	Plots procedure-level `Condition` values against child action `Result` values (Chapter 15).
Unit Converter...	**Tools > Unit Converter...**	Converts values between compatible physical units (Chapter 9).
Protocol Review...	**Tools > Protocol Review...** (`Ctrl+Shift+R`)	Opens the queue of protocols in `In Review` state awaiting approval or rejection (Chapter 8).
Data Completeness...	**Tools > Data Completeness...**	Scans a project or folder against active completeness rules (Chapter 16).
Audit Log...	**Tools > Audit Log...**	Displays timestamped `Create`, `Update`, and `Delete` events across the notebook (Chapter 16).
Recycle Bin...	**Tools > Recycle Bin...**	Restores or permanently deletes binned item subtrees (Chapter 5).

MENU COMMAND / ICON	MENU LOCATION	PURPOSE
Value Types / File Types / Equipment	**Reference Data** menu	Manages the scientific dictionaries, plausible ranges, and reader mappings (Chapter 9).

Sources

- **APS ELN Commercial ("For Teams") Help:** `IconReference.html` (Item Toolbar, Projects Panel Toolbar, Item Type Icons, and Menu Icons); `Signatures.html` (**SIGNED** badge); `Experiments.html` (**HYP** badge).
- **APS ELN Community Help:** `Items.html` and `Projects.html` (the three inline row buttons **Collapse all below**, **Expand all below**, and **Focus**).

Cross-references: Chapter 3; Chapter 5; Chapter 6; Chapter 8; Chapter 11; Chapter 12; Chapter 13; Chapter 16.

Instrument Viewers and File Formats

IN THIS CHAPTER

- All 26 built-in viewer plugins and their registered file extensions
- Data file reader IDs, versions, and extracted key names
- Shared viewer selection modes for FIBPS, MIT, and ZRW

APS ELN ships with 26 built-in **Content Viewer** plugins (Chapter 7: How file attachment and extension routing work). When a file is selected in the **Items** panel (or opened in the standalone `ELNFileViewer.exe`), the host selects the viewer matching the longest registered extension of the filename. Viewers that implement a data file reader (DFR) also extract structured numeric conditions and results via **Extract Values** (`extract_file_key_values` in MCP).

General document, image, and data viewers

Table B-1 lists the 11 general-purpose viewers for documents, images, spreadsheets, notes, protocols, and tabular text.

Table B-1: General-purpose document, media, and record viewers.

VIEWER PLUGIN	REGISTERED EXTENSIONS	RENDERING AND EDITING CAPABILITIES
PDF	`.pdf`	Renders PDF documents inline via WebView2; indexed by **Search by meaning** when PDFium is installed (up to 2,000 pages).
Image	`.png`, `.bmp`, `.jpg`, `.jpeg`, `.tif`, `.tiff`	WIC-based image display with zoom and automatic preview thumbnail generation.

VIEWER PLUGIN	REGISTERED EXTENSIONS	RENDERING AND EDITING CAPABILITIES
HTML	`.html`, `.htm`	Renders HTML documents via WebView2; also used to preview **File > Export Report...** output.
Markdown	`.md`	Renders Markdown to HTML, resolves attached images and `[[item:N]]` / `eln://` links, and opens the **Markdown Editor** (`Ctrl+S`).
Columnar Data	`.dat`, `.csv`, `.txt`, `.ascii`	Auto-detects delimiters and headers; displays tabular data and interactive charts; supports `list_file_columns` and `get_file_columns`.
JSON	`.json`	Formatted JSON viewer (files ending in `.depo.json` route to the specialist **Deposition** viewer instead).
OLE	`.xlsx`, `.docx`, `.pptx`	Embeds Microsoft Office documents via OLE when the matching Office application is installed; text is indexed by **Search by meaning**.
OriginLab (OPJU)	`.opju`	Displays embedded graph preview thumbnails from OriginLab project files.
Note	Internal rich-text entry	Displays formatted rich-text notes created with **Item > Add Note**.
Protocol	`.protocol` (internal entry)	Renders structured protocol forms created with **Item > Add Protocol** (Chapter 8).
File	Fallback (any extension)	Uses the Windows shell preview handler when no specialist plugin matches the file extension.

Deposition, recipe, and process log viewers

Table B-2 lists the five viewers dedicated to thin-film deposition recipes, run snapshots, and time-series process logs.

Table B-2: Deposition, recipe, and process log viewers.

VIEWER PLUGIN	EXTENSIONS	READER ID	DISPLAYED VIEWS AND EXTRACTED KEYS
Deposition	`.depo.json`, `.tsv`	`aps.deposition` (v1)	Displays **Overview**, **Recipe**, **Pressure**, **Temperature**, **Gas Flow**, **Bias**, **V / I**, and **Stage** tabs when `.depo.json` and its paired `.tsv` are attached to the same item. Extracts recipe conditions (`Parameters.*`, `Deposition Conditions.*`, `Layer <Mat>.*`) and run results (`Duration`, `BasePressure`) from `.depo.json`.
Recipe	`.rcpx`	`aps.recipe`	Displays read-only XML process recipes with line numbers; extracts numeric parameters as `Group.Parameter` conditions.
Process Log	`.process.log`	Built-in extractor	Displays process event logs and outcome; extracts `Process.Duration`, `Process.AlarmCount`, and `Process.Outcome` (1 complete, 2 failed, 3 aborted, 4 incomplete).
DSC	`.dsc`	`aps.dsc` (v2)	Syntax-highlighted sputtering script viewer; extracts top-level `var` declarations as `<Kind>.<Name>` conditions (with `#2` disambiguation and power-supply pulse units).

VIEWER PLUGIN	EXTENSIONS	READER ID	DISPLAYED VIEWS AND EXTRACTED KEYS
DSR	`.dsr`	None	Structured XML form viewer for sputtering deposition result files.

Characterization, tribology, and simulation viewers

Table B-3 lists the ten specialist viewers for X-ray optics, surface profilometry, nanoindentation, tribometry, electron microscopy, and molecular dynamics simulations.

Table B-3: Characterization, tribology, microscopy, and simulation viewers.

VIEWER PLUGIN	EXTENSIONS	READER ID	DISPLAYED VIEWS AND EXTRACTED KEYS
XRDML	`.xrdml`	`aps.xrdml`	Interactive chart of X-ray diffraction and reflectivity scans; reads scan metadata (`Wavelength`, `Tube Voltage`, `Tube Current`).
X-Ray Calc	`.xrcx`	`aps.xrcx` (v2)	Log-scale reflectivity chart (measured and calculated curves) plus layer **Structure** table (*H*, σ, ρ). Extracts main-stack mean `Period`, `<Material> Thickness`, `<Material> Roughness`, substrate roughness, `Chi2` (when `[FITRESULT]` is present), and calculation conditions.
XRFX	`.xrfx`	`aps.xrfx`	Multi-panel chart view of X-ray fluorescence mirror optimization results (peak positions, reflectivity, FWHM, SNR).

VIEWER PLUGIN	EXTENSIONS	READER ID	DISPLAYED VIEWS AND EXTRACTED KEYS
FIBPS	`.fibps`	`aps.fibps` (v3)	Filmetrics Profilm3D height map with **Profile**, **Distance**, **Roughness** (R_a, R_q, R_z), **Flatten**, **Step Height**, and **Wear Track** tools. Extracts optical conditions plus `Step Height` (± `Step Height STD`) or `Wear Area` (± `Wear Area STD`).
VK4	`.vk4`	`aps.vk4`	Keyence laser confocal microscope viewer showing height, intensity, and color channels with line-profile and roughness tools.
MIT	`.mit`	`aps.mit` (v2/v3)	Anton Paar UNHT nanoindentation load-displacement curves with frame-compliance subtraction and **Records:** indent checklist. Extracts mean `Hardness` (H_{IT}), `Modulus` (E_{IT}), `Reduced Modulus` (E_r), `Max Force`, `Indent Count`, and `Indents Excluded` with `STD` tolerances.
ZRW	`.zrw`, `.zrv`	`aps.zrw` (v2)	DaVinci / Tribo Tester reciprocating and rotating friction viewer (`Data.dat` and logger ZIP formats). Extracts conditions `Fn` (converted to N), `Cycles`, `Stroke`, `Speed`, and `Sampling Rate/Interval`, plus result `COF` (with `COF STD` as tolerance).
Tribo	`.raw`	`aps.tribo`	Plain-text tribometer viewer plotting Cycles against COF with test parameters in the status bar.

VIEWER PLUGIN	EXTENSIONS	READER ID	DISPLAYED VIEWS AND EXTRACTED KEYS
DM3	`.dm3`	`aps.dm3`	Gatan DigitalMicrograph TEM image viewer with calibrated pixel dimensions and scale bar metadata.
LAMMPS	`.lmpin`, `.dump`, `.data`, `.lammps`	`aps.lammps`	Molecular dynamics input script and trajectory/structure viewer with XZ cross-section controls.

Extracted keys and shared viewer selections

Three instrument readers (`aps.fibps`, `aps.mit`, and `aps.zrw`) depend on an evaluation selection saved in the notebook. That selection is recorded in each extracted value's `analysis_snapshot` (Chapter 7: Table 7-1 and Chapter 20):

- **FIBPS (`aps.fibps` v3):** When the **Step Height** tool is active or shared step rectangles exist on the file, extraction emits `Step Height` and `Step Height STD`. Its snapshot mode is `regions` (storing `rect_a`, `rect_b`, `pix_w`, and `pix_h`) or `automatic`. Otherwise, if a wear groove is detected, it emits `Wear Area` and `Wear Area STD`.
- **MIT (`aps.mit` v2/v3):** Averages `Hardness`, `Modulus`, and `Reduced Modulus` over the indents ticked in **Records:** and records the selected indent list in the value's source snapshot.
- **ZRW (`aps.zrw` v2):** Computes `COF` and `COF STD` over the cycle interval saved by clicking **Apply** (or the `Automatic` plateau if no manual interval was applied). It stores the interval bounds and file hash in `analysis_snapshot`.

Sources

- **APS ELN Community Help:** `FilesAndNotes.html` (the 26 shipped viewers and compound extension routing); `Values.html` (reader outputs and snapshots for `aps.zrw`, `aps.mit`, `aps.dsc`, and `aps.xrcx`).

- **APS ELN Commercial ("For Teams") Help:** `Plugins.html` (general-purpose plugins, instrument plugins, FIBPS tools and limits, ZRW interval rules, Recipe/Process Log/Deposition pairing, and X-Ray Calc viewer).
- **APS ELN Community README:** `APS-ELN-Community/README.md` (viewer plugin list and cleanroom provenance of the MIT, FIBPS, and VK4 readers).

Cross-references: Chapter 7; Chapter 9; Chapter 19; Chapter 20; Appendix C.

Value Sources, Units, and Formulas

IN THIS CHAPTER

- Value source kinds (Reader, Derived, Manual, Planned, Unknown) and their fields
- Standard Value Types, symbols, stored units, and reader conversion factors
- Server-calculated Derived formulas and Check Value Sources status codes

This appendix summarizes the value-level provenance fields, unit conversions, server formulas, and diagnostic status codes used when storing and auditing numeric values in APS ELN (Chapter 9 and Chapter 18).

Value source kinds and provenance fields

Every numeric value stored on a **Procedure** (`Proc`) or **Action** (`Action`) carries a record type (`EXP_CONDITION` for inputs or `TEST_RESULT` for outputs) and one of five `source_kind` values (Table C-1).

Table C-1: Value source kinds and their stored provenance fields.

SOURCE KIND	HOW IT IS CREATED	REQUIRED AND OPTIONAL PROVENANCE FIELDS
`Reader`	**Extract Values** in the GUI, or `extract_file_key_values` + `add_value` / `add_values` in MCP	Requires `source_file_id` (must belong to the item, an ancestor, or a descendant in the same project), `reader_id`, `key_name`, `reader_units`, and finite non-zero `conversion_factor`; optional `reader_version` and `analysis_snapshot` (JSON with matching `file_id` and `file_hash`).
`Derived`	Right-click item > **Add > Derived Value...**, or `add_derived_value` in MCP	Created by the server from `formula_id`, `formula_version`, and `inputs` (role -> stored `value_id` or `value_id[]`). Client-supplied result numbers are refused.
`Manual`	**Item > Add Value** in the GUI, `add_value` in MCP without `reader_id`, or hand-editing a `Reader`/`Derived` value	Optional `source_file_id` and `source_note` (up to 1,000 characters, such as a script or job identifier). Reader and conversion fields are refused on `Manual` values.
`Planned`	**Make deposition...** in **Project > Series...** (Chapter 12)	Written from a series plan point or fixed condition (`planned: true`). Shown in grey italic; omitted from series plots/exports, refused as hypothesis evidence, and refused as a `Derived` input until confirmed via **Confirm planned values** in the Desktop GUI.
`Unknown`	Legacy records created before format 5 source tracking	No source fields; cannot be re-evaluated from files by `check_value_drift`.

Standard quantities, units, and conversions

Table C-2 lists the principal quantities extracted in Study A (`261015A`) and Study B (`261014B`). For each quantity, it shows the emitted reader unit, the conversion factor applied during extraction, and the unit stored in the notebook (`ValueTypes.csv`).

Table C-2: Quantities, reader units, conversion factors, and stored units in Study A and Study B.

Quantity / Reader key	Reader ID	Emitted unit	Conversion factor	Stored unit
Sputtering pressure (`Parameters.P`, `Deposition Conditions.P`)	`aps.deposition`	`mTorr`	0.133322	Pa
Base pressure (`BasePressure`)	`aps.deposition`	`Torr`	133.322	Pa
Target current (`Layer Co.currentCo`, `Layer C.currentC`)	`aps.deposition`	`mA`	0.001	A
Pulse frequency (`Layer Co.freqCo`, `Layer C.freqC`)	`aps.deposition`	`kHz`	1000	Hz
Step height (`Step Height`)	`aps.fibps`	`um` or `m`	Automatic to target unit	m (or nm)

Quantity / Reader key	Reader ID	Emitted unit	Conversion factor	Stored unit
Wear track cross-sectional area (`Wear Area`)	`aps.fibps`	`um2` or `mm2`	10^{-6} (from μm^2)	mm^2
Indentation hardness (`Hardness`, H_{IT})	`aps.mit`	`GPa`	1	GPa
Indentation modulus (`Modulus`, E_{IT})	`aps.mit`	`GPa`	1	GPa
Normal load (`Fn`)	`aps.zrw`	`gf`, `kgf`, `mN`, or `N`	9.80665 (from kgf)	N
Coefficient of friction (`COF`)	`aps.zrw`	Dimensionless	1	Dimensionless
Multilayer period (`Period`) and layer thickness (`<Material> Thickness`)	`aps.xrcx`	`A` (Å)	10^{-10}	m
Goodness of fit (`Chi2`, χ^2)	`aps.xrcx`	Dimensionless	1	Dimensionless

Derived value formulas

Choosing **Add > Derived Value...** in the Desktop GUI or calling `add_derived_value` through MCP evaluates a versioned server formula (Table C-3).

Table C-3: Built-in Derived value formula specification.

Formula ID & Version	Equation	Input roles and required units	Output Value Type and unit
`wear_rate_reciprocating` (version 1)	$K = A / (2 \cdot F_n \cdot N)$	`area`: `Wear area` in mm^2 (accepts one value ID or an array of value IDs, using their arithmetic mean; $A \geq 0$); `normal_force`: `Normal force` in N ($F_n > 0$); `cycles`: `Cycles` without unit ($N > 0$). All inputs must sit on the target item or its descendants in the same project.	`Wear rate` (`TEST_RESULT`) in $mm^3/(N \cdot mm)$

Plausible-range and drift diagnostic codes

Table C-4 lists the diagnostic codes returned by value entry validation (`add_value`, `add_values`, `report_value_issues`), storage verification (`check_storage`), and provenance drift auditing (**Check Value Sources…** / `check_value_drift`).

Table C-4: Validation, storage, and drift status codes.

TOOL / CHECK	CODE	CONDITION THAT TRIGGERS THE CODE
`add_value` / `report_value_issues`	`below_range`, `above_range`	The stored value is below `range_min` or above `range_max` of its Value Type.
`add_value` / `report_value_issues`	`tolerance_exceeds_value`	A non-zero tolerance is strictly larger than |value|.
`add_value` / `report_value_issues`	`tolerance_scale_mismatch`	A non-zero tolerance is smaller than |value| / `tolerance_factor` (default 1000).

TOOL / CHECK	CODE	CONDITION THAT TRIGGERS THE CODE
`check_storage` / `download_file`	`ok`, `missing`, `content_mismatch`, `no_hash`, `error`	Attachment bytes match stored MD5 (`ok`), file is absent from storage (`missing`), bytes on disk differ from stored MD5 (`content_mismatch`), legacy file record has no hash (`no_hash`), or I/O error (`error`).
`check_value_drift` (`Reader`)	`value_changed`, `reader_version_changed`, `reader_changed`, `reader_units_changed`	Re-running the current reader on the stored file produces a different number, uses a newer reader version or different reader plugin, or emits a different unit.
`check_value_drift` (`Reader`)	`key_missing`, `file_missing`, `read_failed`	The source key is no longer emitted, the source file is missing, or the reader could not parse the file.
`check_value_drift` (Snapshot)	`source_changed`, `selection_changed`, `interval_unusable`	The file hash differs from `analysis_snapshot`, the shared `.fibps` regions or `.zrw` interval changed after extraction, or the stored interval has fewer than 2 samples in the file.
`check_value_drift` (`Derived`)	`input_changed`, `input_missing`, `formula_version_changed`, `formula_version_mismatch`, `dependency_cycle`, `not_fully_rerunnable`	An input value drifted or was deleted, a newer formula version exists, the saved formula version is unavailable, a cycle exists in inputs, or an input in the chain is `Manual` / `Unknown`.

Sources

- **APS ELN Community Help:** `Values.html` (units, plausible ranges, tolerance factor, reader mappings, `Reader` / `Derived` / `Manual` / `Unknown` sources, `wear_rate_reciprocating` v1, and **Check Value Sources…** codes); `MCP.html` (`add_value`, `add_values`, `add_derived_value`, `check_value_drift`, `check_storage`, and `report_value_issues`).
- **APS ELN Commercial ("For Teams") Help:** `AIIntegration.html` (source field validation limits and refusal rules).

Cross-references: Chapter 9; Chapter 12; Chapter 18; Chapter 19; Chapter 20; Appendix B.

Keyboard Shortcuts

IN THIS CHAPTER

- Global and navigation shortcuts in Community and Team editions
- Projects panel and Items panel shortcuts and multi-select keys
- Markdown Editor, Help window, and ELN File Viewer shortcuts

This appendix lists the keyboard shortcuts supported in **APS ELN Community** and **APS ELN ("For Teams")** (Chapter 3: The Dashboard and keyboard navigation). Shortcuts that operate on a specific panel require that panel to have keyboard focus and are inactive while the cursor is inside a search box or text field.

Global and main window shortcuts

Table D-1 lists the global navigation and window shortcuts. Note where **APS ELN Community** and the **Commercial ("For Teams")** edition assign different actions to `Ctrl+D` and `F5`.

Table D-1: Global and main window shortcuts.

SHORTCUT	ACTION IN APS ELN COMMUNITY	ACTION IN TEAM LAB (COMMERCIAL)
`F1`	**Help > Keyboard Shortcuts...**	**Help > Keyboard Shortcuts...**
`Ctrl+F`	**Navigate > Search...** (notebook search)	Full-text / semantic **Search** across projects
`Ctrl+G`	**Navigate > Go to Item...** (by ID or `eln://` URL)	**Go to item** by ID or `eln://` URL

SHORTCUT	ACTION IN APS ELN COMMUNITY	ACTION IN TEAM LAB (COMMERCIAL)
`Ctrl+D`	**Navigate > Dashboard**	Duplicates the selected item when the **Items** panel has focus (use **Edit > Dashboard** for the Dashboard)
`F5`	Not bound (use **Refresh** in the menu or toolbar)	Refreshes the active panel
`Ctrl+E`	Use **File > Export...** from the menu	**File > Export Package**
`Ctrl+Shift+A`	Use **Show Archived** in the Projects panel	Toggles archived projects visibility
`Ctrl+Shift+C`	Not bound	Copies the selected file path
`Ctrl+Shift+R`	Use **Tools > Protocol Review...** from the menu	Opens the **Protocol Review Queue**

Projects panel and Items panel shortcuts

Table D-2 lists the keyboard and mouse-modifier bindings for the **Projects** panel and the **Items** panel (Chapter 5 and Chapter 6).

Table D-2: Projects panel, Items panel, and multi-selection shortcuts.

SHORTCUT	REQUIRED FOCUS / SCOPE	ACTION
`Ctrl+Up` / `Ctrl+Down`	Focused root item in **Items** panel (both editions)	Moves the selected top-level root procedure up or down in manual order.

SHORTCUT	REQUIRED FOCUS / SCOPE	ACTION
`Ctrl+click`	**Items** panel (both editions)	Adds or removes the clicked row from the current multi-row selection.
`Shift+click`	**Items** panel (both editions)	Selects the contiguous range of rows from the focused row to the clicked row.
`Shift+Up` / `Shift+Down`	**Items** panel (both editions)	Extends the multi-row selection up or down by one row.
`Ins`	**Projects** or **Items** panel (Team edition)	Creates a new project (in Projects panel) or a new item/action (in Items panel).
`F2`	**Projects** or **Items** panel (Team edition)	Opens the **Edit** dialog for the selected project, folder, or single item.
`Del`	**Projects** or **Items** panel (Team edition)	Deletes the selected empty folder, or deletes all selected items/files/notes.
`Ctrl+Shift+D`	**Projects** panel (Team edition)	Creates a **New Folder** inside the selected project or folder.
`Ctrl+N`	**Items** panel (Team edition)	Creates a new item.
`Ctrl+Shift+F`	**Items** panel (Team edition)	Attaches a file (**Item > Add File**).
`Ctrl+Shift+N`	**Items** panel (Team edition)	Adds a rich-text note (**Item > Add Note**).
`Ctrl+Shift+P`	**Items** panel (Team edition)	Attaches a protocol (**Item > Add Protocol**).

SHORTCUT	REQUIRED FOCUS / SCOPE	ACTION
`Ctrl+Shift+V`	**Items** panel (Team edition)	Adds a numeric value (**Item > Add Value**).

Markdown Editor shortcuts

Table D-3 lists the shortcuts active when the **Markdown Editor** has focus (Chapter 7 and Chapter 13).

Table D-3: Markdown Editor shortcuts (both editions).

SHORTCUT	ACTION
`Ctrl+S`	Saves the edited Markdown body through the notebook attachment service (prompting on conflict).
`Ctrl+B`	Wraps the selected text in Markdown bold markers (`**...**`).
`Ctrl+I`	Wraps the selected text in Markdown italic markers (`*...*`).
`Ctrl+K`	Inserts a Markdown link (`[label](url)`) using the selected text as the label.
`Ctrl+V`	Pastes text normally; if the clipboard holds an image and no text, uploads the image as an item attachment and inserts its Markdown image reference.

Help window and ELN File Viewer shortcuts

Table D-4 lists the shortcuts in the built-in **Help** viewer window, the **FIBPS** viewer, and the standalone **ELN File Viewer** (`ELNFileViewer.exe`).

Table D-4: Help window, FIBPS viewer, and ELN File Viewer shortcuts.

WINDOW / VIEWER	SHORTCUT	ACTION
Help window	`/` or `Ctrl+F`	Moves keyboard focus to the **Search Help** box.
Help window	`Up`, `Down`, `Enter`	Selects a Help search result and navigates to its section.
Help window	`Esc`	Clears the **Search Help** box, or closes **Contents** in a narrow window.
Help window	`Alt+Left` / `Alt+Right`	Navigates back or forward between viewed Help pages.
FIBPS viewer	`Shift+click` (in **Profile** tool)	Plots a vertical line profile instead of a horizontal line profile.
FIBPS viewer	`Esc`	Cancels the active tool, turns **Flatten** off, and clears temporary chart regions.
ELN File Viewer	`Ctrl+O`	Opens a local file in `ELNFileViewer.exe`.

Sources

- **APS ELN Community Help:** `Shortcuts.html` (main window bindings `Ctrl+F`, `Ctrl+G`, `Ctrl+D`, `F1`, item-tree `Ctrl+Up`/`Ctrl+Down`, Markdown Editor `Ctrl+S`, `Ctrl+B`, `Ctrl+I`, `Ctrl+K`, `Ctrl+V`, and Help window keys); `Items.html` (`Ctrl+click`, `Shift+click`, `Shift+Up`/`Shift+Down`).
- **APS ELN Commercial ("For Teams") Help:** `Shortcuts.html` (global, Projects panel, Items panel, Help window, and `ELNFileViewer.exe` `Ctrl+O` shortcuts); `Plugins.html` (FIBPS viewer `Shift+click` and `Esc`).

Cross-references: Chapter 3; Chapter 5; Chapter 6; Chapter 7; Chapter 13; Chapter 14.

Glossary

IN THIS CHAPTER

- Definitions of APS ELN and data-provenance terms in alphabetical order
- Cross-references to the chapters where each concept is introduced

Each entry defines a term as this book uses it and links to the chapter section where it is introduced. Toolbar and tree icons are listed in Appendix A. Viewer plugins and file extensions are listed in Appendix B. Value sources, units, and formulas are listed in Appendix C.

A

Action (`Action`). A preparation, treatment, measurement, test, or analysis step in the scientific hierarchy, placed below a **Procedure**, a **Specimen**, or a parent **Action**. See Chapter 3: Reading the item tree and its row buttons; Chapter 6: Designing a clean item hierarchy.

Analysis snapshot (`analysis_snapshot`). A JSON object stored on a `Reader` value that records the exact viewer evaluation state used when the number was extracted. Examples include the pixel rectangles (`regions`) in a `.fibps` height map, the ticked indents in a `.mit` nanoindentation file, and the cycle interval and file hash in a `.zrw` friction file. See Chapter 7: Table 7-1; Chapter 20: Passing viewer selections and Derived inputs.

Audit log. The chronological table of `Create`, `Update`, and `Delete` operations across the notebook, inspected via **Tools > Audit Log…** or `get_audit_log`. See Chapter 16: Inspecting the Audit Log.

AutoFill. A binding on a protocol template field (such as `values.all`, `conditions.all`, `results.all`, or `values.<Symbol>`). It copies a snapshot of

the owning item's values or metadata into the form when the protocol is created. See Chapter 8: How protocol templates and AutoFill work.

C

Check Value Sources (`check_value_drift`). A verification command that re-runs the current data file readers on the source files of all `Reader` values in an item subtree. It also recalculates all `Derived` values recursively from their inputs and reports any drift or changed selections. See Chapter 9: Checking value sources and understanding drift codes; Chapter 18: Running the three project-wide health checks.

Completeness rules. Rules configured in **Tools > Completeness Rules...** that specify which **Required File Types** and **Required Value Types** must exist on an item for a given **Action Type** and **Equipment**. Checked via **Tools > Data Completeness...**. See Chapter 16: Completeness rules and Data Quality Checks.

Compound extension. A multi-part filename ending such as `.depo.json` or `.process.log` that routes the attachment to a specialist viewer (**Deposition** or **Process Log**) rather than a generic `.json` or `.log` viewer. See Chapter 7: How file attachment and extension routing work.

Condition (`EXP_CONDITION`). A numeric value that records an experimental input or setpoint, such as sputtering power, gas pressure, deposition time, normal load, or cycle count. Plotted on the horizontal (X) axis in **Tools > Analysis...**. See Chapter 9: Conditions, Results, units, and plausible-range checks.

Content seal (`Sealed`). A checksum computed over a closed or accepted hypothesis's statement, change description, conclusion, outcome, attached runs, and cited evidence (**Close with conclusion** in Community or **Accept** in Commercial). See Chapter 11: Concluding, closing or accepting, and checking the content seal.

D

Data file reader (DFR). The code inside a viewer plugin that extracts numeric keys and units from an attached file. Each reader is identified by `reader_id` and integer `reader_version` (such as `aps.deposition` v1, `aps.fibps` v3, `aps.mit` v2/v3, `aps.zrw` v2, or `aps.xrcx` v2). See Chapter 9: Reader mappings, unit factors, and standard deviations; Appendix B.

Derived value (`Derived`). A numeric value calculated by APS ELN from stored input values using a versioned server formula (such as `wear_rate_reciprocating` version 1), storing the `formula_id`, `formula_version`, and input `value_id` list. See Chapter 9: The four value sources and Derived formulas.

E

Electronic signature (`SIGNED`). A permanent cryptographic seal applied to an item via **Signature > Sign Item...** with a user password and meaning (`Authored`, `Reviewed`, `Approved`, or `Witnessed`). Locks the item against all further edits. See Chapter 16: Signing an item and verifying signature integrity.

Experiment template (`.elntemplate`). A reusable hierarchy blueprint managed in **Reference Data > Experiment Templates...** and applied via **Item > New from Template...** to create a complete `Procedure -> Specimen -> Action` tree in one atomic operation. See Chapter 6: Building and applying Experiment Templates.

F

Folder. A container inside a project in the left **Projects** panel that organizes procedures and specimens. Distinct from scientific items in the center **Items** panel. See Chapter 5: How Groups, Projects, and Folders fit together.

Format 6. The current local SQLite schema version used by **APS ELN Community**, supporting `Derived` value provenance, experiment series (**Project > Series...**), default reader mappings, and tribology value types. See Chapter 2: Creating a local notebook in Community.

G

Group. A top-level organizational heading in the left **Projects** panel (**Project > New Group**) that groups projects visually without holding scientific records or affecting permissions. See Chapter 5: How Groups, Projects, and Folders fit together.

H

Hypothesis. A project record (**Project > Hypotheses**, opened in its own window) that links an expectation (`Proposal`), the experimental items that test it (`Runs`), the cited files and values (`Evidence`), and the final outcome (`Conclusion`). In Community, hypotheses move `Draft -> Open -> Closed`; in Commercial, they move `Proposed -> Approved -> In review -> Accepted`. See Chapter 11: Why connect experiments to a hypothesis tree?.

M

Manual value (`Manual`). A value whose number was typed by a person in **Item > Add Value** (or edited by hand after extraction), carrying no automated reader or formula link. See Chapter 9: The four value sources and Derived formulas.

Model Context Protocol (MCP). The line-based JSON-RPC 2.0 standard over stdio used by `ELNCommunityMCP.exe` and `ELNMCPServer.exe` to expose notebook tools to AI assistants. See Chapter 17: What the ELN MCP server does.

P

Planned value (`Planned`). A condition value written onto a procedure by **Make deposition…** in an **Experiment Series** before the run is confirmed. Displayed in grey italic (`planned`), excluded from series plots and CSV exports, refused as hypothesis evidence, and refused as a `Derived` formula input until confirmed with **Confirm planned values** in the Desktop GUI. See Chapter 12: Making or linking depositions and confirming planned values.

Procedure (`Proc`). A top-level experimental record in a project or folder representing a synthesis, deposition run, or experimental campaign (such as `Deposition 261015A`). See Chapter 3: Table 3-1; Chapter 6: Designing a clean item hierarchy.

Protocol. A structured form (or free-text entry) attached to an item via **Item > Add Protocol** and reviewed through `Draft`, `In Review`, `Approved`, and `Rejected` states. See Chapter 8: Adding and completing a protocol on an item.

R

Reader value (`Reader`). A value extracted from an attached file by a data file reader, recording `source_file_id`, file MD5, `reader_id`, `reader_version`, `key_name`, `reader_units`, `conversion_factor`, and optional `analysis_snapshot`. See Chapter 9: The four value sources and Derived formulas.

Result (`TEST_RESULT`). A numeric value that records a measured observation or calculated output, such as step height, hardness, friction coefficient, wear rate, or fitted period. Averaged on the vertical (Y) axis in **Tools > Analysis…**. See Chapter 9: Conditions, Results, units, and plausible-range checks.

S

Search by meaning (`search_semantic`). The hybrid semantic search engine (`ELN.Search`), running offline on your PC in Community (`.search.sqlite`) or on the shared laboratory search server in the Team edition (`notebook.search.sqlite`). It indexes record text and attached documents (`.pdf`, `.docx`, `.pptx`, `.xlsx`, `.md`, `.html`, `.json`, `.xml`, `.csv`, `.rtf`, `.txt`) and optionally re-ranks the top 20 item groups with a cross-encoder model. See Chapter 14: Searching by meaning (Semantic Search) and reranking.

Series (`Experiment Series`). A project-level experimental plan, table, and plot window (**Project > Series...**) that defines 1 to 7 varied **Factors** (with optional labels and levels), **Point generation** (**Full grid**, **Taguchi orthogonal array** L4–L27, or **No initial points**), **Fixed conditions**, and **Responses**, and displays all linked depositions in a unified **Table** and **Plot** (including **Level means (Taguchi)** with Δ and rank) with OriginLab export. See Chapter 12: Planning a series in the Series window.

Specimen (`Item`). A physical sample, substrate piece, or model produced or studied inside a **Procedure** (such as `261015A` or `261014B`), holding child measurement and test **Actions**. See Chapter 3: Table 3-1.

T

Taguchi orthogonal array. A fractional factorial design table (L4, L8, L9, L16, L18, or L27) in **Project > Series...** where every level of each factor and every pair of levels of any two factors occur equally often, allowing 2 to 7 factors to be screened in 4 to 27 runs. See Chapter 10: Sweeps, full grids, and Taguchi orthogonal arrays.

Tolerance. A non-negative numeric field on a stored value, expressed in the stored unit of the Value Type. Automatically populated from `<Key> STD` during reader extraction. See Chapter 9: Reader mappings, unit factors, and standard deviations.

U

Upload root (`--upload-root` / `MCPUploadRoot`). The local folder to which MCP file uploads (`upload_file`) and file downloads (`download_file`) are strictly confined. See Chapter 17: Security, upload-root, and review boundaries.

V

Value Route. The laboratory rule for agent-written records requiring every stored number to come from `extract_file_key_values` (`Reader` source) or `add_derived_value` (`Derived` source) with zero hand-typed `Manual` values. See Chapter 19: Why agent-written records need the Value Route.

W

Wiki page. A Markdown-backed item of type **Wiki** stored inside one of the five standard project Wiki folders (`Goals`, `Ideas`, `Literature`, `Analysis`, `References`). See Chapter 13: The five-section Project Wiki.

Working draft. The separate local copy created when an attached file is opened in an external program or the **Markdown Editor**, saved back into notebook storage via **Tools > Save Attachment Working Draft**. See Chapter 7: Working drafts, external edits, and conflict resolution.

Sources

- **Canonical terminology:** `NOTATION.md` in this repository, compiled from the **APS ELN Community** and **APS ELN Commercial** built-in Help manuals and the ELN provenance study (arXiv:2610.27859).

Cross-references: Appendix A; Appendix B; Appendix C; Appendix D.